Knowledge-Based Systems 249 (2022) 108919

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

Instance weighted SMOTE by indirectly exploring the data distribution

Aimin Zhang, Hualong Yu *, Shanlin Zhou, Zhangjun Huan, Xibei Yang

School of Computer, Jiangsu University of Science and Technology, Zhenjiang, China

ARTICLE INFO

Article history:
Received 11 November 2021
Received in revised form 24 April 2022
Accepted 24 April 2022
Available online 4 May 2022

Dataset link: http://archive.ics.uci.edu/ml/datasets.php, https://sci2s.ugr.es/keel/datasets.php, https://github.com/Jung-Alen/IW-SMOTE

Keywords:
Class imbalance learning
Resampling
SMOTE
Data distribution information
UnderBagging

ABSTRACT

The synthetic minority oversampling technique (SMOTE) algorithm is considered a benchmark algorithm for addressing the class imbalance learning (CIL) problem. However, SMOTE fails to observe the distribution of the training data and to explore its internal structure, resulting in an unstable and non-robust classification result. Recently, more than 100 SMOTE variants have been developed to solve this problem. Most of them attempt to directly explore the prior distribution information of the training data, which may provide extremely inaccurate guidance in some classification scenarios. In this study, we present the instance weighted SMOTE (IW-SMOTE) algorithm, a more robust and universal solution for improving SMOTE by exploit distribution data indirectly. In particular, an UnderBagging-alike undersampling ensemble algorithm that uses classification and regression tree (CART) as the base classifier is first adopted to classify each training instance and acquire the corresponding confusing information. We can accurately estimate location information for each instance, including noise, borders and safety, based on the confusing information. Then, the noisy instances can be removed, and the borderline instances can be given more chances than the safe instances to be seed instances in the SMOTE procedure. Finally, the balanced instance set was used to train the CART, K nearest neighbors (KNN) and support vector machine (SVM) classifiers to verify whether the proposed algorithm is irrelevant to the specific classification model. We compare IW-SMOTE with several state-of-the-art SMOTE-based algorithms on many class imbalance data sets, and IW-SMOTE shows promising results.
© 2022 Elsevier B.V. All rights reserved.

1. Introduction

Learning from imbalanced data is an important topic in machine learning, because it has been widely applied to diagnose and classify diseases [1,2], detect software defects [3,4], analyze biological and pharmacological data [5,6], evaluate credit risk [7], predict actionable revenue change and bankruptcy [8,9], diagnose faults in industrial procedure [10,11], classify soil types [12,13], predict crash injury severity [14] and analyze crime linkages [15]. Most, existing supervised learning algorithms are built on the theory of empirical risk minimization, making class imbalance learning (CIL) a challenging task in machine learning. Therefore, traditional learning models tend to favor the majority classes while ignoring minority class performance [16,17].

In the past two decades, hundreds of CIL algorithms have been proposed to address the class-imbalance problem [18,19]. Generally, these methods can be divided into three main categories: data-level, algorithm-level, and ensemble learning. The data level also called resampling, acquires a balanced training set before modeling the classifier by either increasing the number of minority class instances [20–22], decreasing the number of majority class instances [23,24], or both [25,26]. The algorithm-level contains different types of solutions, namely, cost-sensitive learning [27,28], threshold moving strategy [29,30], kernel learning [31], and one-class learning [32]. Algorithm level methods work by modifying existing machine learning algorithms to adapt to the imbalanced data distribution. Finally, ensemble learning improves CIL accuracy and robustness by combining data- or algorithm-level algorithms with the bagging, boosting, or random forest framework [33–38]. The data-level method has two advantages over other CIL algorithms: (1) it is easier to implement and (2) it is unaffected by the classification models used. Therefore, the data-level algorithms are popular for practical applications.

The synthetic minority oversampling technique (SMOTE) [20] is a popular data-level resampling algorithms. It effectively addresses the overfitting issue caused by random oversampling and the information loss caused by random undersampling. However, SMOTE is a completely random algorithm that fails to observe the distribution of the training data and to explore its internal structure, resulting in unstable and non-robust classification results [21]. Many SMOTE variants have been proposed to improve its performance, including several important ones such as ADASYN [22], safe-level SMOTE (SL-SMOTE) [39], majority weighting minority oversampling technology (MWMOTE) [40], surrounding neighborhood-based SMOTE (SN-SMOTE) [41],

* Correspondence to: Jiangsu University of Science and Technology, No. 666, Changhui Road, Zhenjiang, Jiangsu, China.
E-mail address: yuhualong@just.edu.cn (H. Yu).

https://doi.org/10.1016/j.knosys.2022.108919
0950-7051/© 2022 Elsevier B.V. All rights reserved.

weighted kernel-based SMOTE (WK-SMOTE) [42], and SMOTE-based class-specific extreme learning machine (SMOTE-CSELM) [43]. These SMOTE variants explore the data distribution before performing the SMOTE sampling procedure, providing abundant distribution information to guide SMOTE to sampling reasonably. However, we also note that most existing SMOTE variants attempt to directly explore the data distribution based on neighborhood calculations. Particularly, the location information provided by this strategy may be extremely inaccurate because the neighborhood information can only reflect a narrow local but not entire global data distribution; thus, it is very sensitive to data distribution, resulting in worse classification results in some practical scenarios than the original SMOTE [44].

To solve the abovementioned problem, we propose instance weighted SMOTE (IW-SMOTE), a robust and universal solution for improving SMOTE in this study. IW-SMOTE adopts an indirect strategy to explore the prior distribution information, which is insensitive to distribution and more accurate than that acquired by direct exploration strategies. In particular, the indirect prior information exploration strategy is based on an UnderBagging-alike random undersampling ensemble model and its feedback information, which we call confusing information, for each training instance. According to the confusing information, we can evaluate the hard-to-learn level for each instance and further accurately evaluate their locations and significance. That means that, based on the confusing information, the noise instances can be removed, and the borderline instances can be highlighted using weighting. Additionally, although the IW-SMOTE distinguishes noise from the other instances using a hard-cutting threshold, there is no a distinct boundary between borderline and safe instances. All minority instances except noise are weighted according to their specific confusing information, and the weight is associated with the probability of the corresponding instance becoming the seed instance in the SMOTE procedure. Using IW-SMOTE, we can acquire a noise-free and borderline-strengthening balanced training set. The proposed algorithm is compared with SMOTE and several state-of-the-art SMOTE variants on 40 class imbalance datasets, and IW-SMOTE shows promising results, indicating its effectiveness and superiority.

The contributions of this study can be concluded as follows:

(1) An indirect strategy was developed to accurately and robustly explore the prior distribution of training instances and estimate their locations, regardless of the specific distribution types.
(2) A noise filtering mechanism based on accurate confusing information estimation was adopted to prevent the noise propagation phenomenon that occurs during SMOTE synthetic instance generation.
(3) A soft weighting strategy was proposed to assign the appropriate chance for each minority instance to be seed instance in the SMOTE procedure, further highlighting the borderline.

The remainder of this paper is organized as follows. Section 2 reviews related work on SMOTE. In Section 3, we present the motivation for the proposed IW-SMOTE algorithm. Section 4 describes the proposed algorithm and its components. The experimental results and corresponding analyses are presented in Section 5. Finally, Section 6 concludes the study and discusses future work.

2. Related work

As mentioned in Section 1, although there are several CIL solutions, they can be roughly categorized into three groups: data-level, algorithm-level, and ensemble learning. Compared with the algorithm-level and ensemble learning techniques, the data-level technique is easier to implement and irrelevant to the learning models; hence, it is more popular for addressing the CIL problem. The data-level technique can be further divided into undersampling and oversampling, where the former balances the data by removing some majority class instances, and the latter produces a balanced dataset by adding minority class instances. Undersampling algorithms are prone to losing significant classification information, resulting in deceased classification performance. Some previous studies have indicated that oversampling algorithms are generally more robust than the undersampling algorithms [45]. The simplest oversampling algorithm, random oversampling (ROS) algorithm, causes overfitting of the learning model, whereas the SMOTE algorithm can overcome this disadvantage. Therefore, in this study, we focus on modifying the SMOTE technique to make it more robust and universal.

SMOTE, proposed by Chawla et al., in 2002 [20], has become a standard benchmark for learning from imbalanced data. Unlike ROS which duplicates the minority instances randomly, SMOTE generates synthetic instances between a minority instance and one of its neighborhood minority instances. In particular, SMOTE first selects a minority seed instance x randomly, then finds its K nearest minority instances based on the Euclidean distance, and next, one instance x' is randomly selected from these K ones. Finally, a new synthetic minority instance, y, is generated on a random position lying between x and x' using the following equation:

$$y = x + \theta \times (x' - x) \tag{1}$$

where θ is a random number lying in the range of (0, 1), which can guarantee the new instance y emerges on the line connecting x and x'. SMOTE can effectively address the overfitting caused by ROS. However, because SMOTE is a totally random algorithm; hence, it struggles to maintain the original data distribution, putting the classification model's performance at risk.

To improve SMOTE, an adaptive synthetic (ADASYN) [22] sampling algorithm was proposed which uses the ratio of majority instances in the K neighborhood of a minority instance x to indicate its hard-to-learn level. In particular, ADASYN calculates the density distribution of hard-to-learn levels of all instances, and adaptively determines the number of synthetic instances required for each minority instance. The more difficult it is to learn, the more synthetic instances that are generated. In particular, ADASYN focuses on the borderline region, but it might be impacted by the noise, causing noise propagation.

SL-SMOTE [39] divides the new instance generation of SMOTE into five different cases. Compared with ADASYN, SL-SMOTE can effectively prevent the noise propagation, and generate new instances in a safe position, according to the position information of its two seed instances, as much as possible.

MWMOTE [40] aims to prevent synthetic instances from being generated in incorrect positions. Compared with other popular SMOTE variants, MWMOTE identifies the hard-to-learn informative minority class samples and assigns them weights according to their Euclidean distance from the nearest majority class samples. Additionally, MWMOTE explores the internal substructure of training data using a clustering technique to guarantee that a new synthetic minority instance is generated inside the minority clusters. Therefore, the probability of incorrect instance generation can be reduced as much as possible.

SN-SMOTE [41] considers both the proximity and spatial distribution of the instances, and proposes using the surrounding neighborhood to generate the new synthetic instances. SN-SMOTE can effectively help in several situations (finite sample size case), where training instances do not fully represent the

underlying statistics and/or the distance used (irrelevant in the asymptotic case) exhibits some undesirable properties.

WK-SMOTE [42] combines the concepts of SMOTE oversampling and cost-sensitive learning to promote the performance of class imbalance learning. Instead of generating synthetic minority instances in the original input space, WK-SMOTE produces new minority instances in the kernel-transferred feature space by adopting the same generation rule. This guarantees the rationality of new instance positions. Then, WK-SMOTE further integrates both original training instances and newly generated synthetic minority instances to train a cost-sensitive SVM to enhance class imbalance learning performance. WK-SMOTE effectively decreases the risks of generating fallacious minority instances by running SMOTE in the kernel feature space; however, it is still affected by existing noisy information.

Similar to WK-SMOTE, the SMOTE-CSELM algorithm [43] integrates the ideas of both SMOTE oversampling and cost-sensitive learning. The differences between WK-SMOTE and SMOTE-CSELM reflect in the following two aspects: (1) WK-SMOTE generates synthetic instances in the kernel feature space, whereas the SMOTE-CSELM conducts SMOTE in the original input space; (2) WK-SMOTE adopts a cost-sensitive SVM to train the classification model, while SMOTE-CSELM constructs the classification model using the class-specific extreme learning machine (CS-ELM) [46] algorithm. The SMOTE-CSELM algorithm is fast and robust; however, it does not eliminate the influence of noise propagation.

Regardless of SMOTE, ADASYN, SL-SMOTE, MWMOTE, SN-SMOTE, WK-SMOTE or SMOTE-CSELM, they all have two inherent drawbacks. First, they either completely neglect to explore the data distribution, or exploit some non-robust prior distribution information relying on the neighborhood calculation. Second, they either strengthen the noise propagation, or fail to remove the original noisy instances. Therefore, producing a robust sampling result using these methods is difficult when dealing with a relatively complicated data distribution.

Some studies have noted the defects indicated above, and presented several indirect distribution information exploration strategies but not direct ones to solve the existing problems. SMOTE-IPF [26] benefits from the idea of *ensemble filtering* to iteratively and subtly seek out and remove the noisy and borderline instances generated in the SMOTE procedure. Supervised SMOTE [47] first trains an SVM classifier on the original imbalanced data, and then conducts a traditional SMOTE procedure and puts each synthetic minority instance into the trained classifier to estimate its confidence, reserving the high confidential instances and removing the low confidential ones until the dataset arrives at a pre-designated size. The Supervised_SMOTE algorithm could effectively hinder the noise propagation, but failed to stop the noise reservation; meanwhile, it stopped minority borderline strengthening. Selecting minority examples from misclassified data for oversampling (SMMO) [48] first takes advantage of the bagging ensemble to acquire misclassified minority instances, and then conducts a modified SMOTE only on these misclassified minority instances to generate new instances. SMMO can effectively strengthen the borderline, but it also accelerates the noise propagation. Additionally, the bagging ensemble constructed on originally imbalanced data is inevitable for providing biased misclassified results.

According to the review above, we consider it necessary to develop a more accurate and robust prior distribution information exploration strategy that is insensitive to the data distribution. We expect that the strategy can help to accurately locate the noisy, borderline and safe instances, further to remove the real noises and adaptively strengthen those hard-to-learn minority instances by SMOTE sampling.

3. Motivation

SMOTE and its variants have achieved significant success in the context of learning from imbalanced data; however, most of them use neighborhood calculations to directly estimate the data distribution, which can be extremely inaccurate estimation when the global data distribution is sophisticated. In particular, the inaccurate estimation of the data distribution might be induced by either an extremely high class imbalance ratio combining a large overlap or closely conjoint noises.

Fig. 1 presents the first case and shows a clear comparison between the low and high class imbalance ratios. Taking the Euclidean distance to conduct the K neighborhood calculation ($K = 5$), and further dividing the noise, borderline and safe instances for the minority class using the discrimination rules in SL-SMOTE and MWMOTE, it is not difficult to observe that on the low class imbalance ratio data, instance A is noise, instances B, C, D and E are borderline instances, and the rest belong to the safe category (see Fig. 1(a)), but on the high class imbalance data, instances A, B, C, D, and E are all seen as the noise; instances F, G, and H indicate the border; and only I and J can be regarded as safe (see Fig. 1(b)). This means that with as the class imbalance ratio increases, there is a greater risk of using the neighborhood calculation to partially estimate the distribution for the class imbalance data. To explore the reason for explaining this case, we consider that the high class imbalance ratio means in the borderline and even safe regions of the minority class. Although the majority class has a low density, its absolute quantity is still sufficiently large to overwhelm the minority instances, resulting in an extremely biased estimation of data distribution. There is also an exception: if the margin between the two classes is large, the estimation results will not be impacted by the class imbalance ratio variance.

Fig. 2 shows the second case, in which conjoint noise exists. The conjoint noise occurs when two noisy instances stay close to each other and for each one, it emerges in the neighborhood of the other one. In Fig. 2, we observe that instances A and B are clearly minority noisy instances as they are both distributed in the core region of the majority class. Meanwhile, they are conjoint instances as A appears in the K neighborhood ($K = 5$) of B, and vice versa. If adopting the improved SMOTE algorithms which are based on the neighborhood calculation (for example, MWMOTE), these two noisy instances would be wrongly seen as a small disjunction lying on the borderline, further generating many new noisy instances in the SMOTE procedure to reduce the latter modeling quality. Unlike the first case which is apt to recognize the real borderline and even safe instances as noises, this case tends to see noises as the borderline ones.

According to the above analysis, it is not difficult to draw a conclusion: neighborhood calculations are sensitive to data distribution, and cannot guarantee the capture of the correct location information on sophisticated data distributions. Therefore, most existing strategies are not robust solutions for exploring the data distribution information. This motivated us to seek a more robust distribution exploration strategy in this study.

4. Methods

Motivated by the problems stated in the previous section, we devised a novel SMOTE variant called Instance IW-SMOTE. It first adopts an UnderBagging-alike undersampling ensemble algorithm to indirectly explore the data distribution, then, according to the feedback confusion information, finds and removes those noises, and finally assigns the selection probability for each remainder instance based on its corresponding information to guide the SMOTE sampling procedure. The aim of IW-SMOTE is

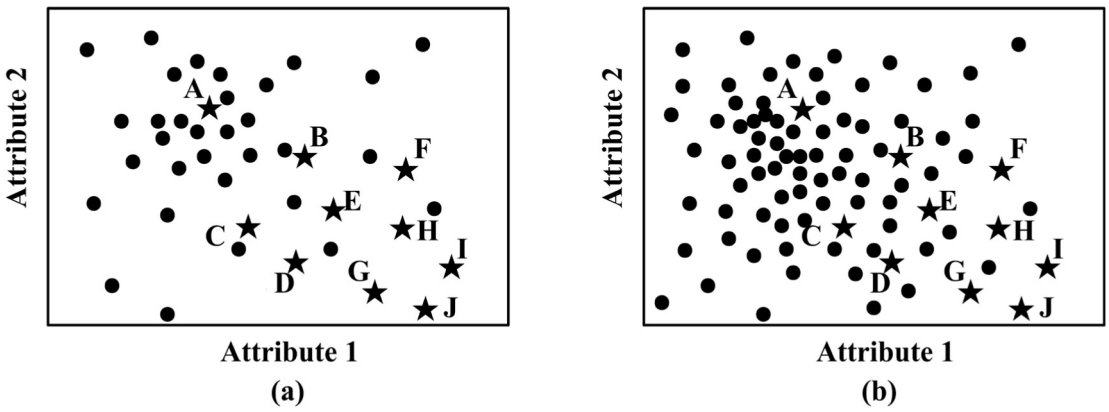


Fig. 1. The distributions with a large overlap, with (a) a low class imbalance ratio and (b) a high class imbalance ratio, and where ● and ★ denote the majority and minority instances, respectively.

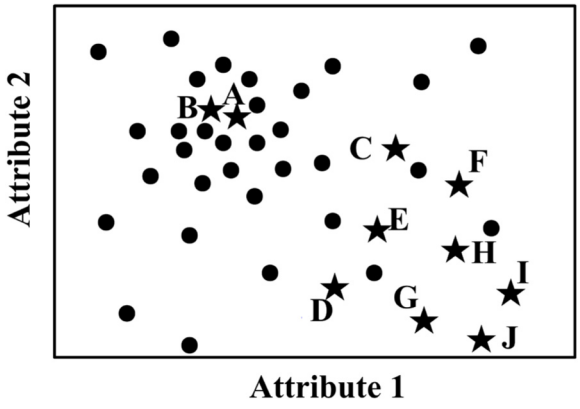


Fig. 2. The distribution with the conjoint noises, where ● and ★ denote the majority and minority instances, respectively.

threefold: (1) to provide accurate and robust estimation regardless of the complexity of the data distribution; (2) to remove the real noises to avoid the noise propagation and (3) to adaptively assign the selection probability to improve the effect of SMOTE.

4.1. Underbagging-alike undersampling ensemble estimation strategy

The distribution of class imbalance data is generally complex and diverse, because there are many factors that may impact the overall data distribution, including class imbalance ratio, noise ratio, sub-clusters within a class, density change, and class overlapping [49]. The fusion of these factors makes it more difficult to directly explore the real data distribution. This explains the reason why directly exploring and quantizing the data distribution information by neighborhood calculation, probability density estimation, clustering techniques or information entropy always fails for some complexly distributed datasets.

Compared with the direct data distribution exploration strategy, the indirect exploration technique may be more accurate and robust, because it uses the distribution-adaptive ability of the learning model. Generally, the indirect data distribution exploration technique feeds back the distribution information in the form of the relative location between the instance and the estimated classification boundary from the learning model. For example, FSVM-CIL [27] first trains an SVM learner on the original training data, and then checks the decision value of each instance relative to the classification hyperplane to distinguish the noises, borders and safe instances. Supervised_SMOTE [47] trains a model on the original data, then only reserves the new synthetic instances that can be accurately classified by the trained model. SMMO [48] conducts a bagging ensemble on the original skewed instances, and only generates instances near to those hard-to-learn, that is, misclassified minority instances, whereas MCNN [50] developed an active learning mechanism that used the Shannon entropy of each instance relative to the current Gaussian naive Bayes learner to iteratively search the significant majority instances, further implementing informative undersampling. These methods seem to be effective; however, the accuracy and robustness would still decrease as the original learner is trained on skewed data. The original imbalanced training data still tends to make the learner, considered as a reference, bias to the majority class, and provide inaccurate estimation of the data distribution.

To solve this problem, we present a novel indirect distribution information exploration strategy that aims to produce an unbiased estimation of the data distribution information. In particular, the idea of the proposed strategy is derived from a well-known CIL algorithm called UnderBagging [33]. As an undersampling ensemble algorithm, UnderBagging first randomly extracts the original majority training instances to constitute many majority subsets, and then combines each majority subset with all minority instances to construct a balanced training set. Finally, it trains the learning model on each balanced training set and makes decisions by majority voting. UnderBagging can be regarded as a combination of random undersampling and bagging ensembles, which not only uses all training instances to avoid information loss, but also produces an approximately unbiased classification boundary on each training set. Accordingly, in UnderBagging, there is a clear underlying hypothesis that *an instance holds a higher error rate, it is farer from the safe region*. In other words, in UnderBagging, the noise always have very high error rates and the safe instances often have fairly low error rates, while the borderline instances correspond to the medium error rates. This characteristic is helpful for accurately locating the instances.

In this work, to avoid overfitting each learner and failing to observe minority noises, we modified the UnderBagging algorithm by synchronously undersampling both classes. In the modified UnderBagging-alike algorithm, 2/3 minority instances and the same number of majority instances are randomly extracted to constitute a training subset in each division. Additionally, we argue that a robust learning model should be adopted to train each subset to provide an excellent generalization ability. In this study, we used CART [51] as the baseline learner because it is sufficiently robust and can be trained rapidly. Readers can replace the learning model in practical applications with whatever they

Algorithm 1: UnderBagging-alike estimation algorithm

Input: an imbalanced data set $\Phi=\{(x_1, y_1),(x_2, y_2),...,(x_N, y_N)\}$, where N denotes the number of instances, and $y_i \in \{+, -\}$; the parameter M which indicates the number of training subsets; the leaning model C.

Output: the vector $CI=\{CI_1, CI_2,..., CI_N\}$ which reserves the confusion information of all instances

1 Divide Φ into two sets, Φ^+ only contains the minority instances, while Φ^- contains all majority instances;
2 Count the number of instances in Φ^+ and Φ^-, and record them as N^+ and N^-, respectively;
3 For i=1:M
4 randomly extract $2N^+/3$ instances from Φ^+ and Φ^-, respectively;
5 train a classifier on the extracted instances by adopting the learning model C;
6 End
7 Integrate all M classifiers and put all instances in Φ, acquiring the corresponding confusion information vector CI;
8 Return the vector CI.

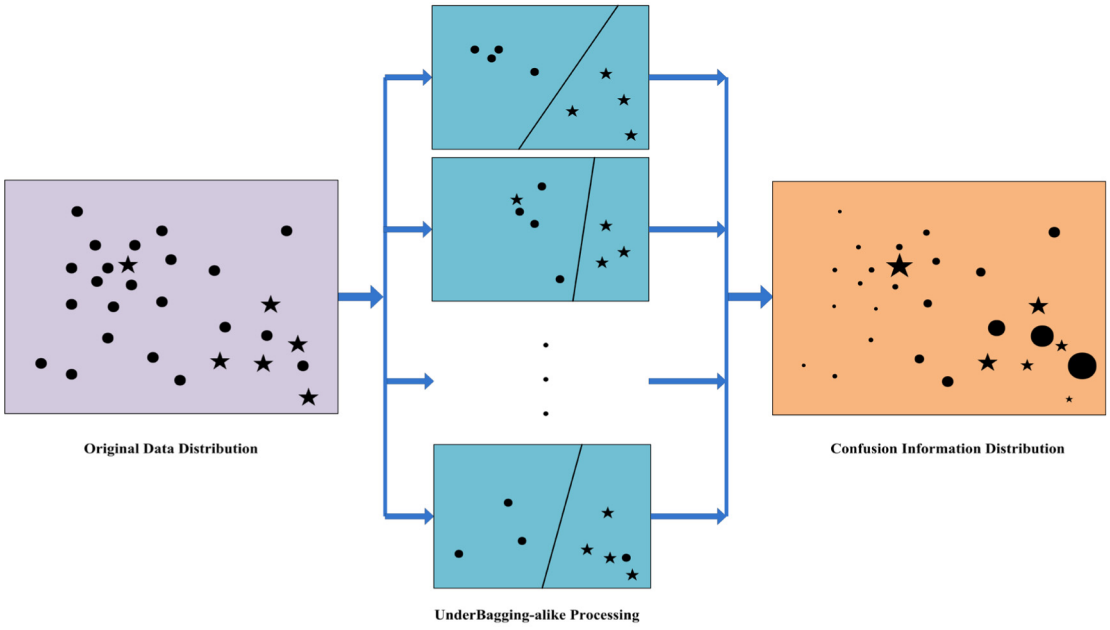


Fig. 3. The description about the procedure of the UnderBagging-alike algorithm, where in the confusion information distribution, the size of each symbol indicates its confusion information, the larger the symbol is, the higher confusion information the corresponding instance holds.

like. We believe that both the double undersampling and the generalized classification models can guarantee the accuracy of the location estimation.

Additionally, considering that the proposed UnderBagging-alike algorithm does not need to directly provide the classification boundary, but only provides the voting error rate information for each instance, we adopt the soft information to estimate each instance's prior distribution. Here, we call the voting error rate of each instance as *confusion information*. Obviously, the confusion information for any instance lies in the range of [0,1], and it can be seen as a specific description of the instance location.

The procedure for the UnderBagging-alike estimation algorithm is described below.

Additionally, Fig. 3 provides an intuitive description of the procedure for the UnderBagging-alike estimation algorithm.

4.2. Noise filtering and selection probability assignment based on confusion information

This confusion information can effectively reflect the location of each instance in the original data distribution. Generally, noise corresponds to a high confusion information, a safe instance matches low confusion information, and an instance lying near the borderline frequently contains a medium confusion information.

Based on the observation above, we designed a noise-filtering mechanism by setting an interceptive threshold λ for confusion information to avoid noise propagation. In particular, the threshold λ should be set between zero and one, and should not be too large or too small. If we assign an oversize value to λ, some noise would still be reserved, and if we designate an excessively small

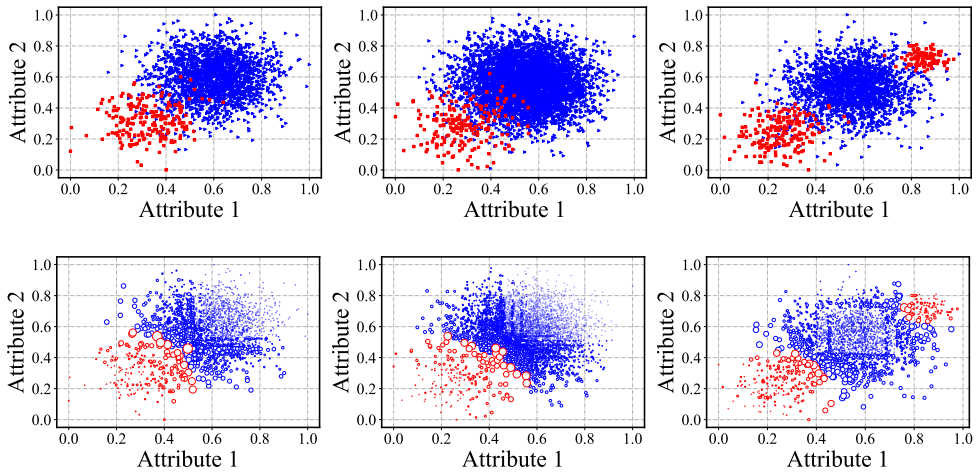


Fig. 4. Noise filtering and selection probability assignment on three different artificial imbalanced data sets where the first column denotes the data set with IR = 10, the second column indicates the data set with IR = 25, and the third column expresses the data set with a small disjunction. In addition, the first row denotes the original data distributions, and the second row denotes the corresponding selection probability distributions with the noise filtering threshold $\lambda = 0.5$, where the size of each symbol represents its confusion information, which can be seen as the selection probability of the corresponding instance, too.

λ, many borderline instances would be removed. In this study, we empirically set λ to 0.5.

Another important issue is how to determine the selection probability for each minority instance, further guiding the synthetic instance generation procedure of SMOTE. As indicated in previous work [22,39,40], the instances surrounding the boundary are generally more important than the safe ones; thus, SMOTE should generate more synthetic minority instances using the borderline instances as the seed instances. Profiting from the previous work, we calculate the selection probability of each minority instance as follows:

$$P_i = {}^{CI_i}/_{Z} \tag{2}$$

where P_i represents the selection probability of the ith minority instance, CI_i denotes the instance's confusion information, and Z is the normalized factor which is calculated as follows:

$$Z = \sum_{i=1}^{N^{+'}} CI_i \tag{3}$$

where $N^{+'}$ denotes the number of minority instances after removing the minority noises; thus, $N^{+'} \leq N^{+}$. Based on the normalized factor, it is not difficult to observe that the sum of the selection probabilities of all reserved minority instances is 1. Therefore, using the ith minority instance as the seed instance, the SMOTE should generate N_i synthetic minority instances which can be calculated as follows:

$$N_i = \beta \times (N^{-'} - N^{+'}) \times P_i \tag{4}$$

where β denotes the sampling coefficient between zero and one which is used to control the total number of generated minority instances, and $N^{-'}$ denotes the number of majority instances after removing the majority noise.

Compared with several previous studies [22,39,40] which used the hard division strategy to distinguish borderline and safe minority instances, and only assigns a selection probability for each borderline instance; our selection probability assignment adopts a soft weighting strategy. This means that there is no clear division between the borderline and safe regions, and each minority instance is assigned a selection probability that relies only its confusion information. We believe that the adoption of this soft weighting strategy would be sounder and could guarantee impartiality of SMOTE sampling.

To illustrate the effectiveness of our proposed noise-filtering mechanism and the selection probability strategy, we randomly generated three artificial imbalanced datasets. The first dataset has a class imbalance ratio of ten, and the second dataset has a higher class imbalance ratio of twenty-five. For the third dataset, we add a small disjunction for the minority class and expect to observe whether the proposed strategy can adapt to a complex data distribution. Fig. 4 shows the data distributions and processed results of the three artificial datasets. Fig. 4 shows that the proposed strategy can effectively find and remove noise, and reasonably assign a selection probability for every training instance. Additionally, the proposed strategy is insensitive to variations in the class imbalance ratio and the complexity of the data distribution. Therefore, we believe that the proposed prior-distribution information estimation method is accurate, robust and universal.

4.3. Description about the IW-SMOTE algorithm

According to the above description, the flow of the IW-SMOTE algorithm can be presented as follows.

Based on the IW-SMOTE algorithm, the three artificial imbalanced datasets presented in Fig. 4 were oversampled, and the results with $\beta = 1$ are shown in Fig. 5. It can be seen that on each dataset, borderline minority instances have been highlighted, which further illustrates the effectiveness of the proposed IW-SMOTE algorithm.

4.4. Time-complexity analysis

Nevertheless, we should focus on the time complexity. The time complexity of training CART is $O(NLD)$, where N is the number of training instances, L denotes the number of attributes, and D represents the depth of a tree. Therefore, the procedure of UnderBagging-alike estimation algorithm requires $O(MNLD)$ time, where M is the number of CART learners. In comparison with the high time complexity of the UnderBagging-alike estimation procedure, the noise filtering and selection probability assignment are extremely time-saving as they only need to consume $O(N)$ time. Finally, the SMOTE procedure saves time. In SMOTE, calculating distances will take $O(N^2L)$ time, whereas sorting will cost $O(N^2 \log N)$ time, and new instance generation will consume $O(NL)$ time.

Algorithm 2: IW-SMOTE algorithm

Input: an imbalanced data set $\Phi=\{(x_1, y_1),(x_2, y_2),...,(x_N, y_N)\}$, where N denotes the number of instances, and $y_i \in \{+, -\}$; the parameter M which indicates the number of training subsets; the leaning model C; the sampling coefficient β; the noise interceptive threshold λ; the neighborhood parameter K.

Output: an oversampling balanced data set Θ

Procedure:

```
Call UnderBagging-alike algorithm to acquire confusion information vector CI;
Use the threshold λ to remove the noisy instances in Φ and put the remainder instances into Θ,
    record the number of the remainder majority and minority instances as N^- and N^+', respectively;
For the remainder minority instances, calculate the corresponding normalized factor Z by
    Eq.(3);
For each remainder minority instance, calculate its selection probability by Eq.(2);
Calculate the number of synthetic instances which need to be generated by N^- - N^+';
For i=1: N^+'
        calculate the number of synthetic instances needing to be generated with adopting the
    ith minority instance as the seed instance by Eq.(4);
        generate the corresponding number of synthetic instances by repeatedly calling SMOTE
    algorithm;
        put the new generated instances into Θ;
End
Return the oversampling balanced data set Θ.
```

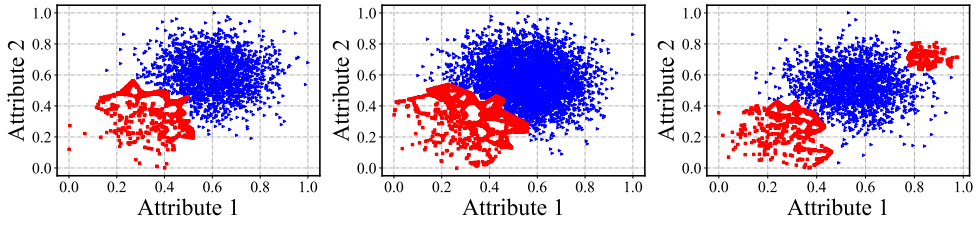


Fig. 5. The oversampling data sets with the sampling coefficient $\beta = 1$ acquired by IW-SMOTE algorithm, where the first subgraph denotes the data set with IR = 10, the second subgraph indicates the data set with IR = 25, and the third subgraph denotes the data set with a small disjunction.

According to the analysis above, we must admit that the proposed IW-SMOTE algorithm has a relatively high time complexity compared to simple SMOTE variants.

5. Results and discussion

5.1. Data sets description

We collected 40 binary-class imbalanced data sets to verify the effectiveness of our proposed algorithm. The collection included 13 data sets acquired from the Keel data repository [52], 26 data sets collected from the UCI machine learning repository [53], and one real bioinformatics data set to distinguish the real microRNA precursors from lots of microRNA candidates [54]. These datasets have 3∼32 attributes, 192∼20 000 instances, and the class imbalance ratio (IR) varies from 1.25 to 129.53. Detailed descriptions of the data sets are provided in Table 1.

5.2. Experimental settings

All experiments were run on a 2.60 GHz Intel(R) Core(TM) i7 6700HQ 8-core CPU with 16 GB RAM using the Python 3.8 running environment.

To validate the effectiveness and superiority of the proposed IW-SMOTE algorithm, we compared it with two baseline algorithms: ORI, which means directly training the classification model on the original skewed data without any class imbalance learning strategy, and SMOTE. The proposed algorithm was also compared with several popular SMOTE variants, including ADASYN [22], SL-SMOTE [39], MWMOTE [40], SN-SMOTE [41], WK-SMOTE [42], SMOTE-CSELM [43], SMOTE-IPF [26], Supervised_SMOTE [47] and SMMO [48]. In the experiments, all algorithms except IW-SMOTE, WK-SMOTE and SMOTE-CSELM, called the smote-variant python tool package [55], which is available at: https://github.com/analyticalmindsltd/smote_variants. Additionally, to determine whether the proposed IW-SMOTE algorithm is irrelevant to the specific classification model, we adopted three different classification algorithms, namely, CART, KNN and SVM, in the experiments.

Table 1
The description of datasets used in this paper.

Dataset	Number of attributes	Number of instances	Minority class	Majority class	IR	Source
banknote	4	1372	Class 1	Class 0	1.25	UCI
seeds2v13	7	210	Class 2	Class 1,3	2.00	UCI
ILPD	10	583	Positive	Negative	2.49	UCI
credit card clients	23	10 000	Default payment next month1	Default payment next month0	3.46	UCI
mfeatmor01	6	2000	Class 0, 1	Remainder classes	4.00	UCI
segment1	19	2310	Class 1	Class 2∼7	6.00	UCI
segmentation-grass	19	210	Grass	Path, window, cement, poliage, sky, brickface	6.00	UCI
pageblocks2345	10	5473	Class 2∼5	Class 1	8.77	UCI
mfeatmor0	6	2000	Class 0	Remainder classes	9.00	UCI
ctg23	21	2126	Class 2, 3	Remainder classes	11.08	UCI
ctg15	21	2126	Class 1, 5	Remainder classes	28.52	UCI
letterA	21	20 000	Letter A	Letters B∼Z	112.64	UCI
abalone19	8	4177	Class 19	Remainder classes	129.53	UCI
ecoli0vs1	7	220	Class 0	Class 1	1.86	Keel
pima	8	768	Positive	Negative	1.87	Keel
iris0	4	150	Class 0	Class 1, 2	2.00	Keel
glass0	9	214	Class 0	Remainder classes	2.06	Keel
yeast1	8	1484	Class 1	Remainder classes	2.46	Keel
haberman	3	306	Positive	Negative	2.78	Keel
vehicle2	18	846	Class 2	Remainder classes	2.88	Keel
vehicle3	18	846	Class 3	Remainder classes	2.99	Keel
glass0123vs456	9	214	Class 0, 1, 2, 3	Class 4, 5, 6	3.20	Keel
ecoli1	7	336	Class 1	Remainder classes	3.36	Keel
yeast3	8	1484	Class 2	Remainder classes	8.10	Keel
ecoli3	7	336	Class 3	Remainder classes	8.60	Keel
yeast2vs4	8	514	Class 2	Class 4	9.08	Keel
yeast05679vs4	8	528	Class 0, 5, 6, 7, 9	Class 4	9.35	Keel
vowel0	13	988	Class 0	Remainder classes	9.98	Keel
glass016vs2	9	192	Class 0, 1, 6	Class 2	10.29	Keel
glass2	9	214	Class 2	Remainder classes	11.59	Keel
yeast1vs7	8	459	Class 1	Class 7	14.30	Keel
ecoli4	7	336	Class 4	Remainder classes	15.80	Keel
yeast1458vs7	8	693	Class 1, 4, 5, 8	Class 7	22.10	Keel
yeast2vs8	8	482	Class 2	Class 8	23.10	Keel
yeast4	8	1484	Class 4	Remainder classes	28.10	Keel
yeast5	8	1484	Class 5	Remainder classes	32.73	Keel
yeast6	8	1484	Class 6	Remainder classes	41.40	Keel
poker89vs5	10	2075	Class 8, 9	Class 5	82.00	Keel
poker8vs6	10	1477	Class 8	Class 6	85.88	Keel
microRNA	32	8587	Real microRNA precursors	Remainder microRNA candidates	44.01	Ref. [54]

Regarding the parameter setting, all sampling algorithms share a common neighborhood parameter $K = 5$ to guarantee the impartiality of the comparison. This is also the most popular and general parameter setting in SMOTE and its variants [20,22,25,26,39–43]. We adopted the recommended values as the default values for specific parameters arising in various SMOTE variants. We also noted that there are three important specific parameters in the proposed IW-SMOTE algorithm: the noise interceptive threshold λ, the number of training subsets in the UnderBagging-alike strategy M, and the final one is the sampling coefficient β. In the experiments, we provided an empirical default value 0.5 of λ, and made M an adaptive parameter associated with the class imbalance ratio IR, that is, $M = 6 \times \text{IR}$. Considering that each subset contains only 2/3 minority instances and the same number of majority instances, this parameter setting guarantees in UnderBagging, each majority and minority instance is extracted 4 and $4 \times$ IR times, respectively. β is designated as one when the IR of a dataset is lower than or equal to two, while for other datasets with an IR higher than two, an internal model selection procedure would be conducted in the range of $\beta = \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0\}$. For the classification models, the parameter K in KNN is designated as five, and the SVM uses the Gaussian radial basis (RBF) kernel function, in which the width γ and the penalty factor C were set to be 1 and 100, respectively.

Before training any classifier, each data set was scaled into a [0,1] interval. Additionally, considering the CIL problem, the total classification accuracy is no longer an excellent performance evaluation metric; hence, we adopted three important metrics to evaluate the performance of the various CIL algorithms; Recall, G-mean and AUC. Recall can be calculated as follows:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{5}$$

where TP and FN denote the number of minority instances classified correctly and the number of majority instances identified incorrectly, respectively. The G-mean can be calculated using the following equation,

$$\text{G-mean} = \sqrt{\text{Acc}_+ \times \text{Acc}_-} \tag{6}$$

where Acc_+ and Acc_- denote the accuracies of the minority and majority classes, respectively. The G-mean metric evaluates the tradeoff between the accuracy of the minority and majority classes. AUC represents the area under the receiver operating curve (ROC), which is a popular metric for evaluating the performance of a CIL algorithm.

Finally, to impartially compare the performance of various algorithms, we adopted ten random external five-fold cross validations to calculate the final results which are provided in the form of mean ± standard deviation.

5.3. Comparison with the baselines and several SMOTE variants

The Recall, G-mean and AUC results of various algorithms based on the CART, KNN and SVM classifiers are presented in Tables 2–10.

By observing the results in Tables 2–10, it is not difficult to conclude the following.

Table 2
Recall results of twelve comparative algorithms on forty imbalanced data sets based on CART classifier.

Dataset	ORI	SMOTE	SL-SMOTE	SN-SMOTE	MWMOTE	ADASYN	SMOTE-IPF	SMMO	Supervised_SMOTE	WK-SMOTE	SMOTE-CSELM	IW-SMOTE
banknote	0.9836 ± 0.0043	0.9836 ± 0.0016	0.9831 ± 0.0019	0.9858 ± 0.0058	0.9863 ± 0.0025	0.9836 ± 0.0028	0.9814 ± 0.0047	0.9858 ± 0.0019	0.9825 ± 0.0034	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
seeds2v13	0.9667 ± 0.0082	0.9762 ± 0.0082	0.9619 ± 0.0165	0.9667 ± 0.0218	0.9714 ± 0.0143	0.9667 ± 0.0082	0.9667 ± 0.0165	0.9571 ± 0.0286	0.9714 ± 0.0000	0.9857 ± 0.0143	0.9857 ± 0.0143	**1.0000 ± 0.0000**
ILPD	0.3913 ± 0.0399	0.5355 ± 0.0275	0.5150 ± 0.0300	0.4830 ± 0.0393	0.4670 ± 0.0370	0.4623 ± 0.0172	0.5067 ± 0.0449	0.4696 ± 0.0362	0.4735 ± 0.0209	0.7054 ± 0.0031	0.7321 ± 0.0026	**0.7405 ± 0.0311**
credit card clients	0.4161 ± 0.0087	0.4696 ± 0.0016	0.4825 ± 0.0054	0.4698 ± 0.0063	0.4374 ± 0.0149	0.4533 ± 0.0139	0.4657 ± 0.0042	0.4262 ± 0.0051	0.4277 ± 0.0034	0.6133 ± 0.0013	0.6257 ± 0.0004	**0.6372 ± 0.0057**
mfeatmor01	0.9300 ± 0.0087	0.9458 ± 0.0052	0.9333 ± 0.0014	0.9450 ± 0.0066	0.9425 ± 0.0066	0.9608 ± 0.0029	0.9433 ± 0.0076	0.9417 ± 0.0029	0.9425 ± 0.0043	0.9750 ± 0.0000	0.9667 ± 0.0014	**0.9783 ± 0.0005**
segment1	0.9747 ± 0.0017	0.9848 ± 0.0091	0.9818 ± 0.0061	0.9869 ± 0.0035	0.9859 ± 0.0093	0.9778 ± 0.0035	0.9889 ± 0.0046	0.9768 ± 0.0017	0.9848 ± 0.0030	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
segmentation-grass	0.9778 ± 0.0192	0.9889 ± 0.0192	0.9778 ± 0.0192	0.9778 ± 0.0192	0.9889 ± 0.0192	**1.0000 ± 0.0000**	0.9778 ± 0.0192	0.9889 ± 0.0192	0.9778 ± 0.0192	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
pageblocks2345	0.8220 ± 0.0074	0.8708 ± 0.0045	0.8488 ± 0.0037	0.8726 ± 0.0129	0.8810 ± 0.0072	0.8845 ± 0.0119	0.8726 ± 0.0057	0.8488 ± 0.0170	0.8667 ± 0.0074	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
mfeatmor0	0.9850 ± 0.0000	0.9867 ± 0.0029	0.9850 ± 0.0000	0.9883 ± 0.0029	0.9850 ± 0.0000	0.9900 ± 0.0000	0.9883 ± 0.0029	0.9850 ± 0.0000	0.9850 ± 0.0000	0.9950 ± 0.0029	0.9967 ± 0.0000	**0.9967 ± 0.0029**
ctg23	0.9050 ± 0.0033	0.8841 ± 0.0150	0.9013 ± 0.0091	0.8920 ± 0.0052	0.8975 ± 0.0152	0.8633 ± 0.0203	0.8707 ± 0.0085	0.8599 ± 0.0202	0.9239 ± 0.0090	0.9018 ± 0.0009	0.9155 ± 0.0153	**0.9162 ± 0.0032**
ctg15	0.5435 ± 0.0191	0.5875 ± 0.0504	0.4952 ± 0.0659	0.5899 ± 0.0068	0.5982 ± 0.0331	0.5905 ± 0.0723	0.6006 ± 0.0326	0.5488 ± 0.0225	0.5524 ± 0.0627	0.7495 ± 0.0549	0.7173 ± 0.0548	**0.7190 ± 0.0423**
letterA	0.6115 ± 0.0028	0.6029 ± 0.0154	0.5683 ± 0.0062	0.6046 ± 0.0177	0.5791 ± 0.0344	0.5889 ± 0.0086	0.6001 ± 0.0255	0.5060 ± 0.0213	0.6459 ± 0.0136	0.6289 ± 0.0121	0.7066 ± 0.0221	**0.9526 ± 0.0026**
abalone19	0.0500 ± 0.0464	0.2000 ± 0.0722	0.0639 ± 0.0625	0.1361 ± 0.1220	0.1250 ± 0.0300	0.1833 ± 0.0167	0.1917 ± 0.0577	0.0944 ± 0.0376	0.0444 ± 0.0192	0.9300 ± 0.0029	0.9600 ± 0.0056	0.6889 ± 0.0210
ecoli0vs1	0.9749 ± 0.0080	0.9724 ± 0.0120	0.9584 ± 0.0322	0.9840 ± 0.0175	0.9743 ± 0.0161	0.9649 ± 0.0069	0.9656 ± 0.0069	0.9705 ± 0.0104	0.9749 ± 0.0040	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
pima	0.5859 ± 0.0295	0.6021 ± 0.0347	0.6079 ± 0.0125	0.5905 ± 0.0110	0.6067 ± 0.0430	0.5959 ± 0.0078	0.5846 ± 0.0285	0.5722 ± 0.0178	0.5935 ± 0.0136	0.8721 ± 0.0218	0.8096 ± 0.0018	0.7453 ± 0.0271
iris0	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass0	0.7476 ± 0.0082	0.7714 ± 0.0286	0.7714 ± 0.0143	0.7619 ± 0.0360	0.7714 ± 0.0247	0.7810 ± 0.0595	0.8048 ± 0.0082	0.6905 ± 0.0297	0.7524 ± 0.0502	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.8857 ± 0.0286
yeast1	0.4981 ± 0.0081	0.5424 ± 0.0246	0.5480 ± 0.0251	0.5461 ± 0.0226	0.5991 ± 0.0203	0.5463 ± 0.0281	0.5633 ± 0.0234	0.5441 ± 0.0199	0.5564 ± 0.0070	0.7039 ± 0.0118	**0.7155 ± 0.0083**	0.6962 ± 0.0088
haberman	0.3509 ± 0.0269	0.4162 ± 0.0063	0.3741 ± 0.0727	0.3787 ± 0.0599	0.3944 ± 0.0568	0.4125 ± 0.0210	0.4199 ± 0.0347	0.3833 ± 0.0315	0.4403 ± 0.0069	**0.9733 ± 0.0054**	0.7772 ± 0.0045	0.7800 ± 0.0196
vehicle2	0.9206 ± 0.0072	0.9374 ± 0.0137	0.9301 ± 0.0322	0.9300 ± 0.0067	0.9286 ± 0.0024	0.9420 ± 0.0112	0.9329 ± 0.0099	0.9144 ± 0.0092	0.9328 ± 0.0095	0.9504 ± 0.0096	0.9505 ± 0.0095	**0.9526 ± 0.0097**
vehicle3	0.4830 ± 0.0105	0.5616 ± 0.0189	0.5977 ± 0.0269	0.5349 ± 0.0319	0.5586 ± 0.0077	0.5647 ± 0.0264	0.5664 ± 0.0330	0.5444 ± 0.0406	0.5895 ± 0.0246	0.7804 ± 0.0090	**0.7981 ± 0.0033**	0.7841 ± 0.0368
glass0123vs456	0.8956 ± 0.0315	0.8522 ± 0.0302	0.8789 ± 0.0214	0.8778 ± 0.0252	0.9222 ± 0.0184	0.8711 ± 0.0636	0.8411 ± 0.0506	0.8689 ± 0.0444	0.9033 ± 0.0167	**1.0000 ± 0.0000**	0.9600 ± 0.0054	0.9633 ± 0.0115
ecoli1	0.7387 ± 0.0322	0.8089 ± 0.0281	0.7661 ± 0.0234	0.8190 ± 0.0233	0.8119 ± 0.0295	0.7869 ± 0.0330	0.7958 ± 0.0381	0.7887 ± 0.0582	0.8125 ± 0.0082	**1.0000 ± 0.0000**	1.0000 ± 0.0000	0.9369 ± 0.0010
yeast3	0.6870 ± 0.0061	0.7582 ± 0.0126	0.7487 ± 0.0165	0.7756 ± 0.0418	0.7433 ± 0.0141	0.7706 ± 0.0042	0.7697 ± 0.0230	0.7339 ± 0.0310	0.7458 ± 0.0101	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
ecoli3	0.6028 ± 0.0818	0.5361 ± 0.0536	0.5611 ± 0.1029	0.5611 ± 0.0255	0.6167 ± 0.0083	0.6556 ± 0.1110	0.6250 ± 0.1090	0.6167 ± 0.0546	0.5500 ± 0.0220	**1.0000 ± 0.0000**	0.9429 ± 0.0090	0.9444 ± 0.0173
yeast2vs4	0.7389 ± 0.0643	0.7044 ± 0.0417	0.6900 ± 0.0578	0.7633 ± 0.0664	0.7589 ± 0.0241	0.7800 ± 0.0231	0.7644 ± 0.0536	0.7633 ± 0.0524	0.7311 ± 0.0135	0.8156 ± 0.0135	0.8278 ± 0.0192	**1.0000 ± 0.0000**
yeast05679vs4	0.4456 ± 0.0212	0.5267 ± 0.0433	0.4333 ± 0.0529	0.5722 ± 0.0726	0.5800 ± 0.0731	0.5400 ± 0.0433	0.4911 ± 0.0337	0.3867 ± 0.0306	0.5078 ± 0.0435	0.3782 ± 0.0092	**0.8600 ± 0.0000**	0.7700 ± 0.0318
vowel0	0.9222 ± 0.0192	0.9407 ± 0.0064	0.9259 ± 0.0231	0.9074 ± 0.0064	0.9370 ± 0.0128	0.9148 ± 0.0170	0.9407 ± 0.0128	0.9370 ± 0.0128	0.9222 ± 0.0111	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass016vs2	0.2333 ± 0.0289	0.4333 ± 0.0764	0.3833 ± 0.0764	0.3500 ± 0.0500	0.3500 ± 0.0866	0.3833 ± 0.1607	0.4167 ± 0.2021	0.2833 ± 0.0764	0.3667 ± 0.0289	0.6677 ± 0.0289	0.7100 ± 0.0500	**0.7333 ± 0.0289**
glass2	0.2167 ± 0.0764	0.5000 ± 0.1000	0.3000 ± 0.0866	0.3833 ± 0.1443	0.4333 ± 0.0764	0.4167 ± 0.1155	0.4667 ± 0.0289	0.2667 ± 0.0764	0.4333 ± 0.1258	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.7500 ± 0.0866
yeast1vs7	0.3667 ± 0.0577	0.4444 ± 0.0385	0.3222 ± 0.0385	0.3333 ± 0.1528	0.4778 ± 0.1171	0.3778 ± 0.0192	0.5222 ± 0.0192	0.4111 ± 0.0839	0.4444 ± 0.0962	**0.2333 ± 0.0290**	**1.0000 ± 0.0000**	0.6333 ± 0.0667
ecoli4	0.6667 ± 0.0289	0.7667 ± 0.0289	0.7833 ± 0.0289	0.8000 ± 0.0500	0.8167 ± 0.0289	0.7500 ± 0.0500	0.7833 ± 0.0577	0.7000 ± 0.0500	0.7667 ± 0.0289	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.7833 ± 0.0289
yeast1458vs7	0.1000 ± 0.0000	0.2556 ± 0.0192	0.1889 ± 0.1072	0.2667 ± 0.0000	0.2333 ± 0.0882	0.2444 ± 0.0509	0.2889 ± 0.0192	0.1333 ± 0.0577	0.0889 ± 0.0509	0.5111 ± 0.0770	0.5333 ± 0.0577	**0.5556 ± 0.0192**
yeast2vs8	0.4500 ± 0.0000	0.6333 ± 0.0289	0.4000 ± 0.0000	0.5333 ± 0.0764	0.6167 ± 0.0577	0.5167 ± 0.1041	0.6667 ± 0.0289	0.4500 ± 0.0500	0.5333 ± 0.0289	0.9200 ± 0.0218	**1.0000 ± 0.0000**	0.5667 ± 0.0577
yeast4	0.5097 ± 0.0102	0.5685 ± 0.0136	0.5529 ± 0.0283	0.5694 ± 0.0290	0.5760 ± 0.0227	0.5436 ± 0.0042	0.5587 ± 0.0253	0.5059 ± 0.0170	0.5343 ± 0.0120	0.7208 ± 0.0076	0.7313 ± 0.0255	**0.7333 ± 0.0122**
yeast5	0.6800 ± 0.0500	0.7483 ± 0.0625	0.6883 ± 0.1193	0.7633 ± 0.0580	0.7550 ± 0.0397	0.7683 ± 0.0225	0.7533 ± 0.0473	0.6733 ± 0.0814	0.7167 ± 0.0247	0.9117 ± 0.0153	0.9200 ± 0.0132	**0.9767 ± 0.0029**
yeast6	0.5194 ± 0.0127	0.5472 ± 0.0241	0.5111 ± 0.0636	0.5333 ± 0.0220	0.5556 ± 0.0268	0.5528 ± 0.0542	0.5583 ± 0.0220	0.4167 ± 0.0433	0.4639 ± 0.0210	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.7778 ± 0.0268
poker89vs5	0.2222 ± 0.0255	0.1889 ± 0.0839	0.1389 ± 0.0674	0.1722 ± 0.0419	0.1500 ± 0.1000	0.2333 ± 0.0833	0.1611 ± 0.1018	0.1056 ± 0.0536	0.1167 ± 0.0441	0.4159 ± 0.0945	0.4935 ± 0.0691	**0.5389 ± 0.0347**
poker8vs6	0.0833 ± 0.0289	0.3833 ± 0.0764	0.1833 ± 0.1041	0.2500 ± 0.1323	0.2167 ± 0.2021	0.5500 ± 0.1000	0.4167 ± 0.1258	0.5333 ± 0.2021	0.4500 ± 0.0500	0.8554 ± 0.0765	**0.8592 ± 0.0731**	0.4500 ± 0.0866
microRNA	0.4924 ± 0.0059	0.5788 ± 0.0165	0.4786 ± 0.0278	0.5963 ± 0.0238	0.6015 ± 0.0090	0.5714 ± 0.0206	0.5807 ± 0.0179	0.4753 ± 0.0144	0.5701 ± 0.0057	0.7814 ± 0.0156	0.7742 ± 0.0111	**0.7858 ± 0.0005**

Table 3
Recall results of twelve comparative algorithms on forty imbalanced data sets based on KNN classifier.

Dataset	ORI	SMOTE	SL-SMOTE	SN-SMOTE	MWMOTE	ADASYN	SMOTE-IPF	SMMO	Supervised_SMOTE	WK-SMOTE	SMOTE-CSELM	IW-SMOTE
banknote	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
seeds2v13	0.9714 ± 0.0000	0.9714 ± 0.0000	0.9810 ± 0.0082	0.9762 ± 0.0082	0.9857 ± 0.0143	**1.0000 ± 0.0000**	0.9810 ± 0.0082	0.9904 ± 0.0082	0.9714 ± 0.0000	0.9857 ± 0.0143	0.9857 ± 0.0143	0.9905 ± 0.0082
ILPD	0.2979 ± 0.0092	0.6610 ± 0.0218	0.6225 ± 0.0299	0.6371 ± 0.0130	0.6800 ± 0.0373	0.6268 ± 0.0387	0.6513 ± 0.0280	0.3695 ± 0.0183	0.5886 ± 0.0344	0.7054 ± 0.0031	0.7321 ± 0.0026	**0.7370 ± 0.0140**
credit card clients	0.3511 ± 0.0019	0.5995 ± 0.0014	0.5843 ± 0.0103	0.6053 ± 0.0082	0.5390 ± 0.0019	0.6296 ± 0.0012	0.6012 ± 0.0039	0.5012 ± 0.0098	0.4839 ± 0.0044	0.6133 ± 0.0013	0.6257 ± 0.0004	**0.6610 ± 0.0016**
mfeatmor01	0.9458 ± 0.0014	0.9583 ± 0.0038	0.9508 ± 0.0014	0.9617 ± 0.0014	0.9633 ± 0.0014	0.9725 ± 0.0025	0.9608 ± 0.0014	0.9575 ± 0.0025	0.9550 ± 0.0025	0.9750 ± 0.0000	0.9667 ± 0.0014	**0.9758 ± 0.0052**
segment1	0.9808 ± 0.0017	0.9919 ± 0.0035	0.9879 ± 0.0030	0.9939 ± 0.0000	0.9939 ± 0.0000	0.9939 ± 0.0000	0.9939 ± 0.0000	0.9909 ± 0.0000	0.9929 ± 0.0017	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
segmentation-grass	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
pageblocks2345	0.7095 ± 0.0102	0.8905 ± 0.0084	0.7619 ± 0.0191	0.8952 ± 0.0045	0.9113 ± 0.0057	0.9161 ± 0.0062	0.8887 ± 0.0090	0.8345 ± 0.0045	0.8804 ± 0.0036	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
mfeatmor0	0.9850 ± 0.0000	0.9883 ± 0.0058	0.9850 ± 0.0000	0.9883 ± 0.0058	0.9850 ± 0.0000	0.9900 ± 0.0050	0.9850 ± 0.0000	0.9850 ± 0.0000	0.9850 ± 0.0000	0.9950 ± 0.0029	0.9967 ± 0.0000	**0.9983 ± 0.0008**
ctg23	0.7022 ± 0.0042	0.9005 ± 0.0093	0.7662 ± 0.0208	0.9026 ± 0.0091	0.9198 ± 0.0054	0.9142 ± 0.0110	0.9063 ± 0.0093	0.8646 ± 0.0031	0.9086 ± 0.0148	0.9018 ± 0.0009	0.9155 ± 0.0153	**0.9354 ± 0.0058**
ctg15	0.1042 ± 0.0084	0.6714 ± 0.0193	0.2482 ± 0.0758	0.6482 ± 0.0736	0.7220 ± 0.0173	0.6815 ± 0.0277	0.7137 ± 0.0337	0.4857 ± 0.0811	0.6750 ± 0.0331	0.7495 ± 0.0549	0.7173 ± 0.0548	**0.8327 ± 0.0237**
letterA	0.1101 ± 0.0028	0.5131 ± 0.0093	0.1179 ± 0.0072	0.5039 ± 0.0203	0.6080 ± 0.0108	0.5112 ± 0.0119	0.5111 ± 0.0062	0.2647 ± 0.0037	0.5282 ± 0.0109	0.6289 ± 0.0121	**0.7066 ± 0.0221**	0.6936 ± 0.0113
abalone19	0.0000 ± 0.0000	0.3139 ± 0.0585	0.0000 ± 0.0000	0.2528 ± 0.0488	0.1694 ± 0.0127	0.3417 ± 0.0546	0.3333 ± 0.0712	0.1333 ± 0.0144	0.0333 ± 0.0333	0.9300 ± 0.0029	**0.9600 ± 0.0056**	0.6972 ± 0.0394
ecoli0vs1	**1.0000 ± 0.0000**	0.9954 ± 0.0080	0.9908 ± 0.0104	0.9954 ± 0.0040	0.9976 ± 0.0041	0.9606 ± 0.0040	0.9976 ± 0.0041	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
pima	0.5585 ± 0.0045	0.7053 ± 0.0035	0.7065 ± 0.0077	0.6966 ± 0.0076	0.7142 ± 0.0141	0.7140 ± 0.0086	0.7040 ± 0.0129	0.7178 ± 0.0140	0.7014 ± 0.0002	**0.8721 ± 0.0218**	0.8096 ± 0.0018	0.7103 ± 0.0089
iris0	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass0	0.8095 ± 0.0165	0.8952 ± 0.0218	0.8905 ± 0.0082	0.8905 ± 0.0165	0.9190 ± 0.0165	0.9048 ± 0.0218	0.9000 ± 0.0000	0.8571 ± 0.0143	0.9190 ± 0.0082	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.8857 ± 0.0000
yeast1	0.4230 ± 0.0090	0.6715 ± 0.0232	0.6567 ± 0.0165	0.6676 ± 0.0140	0.6908 ± 0.0013	0.6925 ± 0.0129	0.6730 ± 0.0071	0.6490 ± 0.0087	0.6567 ± 0.0087	0.7039 ± 0.0118	0.7155 ± 0.0083	**0.7304 ± 0.0128**
haberman	0.1736 ± 0.0111	0.5139 ± 0.0177	0.4796 ± 0.0516	0.5185 ± 0.0323	0.5597 ± 0.0431	0.5153 ± 0.0378	0.5185 ± 0.0369	0.4861 ± 0.0257	0.4644 ± 0.0327	0.9733 ± 0.0054	**0.7772 ± 0.0045**	0.5259 ± 0.0395
vehicle2	0.9267 ± 0.0047	0.9862 ± 0.0079	0.9710 ± 0.0107	0.9909 ± 0.0045	0.9877 ± 0.0027	0.9955 ± 0.0045	0.9877 ± 0.0027	0.9939 ± 0.0026	0.9846 ± 0.0028	0.9504 ± 0.0096	0.9505 ± 0.0095	**0.9802 ± 0.0070**
vehicle3	0.3867 ± 0.0185	0.7500 ± 0.0181	0.6696 ± 0.0140	0.7573 ± 0.0252	0.7737 ± 0.0082	0.7783 ± 0.0238	0.7625 ± 0.0146	0.5923 ± 0.0072	0.6979 ± 0.0259	0.7804 ± 0.0090	0.7981 ± 0.0033	**0.8426 ± 0.0155**
glass0123vs456	0.8411 ± 0.0310	0.9222 ± 0.0038	0.8978 ± 0.0353	0.9289 ± 0.0154	0.9478 ± 0.0135	0.9611 ± 0.0019	0.9478 ± 0.0107	0.8844 ± 0.0150	0.9544 ± 0.0096	**1.0000 ± 0.0000**	0.9600 ± 0.0054	0.9811 ± 0.0019
ecoli1	0.7845 ± 0.0216	0.8863 ± 0.0186	0.8738 ± 0.0105	0.8863 ± 0.0105	0.8946 ± 0.0135	0.9161 ± 0.0064	0.8863 ± 0.0129	0.8732 ± 0.0047	0.8857 ± 0.0047	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9119 ± 0.0209
yeast3	0.6809 ± 0.0059	0.8629 ± 0.0133	0.7832 ± 0.0300	0.8543 ± 0.0088	0.8832 ± 0.0066	0.8789 ± 0.0036	0.8707 ± 0.0127	0.8219 ± 0.0277	0.8608 ± 0.0122	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.8957 ± 0.0110
ecoli3	0.6111 ± 0.0268	0.8861 ± 0.0096	0.7667 ± 0.0583	0.8528 ± 0.0096	0.8667 ± 0.0083	0.8778 ± 0.0127	0.8639 ± 0.0347	0.7528 ± 0.0488	0.8861 ± 0.0096	**1.0000 ± 0.0000**	0.9429 ± 0.0090	0.9139 ± 0.0048
yeast2vs4	0.6556 ± 0.0096	0.8567 ± 0.0285	0.6933 ± 0.0208	0.8433 ± 0.0167	0.8500 ± 0.0203	0.8689 ± 0.0126	0.8633 ± 0.0186	0.8044 ± 0.0367	0.8233 ± 0.0318	0.8156 ± 0.0135	0.8278 ± 0.0192	**0.8756 ± 0.0107**
yeast05679vs4	0.3189 ± 0.0126	0.7744 ± 0.0287	0.5100 ± 0.1011	0.7744 ± 0.0096	0.7822 ± 0.0184	0.7678 ± 0.0107	0.7678 ± 0.0212	0.7067 ± 0.0153	0.7089 ± 0.0096	0.3782 ± 0.0092	**0.8600 ± 0.0000**	0.8267 ± 0.0115
vowel0	0.9667 ± 0.0111	**1.0000 ± 0.0000**	0.9667 ± 0.0111	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9667 ± 0.0111	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9926 ± 0.0064
glass016vs2	0.0000 ± 0.0000	0.6500 ± 0.1000	0.3667 ± 0.0764	0.4833 ± 0.1258	0.5833 ± 0.1155	0.6333 ± 0.0289	0.6500 ± 0.0500	0.1667 ± 0.0289	0.4667 ± 0.1041	0.6677 ± 0.0289	0.7100 ± 0.0500	**0.7500 ± 0.0866**
glass2	0.0167 ± 0.0289	0.6333 ± 0.0577	0.2167 ± 0.1041	0.3833 ± 0.0289	0.5833 ± 0.0577	0.5833 ± 0.0764	0.6167 ± 0.0764	0.1000 ± 0.0500	0.4500 ± 0.0000	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.8000 ± 0.1000
yeast1vs7	0.1333 ± 0.0333	0.6556 ± 0.0509	0.3222 ± 0.0509	0.6333 ± 0.0577	0.6333 ± 0.0333	0.7000 ± 0.0333	0.6889 ± 0.0192	0.3778 ± 0.1171	0.6000 ± 0.0333	0.2333 ± 0.0290	**1.0000 ± 0.0000**	0.7000 ± 0.0333
ecoli4	0.7667 ± 0.0289	0.8333 ± 0.0289	0.8000 ± 0.0000	0.8333 ± 0.0289	0.8333 ± 0.0289	0.8167 ± 0.0289	0.8333 ± 0.0289	0.8000 ± 0.0500	0.8333 ± 0.0289	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.8500 ± 0.0000
yeast1458vs7	0.0000 ± 0.0000	0.5889 ± 0.0509	0.2222 ± 0.1171	0.6111 ± 0.0385	0.5889 ± 0.0509	0.6222 ± 0.0770	0.5778 ± 0.0694	0.3556 ± 0.1388	0.4556 ± 0.0192	0.5111 ± 0.0770	0.5333 ± 0.0577	**0.6444 ± 0.0385**
yeast2vs8	0.5500 ± 0.0000	0.7500 ± 0.0000	0.5833 ± 0.0289	0.7500 ± 0.0000	0.7167 ± 0.0289	0.7667 ± 0.0289	0.7333 ± 0.0289	0.6667 ± 0.0577	0.6500 ± 0.0000	0.9200 ± 0.0218	**1.0000 ± 0.0000**	0.7833 ± 0.0289
yeast4	0.5114 ± 0.0020	0.7135 ± 0.0160	0.6796 ± 0.0101	0.7095 ± 0.0224	0.7339 ± 0.0112	0.7310 ± 0.0140	0.7108 ± 0.0019	0.6343 ± 0.0378	0.6464 ± 0.0106	0.7208 ± 0.0076	0.7313 ± 0.0255	**0.7887 ± 0.0140**
yeast5	0.6933 ± 0.0325	0.9383 ± 0.0104	0.7617 ± 0.0379	0.9383 ± 0.0104	0.9383 ± 0.0104	0.9383 ± 0.0104	0.9383 ± 0.0104	0.9217 ± 0.0333	0.9383 ± 0.0104	0.9117 ± 0.0153	0.9200 ± 0.0132	**0.9600 ± 0.0132**
yeast6	0.5361 ± 0.0347	0.7500 ± 0.0000	0.5472 ± 0.0536	0.7917 ± 0.0144	0.7917 ± 0.0144	0.7694 ± 0.0173	0.7778 ± 0.0048	0.6167 ± 0.0300	0.7306 ± 0.0241	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.8361 ± 0.0210
poker89vs5	0.0000 ± 0.0000	0.2278 ± 0.0096	0.0000 ± 0.0000	0.2722 ± 0.0385	0.2556 ± 0.0509	0.2389 ± 0.0255	0.2556 ± 0.0347	0.2389 ± 0.0192	0.1444 ± 0.0096	0.4159 ± 0.0945	0.4935 ± 0.0691	**0.5278 ± 0.0788**
poker8vs6	0.3167 ± 0.0577	0.9833 ± 0.0289	0.3500 ± 0.1000	0.8167 ± 0.0289	0.8833 ± 0.0289	0.9833 ± 0.0289	0.9833 ± 0.0289	0.8333 ± 0.0289	0.9667 ± 0.0577	0.8554 ± 0.0765	0.8592 ± 0.0731	**0.9833 ± 0.0289**
microRNA	0.3175 ± 0.0101	0.8374 ± 0.0292	0.4905 ± 0.0161	0.8203 ± 0.0232	0.9033 ± 0.0199	0.8425 ± 0.0150	0.8323 ± 0.0196	0.6769 ± 0.0150	0.7566 ± 0.0133	0.7814 ± 0.0156	0.7742 ± 0.0111	**0.8772 ± 0.0131**

Table 4
Recall results of twelve comparative algorithms on forty imbalanced data sets based on SVM classifier.

Dataset	ORI	SMOTE	SL-SMOTE	SN-SMOTE	MWMOTE	ADASYN	SMOTE-IPF	SMMO	Supervised_SMOTE	WK-SMOTE	SMOTE-CSELM	IW-SMOTE
banknote	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
seeds2v13	**0.9857 ± 0.0143**	**0.9857 ± 0.0143**	**0.9952 ± 0.0082**	**0.9857 ± 0.0143**	**0.9810 ± 0.0218**	**0.9857 ± 0.0143**	**0.9857 ± 0.0143**	**0.9857 ± 0.0143**	**0.9857 ± 0.0143**	**0.9857 ± 0.0143**	**0.9857 ± 0.0143**	**0.9857 ± 0.0143**
ILPD	0.1537 ± 0.0267	0.7945 ± 0.0032	0.8371 ± 0.0194	0.8126 ± 0.0138	0.8127 ± 0.0096	0.8224 ± 0.0073	0.8028 ± 0.0002	0.5114 ± 0.0229	0.7113 ± 0.0185	0.7054 ± 0.0031	0.7321 ± 0.0026	**0.8088 ± 0.0059**
credit card clients	0.3563 ± 0.0055	0.5571 ± 0.0039	0.5198 ± 0.0065	0.5495 ± 0.0025	0.4754 ± 0.0009	0.6273 ± 0.0039	0.5477 ± 0.0057	0.5102 ± 0.0055	0.4541 ± 0.0005	0.6133 ± 0.0013	0.6257 ± 0.0004	**0.6335 ± 0.0071**
mfeatmor01	0.9525 ± 0.0043	0.9592 ± 0.0014	0.9542 ± 0.0014	0.9592 ± 0.0014	0.9567 ± 0.0029	0.9750 ± 0.0000	0.9550 ± 0.0025	0.9583 ± 0.0052	0.9567 ± 0.0014	**0.9750 ± 0.0000**	0.9667 ± 0.0014	0.9742 ± 0.0052
segment1	0.9848 ± 0.0000	0.9848 ± 0.0000	0.9848 ± 0.0000	0.9848 ± 0.0000	0.9859 ± 0.0017	**1.0000 ± 0.0000**	0.9848 ± 0.0000	0.9909 ± 0.0000	0.9848 ± 0.0000	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
segmentation-grass	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
pageblocks2345	0.7333 ± 0.0037	0.9286 ± 0.0047	0.8643 ± 0.0082	0.9315 ± 0.0057	0.9214 ± 0.0036	0.9524 ± 0.0045	0.9369 ± 0.0045	0.9321 ± 0.0036	0.8798 ± 0.0010	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9619 ± 0.0010
mfeatmor0	0.9850 ± 0.0000	0.9850 ± 0.0000	0.9867 ± 0.0029	0.9900 ± 0.0000	0.9850 ± 0.0000	0.9900 ± 0.0000	0.9867 ± 0.0029	0.9850 ± 0.0000	0.9850 ± 0.0000	0.9950 ± 0.0029	0.9967 ± 0.0000	**0.9983 ± 0.0029**
ctg23	0.8918 ± 0.0009	0.8955 ± 0.0153	0.8883 ± 0.0094	0.8975 ± 0.0103	0.8932 ± 0.0205	0.9015 ± 0.0149	0.8973 ± 0.0054	0.8975 ± 0.0064	0.8976 ± 0.0124	0.9018 ± 0.0009	0.9155 ± 0.0153	**0.9300 ± 0.0190**
ctg15	0.5827 ± 0.0207	0.7595 ± 0.0549	0.6173 ± 0.0548	0.7500 ± 0.0632	0.7595 ± 0.0440	0.7542 ± 0.0340	0.7506 ± 0.0536	0.6720 ± 0.0550	0.7637 ± 0.0134	0.7495 ± 0.0549	0.7173 ± 0.0548	**0.8393 ± 0.0292**
letterA	0.4336 ± 0.0038	0.5289 ± 0.0121	0.4066 ± 0.0221	0.5333 ± 0.0180	0.5458 ± 0.0202	0.5404 ± 0.0177	0.5327 ± 0.0130	0.4168 ± 0.0066	0.5853 ± 0.0007	0.6289 ± 0.0121	0.7066 ± 0.0221	**0.7462 ± 0.0125**
abalone19	0.0000 ± 0.0000	0.3444 ± 0.0337	0.0222 ± 0.0192	0.3056 ± 0.0567	0.1889 ± 0.0096	0.3444 ± 0.0337	0.3444 ± 0.0337	0.2917 ± 0.0144	0.2833 ± 0.0083	0.9300 ± 0.0029	**0.9600 ± 0.0056**	0.7083 ± 0.0363
ecoli0vs1	0.9906 ± 0.0038	0.9835 ± 0.0043	0.9908 ± 0.0040	0.9905 ± 0.0041	0.9883 ± 0.0040	0.9625 ± 0.0039	0.9906 ± 0.0038	0.9929 ± 0.0000	0.9906 ± 0.0038	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
pima	0.5410 ± 0.0253	0.6532 ± 0.0097	0.6591 ± 0.0080	0.6320 ± 0.0283	0.6465 ± 0.0057	0.6628 ± 0.0283	0.6422 ± 0.0194	0.6629 ± 0.0056	0.6406 ± 0.0188	**0.8721 ± 0.0218**	0.8096 ± 0.0018	0.7513 ± 0.0087
iris0	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass0	0.7429 ± 0.0247	0.8476 ± 0.0218	0.8238 ± 0.0218	0.8381 ± 0.0297	0.8286 ± 0.0143	0.8619 ± 0.0218	0.8333 ± 0.0165	0.7857 ± 0.0143	0.8238 ± 0.0218	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.8810 ± 0.0082
yeast1	0.4142 ± 0.0054	0.7203 ± 0.0162	0.7102 ± 0.0277	0.7281 ± 0.0073	0.6939 ± 0.0118	0.7855 ± 0.0083	0.7272 ± 0.0071	0.6798 ± 0.0120	0.6255 ± 0.0035	0.7039 ± 0.0118	0.7155 ± 0.0083	**0.7282 ± 0.0037**
haberman	0.1917 ± 0.0419	0.4745 ± 0.0293	0.4282 ± 0.1045	0.4574 ± 0.0865	0.4546 ± 0.0712	0.4921 ± 0.0722	0.4708 ± 0.0127	0.4824 ± 0.0230	0.3829 ± 0.0354	**0.9733 ± 0.0054**	0.7772 ± 0.0045	0.5843 ± 0.0066
vehicle2	0.9634 ± 0.0081	0.9573 ± 0.0097	0.9696 ± 0.0113	0.9603 ± 0.0073	0.9649 ± 0.0116	0.9574 ± 0.0096	0.9605 ± 0.0095	0.9603 ± 0.0071	0.9618 ± 0.0027	0.9504 ± 0.0096	0.9505 ± 0.0095	**0.9725 ± 0.0049**
vehicle3	0.6634 ± 0.0148	0.7277 ± 0.0032	0.7670 ± 0.0185	0.7404 ± 0.0090	0.7481 ± 0.0033	0.7417 ± 0.0023	0.7245 ± 0.0104	0.6977 ± 0.0005	0.7060 ± 0.0195	0.7804 ± 0.0090	0.7981 ± 0.0033	**0.8209 ± 0.0249**
glass0123vs456	0.7922 ± 0.0126	0.8056 ± 0.0184	0.8111 ± 0.0139	0.8044 ± 0.0168	0.8178 ± 0.0192	0.8178 ± 0.0278	0.8056 ± 0.0184	0.8056 ± 0.0184	0.8311 ± 0.0077	**1.0000 ± 0.0000**	0.9600 ± 0.0054	0.9600 ± 0.0000
ecoli1	0.8060 ± 0.0179	0.8696 ± 0.0201	0.8577 ± 0.0209	0.8571 ± 0.0117	0.8619 ± 0.0052	0.8702 ± 0.0207	0.8488 ± 0.0173	0.8452 ± 0.0122	0.8536 ± 0.0109	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.8750 ± 0.0062
yeast3	0.7257 ± 0.0152	0.8322 ± 0.0075	0.8206 ± 0.0067	0.8442 ± 0.0180	0.8240 ± 0.0131	0.8585 ± 0.0111	0.8423 ± 0.0038	0.8154 ± 0.0105	0.8015 ± 0.0073	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9158 ± 0.0073
ecoli3	0.6722 ± 0.0502	0.7667 ± 0.0417	0.8000 ± 0.0300	0.7722 ± 0.0127	0.7722 ± 0.0127	0.7500 ± 0.0382	0.7972 ± 0.0127	0.7111 ± 0.0411	0.7889 ± 0.0502	**1.0000 ± 0.0000**	0.9429 ± 0.0090	0.8972 ± 0.0048
yeast2vs4	0.7122 ± 0.0280	0.7856 ± 0.0171	0.7411 ± 0.0069	0.7856 ± 0.0135	0.8178 ± 0.0192	0.7989 ± 0.0184	0.7911 ± 0.0214	0.7800 ± 0.0406	0.7978 ± 0.0252	0.8156 ± 0.0135	0.8278 ± 0.0192	**0.8889 ± 0.0222**
yeast05679vs4	0.4767 ± 0.0318	0.6267 ± 0.0384	0.6078 ± 0.0670	0.6389 ± 0.0324	0.6422 ± 0.0214	0.6356 ± 0.0389	0.6200 ± 0.0145	0.6011 ± 0.0102	0.6144 ± 0.0107	0.3782 ± 0.0092	**0.8600 ± 0.0000**	0.7589 ± 0.0051
vowel0	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass016vs2	0.0667 ± 0.0289	0.6000 ± 0.0500	0.6000 ± 0.1323	0.6000 ± 0.0500	0.6167 ± 0.0577	0.6667 ± 0.0289	0.6500 ± 0.0500	0.2333 ± 0.0577	0.6333 ± 0.0289	0.6677 ± 0.0289	0.7100 ± 0.0500	**0.7500 ± 0.0866**
glass2	0.1667 ± 0.0764	0.5667 ± 0.0289	0.4833 ± 0.0764	0.5667 ± 0.0289	0.5833 ± 0.0289	0.6167 ± 0.0764	0.5833 ± 0.0289	0.3167 ± 0.0764	0.5667 ± 0.0289	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.8000 ± 0.0866
yeast1vs7	0.4333 ± 0.0000	0.5444 ± 0.0509	0.4444 ± 0.0385	0.5111 ± 0.0192	0.5333 ± 0.0333	0.5222 ± 0.0192	0.5333 ± 0.0333	0.4444 ± 0.0962	0.5333 ± 0.0333	0.2333 ± 0.0290	**1.0000 ± 0.0000**	0.6556 ± 0.0192
ecoli4	0.8000 ± 0.0000	0.8000 ± 0.0000	0.8000 ± 0.0000	0.8000 ± 0.0000	0.8000 ± 0.0000	0.8000 ± 0.0000	0.8000 ± 0.0000	0.8000 ± 0.0000	0.8000 ± 0.0000	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.8000 ± 0.0000
yeast1458vs7	0.0111 ± 0.0192	0.4222 ± 0.0694	0.0889 ± 0.0839	0.4667 ± 0.0577	0.4111 ± 0.0770	0.4333 ± 0.0577	0.4000 ± 0.0333	0.2222 ± 0.1895	0.3889 ± 0.0770	0.5111 ± 0.0770	0.5333 ± 0.0577	**0.5341 ± 0.0882**
yeast2vs8	0.4167 ± 0.0289	0.4000 ± 0.0866	0.3500 ± 0.1000	0.4333 ± 0.0764	0.4000 ± 0.1000	0.4000 ± 0.0866	0.4000 ± 0.0866	0.4000 ± 0.0500	0.5333 ± 0.0289	0.9200 ± 0.0218	**1.0000 ± 0.0000**	0.5167 ± 0.1041
yeast4	0.4888 ± 0.0093	0.7208 ± 0.0043	0.6415 ± 0.0244	0.7196 ± 0.0160	0.7085 ± 0.0158	0.7689 ± 0.0088	0.7208 ± 0.0076	0.6353 ± 0.0255	0.6059 ± 0.0035	0.7208 ± 0.0076	0.7313 ± 0.0255	**0.8031 ± 0.0113**
yeast5	0.7150 ± 0.0507	0.8583 ± 0.0115	0.7267 ± 0.0252	0.8583 ± 0.0126	0.8817 ± 0.0153	0.8600 ± 0.0132	0.8433 ± 0.0029	0.8433 ± 0.0419	0.8500 ± 0.0260	0.9117 ± 0.0153	0.9200 ± 0.0132	**0.9500 ± 0.0087**
yeast6	0.3778 ± 0.0127	0.6833 ± 0.0300	0.4500 ± 0.0795	0.6750 ± 0.0289	0.7222 ± 0.0709	0.6861 ± 0.0096	0.6944 ± 0.0542	0.6000 ± 0.0417	0.6750 ± 0.0083	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.7611 ± 0.0241
poker89vs5	0.0500 ± 0.0441	0.1833 ± 0.0167	0.1278 ± 0.0419	0.2056 ± 0.0419	0.1667 ± 0.0167	0.1611 ± 0.0255	0.1722 ± 0.0096	0.1722 ± 0.0419	0.1778 ± 0.0788	0.4159 ± 0.0945	**0.4935 ± 0.0691**	0.3667 ± 0.0333
poker8vs6	0.8000 ± 0.0500	0.8333 ± 0.0289	0.7167 ± 0.1041	0.7833 ± 0.0289	0.7833 ± 0.0289	0.8333 ± 0.0289	0.8333 ± 0.0289	0.7833 ± 0.0289	0.8333 ± 0.0289	0.8554 ± 0.0765	**0.8592 ± 0.0731**	0.7500 ± 0.0500
microRNA	0.6441 ± 0.0097	0.6441 ± 0.0147	0.6049 ± 0.0148	0.6461 ± 0.0221	0.6447 ± 0.0111	0.6442 ± 0.0125	0.6459 ± 0.0147	0.6493 ± 0.0178	0.6529 ± 0.0202	0.7814 ± 0.0156	0.7742 ± 0.0111	**0.7911 ± 0.0179**

Table 5
G-mean results of twelve comparative algorithms on forty imbalanced data sets based on CART classifier.

Dataset	ORI	SMOTE	SL-SMOTE	SN-SMOTE	MWMOTE	ADASYN	SMOTE-IPF	SMMO	Supervised_SMOTE	WK-SMOTE	SMOTE-CSELM	IW-SMOTE
banknote	0.9839 ± 0.0006	0.9835 ± 0.0012	0.9860 ± 0.0014	0.9840 ± 0.0008	0.9834 ± 0.0005	0.9835 ± 0.0008	0.9846 ± 0.0000	0.9829 ± 0.0005	0.9851 ± 0.0013	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
seeds2v13	0.9601 ± 0.0026	0.9817 ± 0.0002	0.9643 ± 0.0021	0.9830 ± 0.0040	0.9730 ± 0.0019	0.9660 ± 0.0022	0.9667 ± 0.0065	0.9465 ± 0.0085	0.9603 ± 0.0092	0.9821 ± 0.0106	0.9843 ± 0.0117	**0.9847 ± 0.0021**
ILPD	0.5890 ± 0.0416	0.5937 ± 0.0039	0.5740 ± 0.0059	0.5782 ± 0.0112	0.6035 ± 0.0264	0.5893 ± 0.0168	0.6069 ± 0.0229	0.6087 ± 0.0326	0.5871 ± 0.0244	0.6142 ± 0.0348	0.6532 ± 0.0123	**0.6741 ± 0.0190**
credit card clients	0.5805 ± 0.0085	0.6044 ± 0.0046	0.5949 ± 0.0015	0.5892 ± 0.0096	0.5915 ± 0.0042	0.5902 ± 0.0033	0.5931 ± 0.0046	0.5800 ± 0.0050	0.5910 ± 0.0051	0.6128 ± 0.0068	0.6059 ± 0.0087	**0.6676 ± 0.0033**
mfeatmor01	0.9595 ± 0.0019	0.9604 ± 0.0006	0.9611 ± 0.0009	0.9595 ± 0.0046	0.9611 ± 0.0020	0.9313 ± 0.0008	0.9608 ± 0.0041	0.9594 ± 0.0009	0.9668 ± 0.0011	**0.9692 ± 0.0026**	0.9594 ± 0.0016	0.9468 ± 0.0004
segment1	0.9857 ± 0.0013	0.9913 ± 0.0022	0.9836 ± 0.0012	0.9895 ± 0.0050	0.9893 ± 0.0013	0.9882 ± 0.0016	0.9916 ± 0.0011	0.9899 ± 0.0030	0.9894 ± 0.0033	0.9903 ± 0.0008	0.9921 ± 0.0009	**0.9937 ± 0.0001**
segmentation-grass	0.9878 ± 0.0106	0.9939 ± 0.0106	0.9878 ± 0.0106	0.9939 ± 0.0106	0.9878 ± 0.0106	0.9878 ± 0.0106	0.9939 ± 0.0106	0.9939 ± 0.0106	0.9816 ± 0.0000	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
pageblocks2345	0.9029 ± 0.0016	0.9232 ± 0.0043	0.9060 ± 0.0044	0.9203 ± 0.0020	0.9239 ± 0.0013	0.9170 ± 0.0077	0.9253 ± 0.0018	0.9037 ± 0.0055	0.9223 ± 0.0048	0.9100 ± 0.0070	0.9387 ± 0.0033	**0.9484 ± 0.0019**
mfeatmor0	0.9923 ± 0.0002	0.9929 ± 0.0011	0.9915 ± 0.0002	0.9913 ± 0.0019	0.9910 ± 0.0005	0.9931 ± 0.0002	0.9928 ± 0.0013	0.9923 ± 0.0002	0.9923 ± 0.0002	0.9929 ± 0.0028	0.9929 ± 0.0010	**0.9934 ± 0.0016**
ctg23	**0.9419 ± 0.0022**	0.9388 ± 0.0070	0.9406 ± 0.0029	0.9256 ± 0.0047	**0.9419 ± 0.0037**	0.9212 ± 0.0046	0.9386 ± 0.0009	0.9113 ± 0.0034	0.9342 ± 0.0013	0.9350 ± 0.0099	0.9411 ± 0.0108	0.9202 ± 0.0151
ctg15	0.7212 ± 0.0134	0.7430 ± 0.0180	0.7122 ± 0.0278	0.7419 ± 0.0200	0.7423 ± 0.0294	0.7654 ± 0.0281	0.7423 ± 0.0210	0.7163 ± 0.0629	0.7473 ± 0.0171	0.8101 ± 0.0219	0.8029 ± 0.0200	**0.8200 ± 0.0062**
letterA	0.7823 ± 0.0133	0.7826 ± 0.0027	0.7632 ± 0.0216	0.7619 ± 0.0158	0.7637 ± 0.0111	0.7644 ± 0.0135	0.7837 ± 0.0140	0.6978 ± 0.0056	0.7687 ± 0.0238	0.8245 ± 0.0118	0.8453 ± 0.0089	**0.9468 ± 0.0115**
abalone19	0.0384 ± 0.0333	0.2353 ± 0.0816	0.0793 ± 0.0714	0.2819 ± 0.0974	0.1115 ± 0.0047	0.2335 ± 0.1106	0.2007 ± 0.0764	0.1520 ± 0.0326	0.0714 ± 0.0310	0.7545 ± 0.0876	**0.7961 ± 0.0347**	0.7205 ± 0.0261
ecoli0vs1	0.9672 ± 0.0026	0.9668 ± 0.0103	0.9681 ± 0.0031	0.9685 ± 0.0006	0.9703 ± 0.0022	0.9628 ± 0.0048	0.9677 ± 0.0024	0.9748 ± 0.0026	0.9650 ± 0.0061	0.9704 ± 0.0068	**0.9762 ± 0.0046**	0.9704 ± 0.0051
pima	0.6848 ± 0.0086	0.6543 ± 0.0154	0.6542 ± 0.0127	0.6604 ± 0.0215	0.6676 ± 0.0057	0.6333 ± 0.0164	0.6684 ± 0.0072	0.6399 ± 0.0109	0.6533 ± 0.0101	0.7401 ± 0.0208	**0.7765 ± 0.0100**	0.7155 ± 0.0027
iris0	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass0	0.7821 ± 0.0124	0.7977 ± 0.0205	0.7318 ± 0.0140	0.7970 ± 0.0194	0.7923 ± 0.0317	0.7645 ± 0.0412	0.7643 ± 0.0169	0.7504 ± 0.0210	0.7826 ± 0.0099	0.7800 ± 0.0179	0.8251 ± 0.0136	**0.8275 ± 0.0166**
yeast1	0.6379 ± 0.0088	0.6489 ± 0.0161	0.6373 ± 0.0021	0.6457 ± 0.0219	0.6474 ± 0.0048	0.6394 ± 0.0077	0.6419 ± 0.0108	0.6485 ± 0.0074	0.6513 ± 0.0021	0.6473 ± 0.0155	0.6737 ± 0.0146	**0.6941 ± 0.0100**
haberman	0.4643 ± 0.0304	0.4734 ± 0.0238	0.5409 ± 0.0150	0.5102 ± 0.0034	0.5032 ± 0.0059	0.4755 ± 0.0263	0.4914 ± 0.0265	0.4865 ± 0.0289	0.4829 ± 0.0090	0.6522 ± 0.0237	0.6602 ± 0.0301	**0.6734 ± 0.0036**
vehicle2	0.9410 ± 0.0046	0.9453 ± 0.0067	0.9200 ± 0.0113	0.9495 ± 0.0019	0.9574 ± 0.0002	0.9554 ± 0.0001	0.9513 ± 0.0020	0.9553 ± 0.0003	0.9502 ± 0.0076	0.9925 ± 0.0097	0.9949 ± 0.0067	**0.9971 ± 0.0032**
vehicle3	0.6537 ± 0.0022	0.6793 ± 0.0060	0.6804 ± 0.0164	0.6678 ± 0.0071	0.6933 ± 0.0118	0.6852 ± 0.0041	0.6936 ± 0.0042	0.6462 ± 0.0175	0.6749 ± 0.0178	0.7246 ± 0.0255	0.7506 ± 0.0113	**0.7576 ± 0.0033**
glass0123vs456	0.8932 ± 0.0117	0.9052 ± 0.0201	0.9210 ± 0.0281	0.9101 ± 0.0036	0.9000 ± 0.0270	0.8942 ± 0.0179	0.8979 ± 0.0100	0.9170 ± 0.0021	0.8971 ± 0.0585	0.8994 ± 0.0108	0.9194 ± 0.0172	**0.9404 ± 0.0002**
ecoli1	0.8244 ± 0.0028	0.8782 ± 0.0212	0.8317 ± 0.0005	0.8821 ± 0.0072	0.8727 ± 0.0390	0.8294 ± 0.0161	0.8478 ± 0.0426	0.8344 ± 0.0335	0.8514 ± 0.0130	0.8919 ± 0.0216	0.9018 ± 0.0092	**0.9065 ± 0.0020**
yeast3	0.8016 ± 0.0126	0.8521 ± 0.0006	0.8056 ± 0.0148	0.8377 ± 0.0149	0.8343 ± 0.0037	0.8599 ± 0.0201	0.8337 ± 0.0146	0.7970 ± 0.0061	0.8444 ± 0.0112	0.9144 ± 0.0099	0.9354 ± 0.0015	**0.9372 ± 0.0048**
ecoli3	0.6564 ± 0.0172	0.7947 ± 0.0049	0.7012 ± 0.0705	0.7786 ± 0.0479	0.8156 ± 0.0144	0.7704 ± 0.0403	0.8350 ± 0.0242	0.7583 ± 0.0619	0.7864 ± 0.0305	0.8852 ± 0.0520	0.9252 ± 0.0227	**0.9311 ± 0.0048**
yeast2vs4	0.8184 ± 0.0044	0.8177 ± 0.0054	0.8155 ± 0.0864	0.8471 ± 0.0367	0.8538 ± 0.0074	0.8577 ± 0.0179	0.8430 ± 0.0074	0.8196 ± 0.0121	0.8205 ± 0.0222	0.8000 ± 0.0195	0.9473 ± 0.0103	**0.9496 ± 0.0100**
yeast05679vs4	0.5562 ± 0.0792	0.6957 ± 0.0058	0.6911 ± 0.0055	0.6808 ± 0.0120	0.7221 ± 0.0011	0.6802 ± 0.0539	0.7046 ± 0.0170	0.5916 ± 0.0419	0.6699 ± 0.0272	0.7514 ± 0.0452	0.7814 ± 0.0108	**0.7942 ± 0.0211**
vowel0	0.9402 ± 0.0019	0.9729 ± 0.0057	0.9404 ± 0.0048	0.9670 ± 0.0091	0.9686 ± 0.0005	0.9610 ± 0.0049	0.9506 ± 0.0038	0.9639 ± 0.0083	0.9661 ± 0.0018	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass016vs2	0.2854 ± 0.0213	0.3806 ± 0.0266	0.3314 ± 0.0640	0.3850 ± 0.0465	0.4373 ± 0.1211	0.4043 ± 0.0051	0.4485 ± 0.0385	0.3947 ± 0.0015	0.3080 ± 0.0205	0.7900 ± 0.0687	**0.8568 ± 0.0824**	0.6854 ± 0.0504
glass2	0.2578 ± 0.0011	0.4922 ± 0.0574	0.1834 ± 0.2047	0.3961 ± 0.0014	0.4436 ± 0.0413	0.4639 ± 0.0782	0.5404 ± 0.0390	0.4116 ± 0.1265	0.3236 ± 0.1363	0.7907 ± 0.0620	0.8707 ± 0.0832	**0.8924 ± 0.0166**
yeast1vs7	0.5301 ± 0.0086	0.5806 ± 0.0736	0.3899 ± 0.0464	0.6289 ± 0.0548	0.5706 ± 0.0405	0.5784 ± 0.0490	0.6336 ± 0.0137	0.4426 ± 0.0746	0.6279 ± 0.0152	0.7600 ± 0.0822	0.7948 ± 0.0499	**0.8010 ± 0.0648**
ecoli4	0.8487 ± 0.0419	0.8278 ± 0.0805	0.7838 ± 0.0983	0.8041 ± 0.0237	0.8081 ± 0.1118	0.8470 ± 0.0480	0.8193 ± 0.0981	0.7318 ± 0.0233	0.7799 ± 0.0644	0.9221 ± 0.0297	**0.9834 ± 0.0379**	0.9358 ± 0.0313
yeast1458vs7	0.2431 ± 0.1606	0.2849 ± 0.0744	0.1978 ± 0.0202	0.2980 ± 0.0354	0.4085 ± 0.0447	0.3741 ± 0.0203	0.3740 ± 0.0648	0.1851 ± 0.0633	0.2771 ± 0.0938	0.6700 ± 0.1169	**0.6880 ± 0.0346**	0.5745 ± 0.0080
yeast2vs8	0.5446 ± 0.0621	0.6130 ± 0.0241	0.5197 ± 0.0422	0.5222 ± 0.0172	0.6002 ± 0.1228	0.5549 ± 0.0995	0.6513 ± 0.0394	0.4777 ± 0.0190	0.4969 ± 0.1006	0.6713 ± 0.0664	**0.8032 ± 0.0622**	0.5250 ± 0.0542
yeast4	0.6850 ± 0.0218	0.7091 ± 0.0121	0.6925 ± 0.0077	0.6756 ± 0.0008	0.6944 ± 0.0047	0.6650 ± 0.0251	0.6974 ± 0.0177	0.6466 ± 0.0202	0.6994 ± 0.0181	0.7030 ± 0.0124	0.7330 ± 0.0112	**0.7619 ± 0.0023**
yeast5	0.7882 ± 0.0232	0.8379 ± 0.0160	0.8257 ± 0.0356	0.8397 ± 0.0129	0.8528 ± 0.0032	0.8584 ± 0.0126	0.8411 ± 0.0249	0.8278 ± 0.0113	0.8107 ± 0.0173	0.8286 ± 0.0195	0.9386 ± 0.0081	**0.9610 ± 0.0059**
yeast6	0.6563 ± 0.0516	0.6831 ± 0.0097	0.5901 ± 0.0508	0.6991 ± 0.0232	0.7148 ± 0.0384	0.7141 ± 0.0361	0.7120 ± 0.0122	0.6376 ± 0.0003	0.6968 ± 0.0047	0.8701 ± 0.0431	**0.9054 ± 0.0264**	0.8255 ± 0.0336
poker89vs5	0.2738 ± 0.0411	0.0612 ± 0.0069	0.1272 ± 0.1212	0.3234 ± 0.1704	0.1263 ± 0.0001	0.1181 ± 0.0072	0.2696 ± 0.1636	0.1313 ± 0.1059	0.1463 ± 0.0656	0.3152 ± 0.0649	0.4059 ± 0.0945	**0.4433 ± 0.0266**
poker8vs6	0.1941 ± 0.0815	0.4597 ± 0.0173	0.1843 ± 0.0986	0.4963 ± 0.0748	0.3929 ± 0.0384	0.7009 ± 0.0173	0.4881 ± 0.0814	0.7879 ± 0.0403	0.7172 ± 0.0804	0.8372 ± 0.1082	0.8554 ± 0.0765	**0.8576 ± 0.1180**
microRNA	0.6933 ± 0.0224	0.7454 ± 0.0213	0.6868 ± 0.0202	0.7473 ± 0.0147	0.7600 ± 0.0133	0.7547 ± 0.0132	0.7575 ± 0.0137	0.6702 ± 0.0207	0.7559 ± 0.0188	0.8114 ± 0.0213	0.8214 ± 0.0156	**0.8299 ± 0.0083**

Table 6
G-mean results of twelve comparative algorithms on forty imbalanced data sets based on KNN classifier.

Dataset	ORI	SMOTE	SL-SMOTE	SN-SMOTE	MWMOTE	ADASYN	SMOTE-IPF	SMMO	Supervised_SMOTE	WK-SMOTE	SMOTE-CSELM	IW-SMOTE
banknote	0.9987 ± 0.0008	0.9987 ± 0.0028	0.9980 ± 0.0025	0.9987 ± 0.0018	0.9987 ± 0.0016	0.9987 ± 0.0022	0.9987 ± 0.0031	0.9987 ± 0.0024	0.9987 ± 0.0037	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
seeds2v13	0.9745 ± 0.0054	0.9709 ± 0.0083	0.9745 ± 0.0093	0.9780 ± 0.0078	0.9782 ± 0.0076	0.9706 ± 0.0084	0.9709 ± 0.0146	0.9817 ± 0.0058	0.9671 ± 0.0039	0.9821 ± 0.0106	0.9843 ± 0.0117	**0.9852 ± 0.0126**
ILPD	0.4600 ± 0.0178	0.5727 ± 0.0307	0.6240 ± 0.0090	0.5722 ± 0.0232	0.5860 ± 0.0107	0.5965 ± 0.0457	0.5890 ± 0.0282	0.5105 ± 0.0236	0.5662 ± 0.0154	0.6142 ± 0.0348	0.6532 ± 0.0123	**0.6635 ± 0.0166**
credit card clients	0.5696 ± 0.0041	0.6428 ± 0.0073	0.6321 ± 0.0086	0.6436 ± 0.0054	0.6486 ± 0.0049	0.6361 ± 0.0056	0.6438 ± 0.0064	0.5954 ± 0.0059	0.6447 ± 0.0052	0.6128 ± 0.0068	0.6059 ± 0.0087	**0.6525 ± 0.0046**
mfeatmor01	0.9680 ± 0.0042	0.9713 ± 0.0030	0.9691 ± 0.0037	0.9685 ± 0.0021	**0.9719 ± 0.0022**	0.9438 ± 0.0024	0.9701 ± 0.0027	0.9678 ± 0.0024	0.9709 ± 0.0029	0.9692 ± 0.0026	0.9594 ± 0.0016	0.9470 ± 0.0044
segment1	0.9880 ± 0.0029	0.9927 ± 0.0011	0.9909 ± 0.0042	0.9931 ± 0.0018	0.9937 ± 0.0025	0.9931 ± 0.0012	0.9929 ± 0.0022	0.9923 ± 0.0014	0.9921 ± 0.0033	0.9903 ± 0.0008	0.9921 ± 0.0009	**0.9938 ± 0.0009**
segmentation-grass	**1.0000 ± 0.0000**	0.9972 ± 0.0101	**1.0000 ± 0.0000**	0.9972 ± 0.0106	0.9972 ± 0.0101	**1.0000 ± 0.0000**	0.9972 ± 0.0101	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
pageblocks2345	0.8351 ± 0.0057	0.9198 ± 0.0047	0.8653 ± 0.0035	0.9206 ± 0.0062	0.9219 ± 0.0040	0.9248 ± 0.0041	0.9182 ± 0.0041	0.8890 ± 0.0063	0.9212 ± 0.0080	0.9100 ± 0.0070	0.9387 ± 0.0033	**0.9442 ± 0.0043**
mfeatmor0	0.9924 ± 0.0016	0.9941 ± 0.0002	0.9910 ± 0.0002	0.9936 ± 0.0008	0.9916 ± 0.0011	0.9933 ± 0.0003	0.9910 ± 0.0012	0.9919 ± 0.0002	0.9924 ± 0.0003	0.9929 ± 0.0028	0.9929 ± 0.0010	**0.9961 ± 0.0001**
ctg23	0.8167 ± 0.0080	0.9350 ± 0.0128	0.8381 ± 0.0092	0.9276 ± 0.0104	0.9349 ± 0.0117	0.9353 ± 0.0124	0.9347 ± 0.0127	0.9019 ± 0.0057	0.9382 ± 0.0039	0.9350 ± 0.0099	**0.9411 ± 0.0108**	0.9305 ± 0.0060
ctg15	0.2792 ± 0.0468	0.7673 ± 0.0272	0.3786 ± 0.0114	0.7737 ± 0.0358	0.8027 ± 0.0234	0.7885 ± 0.0430	0.7891 ± 0.0168	0.7143 ± 0.0236	0.7882 ± 0.0175	0.8101 ± 0.0219	0.8029 ± 0.0200	**0.8484 ± 0.0230**
letterA	0.3319 ± 0.0064	0.6961 ± 0.0206	0.3599 ± 0.0209	0.6869 ± 0.0147	0.7650 ± 0.0164	0.6961 ± 0.0175	0.6949 ± 0.0164	0.5077 ± 0.0193	0.7077 ± 0.0164	0.8245 ± 0.0118	0.8453 ± 0.0089	**0.8541 ± 0.0090**
abalone19	0.0000 ± 0.0000	0.4828 ± 0.0608	0.0000 ± 0.0000	0.3827 ± 0.0655	0.2143 ± 0.0798	0.4648 ± 0.1306	0.4281 ± 0.0962	0.2203 ± 0.0898	0.1281 ± 0.0737	0.7545 ± 0.0876	**0.7961 ± 0.0347**	0.7113 ± 0.0257
ecoli0vs1	0.9648 ± 0.0046	0.9753 ± 0.0030	0.9852 ± 0.0065	0.9815 ± 0.0068	0.9852 ± 0.0042	0.9641 ± 0.0083	0.9818 ± 0.0131	0.9852 ± 0.0060	0.9787 ± 0.0048	0.9704 ± 0.0068	0.9762 ± 0.0046	**0.9877 ± 0.0066**
pima	0.6827 ± 0.0098	0.7218 ± 0.0135	0.7162 ± 0.0055	0.7287 ± 0.0182	0.7257 ± 0.0093	0.7209 ± 0.0159	0.7123 ± 0.0106	0.7004 ± 0.0169	0.7469 ± 0.0116	0.7401 ± 0.0208	0.7765 ± 0.0100	**0.7811 ± 0.0213**
iris0	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass0	0.7941 ± 0.0250	0.8195 ± 0.0202	0.7923 ± 0.0208	0.8109 ± 0.0258	0.8215 ± 0.0249	0.7991 ± 0.0152	0.8174 ± 0.0167	0.7914 ± 0.0133	0.8149 ± 0.0248	0.7800 ± 0.0179	**0.8251 ± 0.0136**	0.7898 ± 0.0287
yeast1	0.5942 ± 0.0139	0.6902 ± 0.0076	0.6568 ± 0.0108	0.6779 ± 0.0103	0.6965 ± 0.0150	0.6941 ± 0.0104	0.6725 ± 0.0084	0.6752 ± 0.0104	0.6936 ± 0.0114	0.6473 ± 0.0155	0.6737 ± 0.0146	**0.7001 ± 0.0192**
haberman	0.3315 ± 0.0296	0.5392 ± 0.0250	0.5665 ± 0.0566	0.6033 ± 0.0274	0.5618 ± 0.0229	0.5800 ± 0.0208	0.5932 ± 0.0558	0.6002 ± 0.0258	0.5625 ± 0.0485	0.6522 ± 0.0237	**0.6602 ± 0.0301**	0.5871 ± 0.0439
vehicle2	0.9485 ± 0.0075	0.9458 ± 0.0053	0.9274 ± 0.0085	0.9478 ± 0.0079	0.9388 ± 0.0113	0.9392 ± 0.0084	0.9415 ± 0.0021	0.9532 ± 0.0057	0.9419 ± 0.0043	0.9925 ± 0.0097	**0.9949 ± 0.0067**	0.9563 ± 0.0102
vehicle3	0.5938 ± 0.0061	0.7285 ± 0.0207	0.6971 ± 0.0212	0.7276 ± 0.0189	0.7339 ± 0.0165	0.7378 ± 0.0200	0.7212 ± 0.0223	0.6809 ± 0.0125	0.7106 ± 0.0130	0.7246 ± 0.0255	0.7506 ± 0.0113	**0.7546 ± 0.0180**
glass0123vs456	0.8934 ± 0.0205	0.9371 ± 0.0117	0.9422 ± 0.0215	0.9255 ± 0.0183	0.9538 ± 0.0096	0.9450 ± 0.0199	0.9255 ± 0.0082	0.9178 ± 0.0074	0.9544 ± 0.0175	0.8994 ± 0.0108	0.9194 ± 0.0172	**0.9586 ± 0.0317**
ecoli1	0.8613 ± 0.0143	0.8838 ± 0.0307	0.8830 ± 0.0214	0.8666 ± 0.0136	0.8892 ± 0.0093	0.8707 ± 0.0237	0.8652 ± 0.0195	0.8710 ± 0.0097	0.8887 ± 0.0170	0.8919 ± 0.0216	0.9018 ± 0.0092	**0.9026 ± 0.0038**
yeast3	0.8238 ± 0.0052	0.8852 ± 0.0047	0.8484 ± 0.0132	0.8936 ± 0.0152	0.9019 ± 0.0051	0.8967 ± 0.0135	0.8989 ± 0.0170	0.8750 ± 0.0089	0.8959 ± 0.0188	0.9144 ± 0.0099	0.9354 ± 0.0015	**0.9419 ± 0.0100**
ecoli3	0.6973 ± 0.0636	0.8787 ± 0.0914	0.8081 ± 0.0641	0.8657 ± 0.0854	0.8789 ± 0.0356	0.8754 ± 0.0332	0.8805 ± 0.0431	0.8556 ± 0.0126	0.8804 ± 0.0320	0.8852 ± 0.0520	**0.9252 ± 0.0227**	0.8682 ± 0.0665
yeast2vs4	0.7976 ± 0.0234	0.8807 ± 0.0323	0.8014 ± 0.0287	0.8862 ± 0.0400	0.8859 ± 0.0213	0.8968 ± 0.0167	0.8934 ± 0.0458	0.8826 ± 0.0157	0.8664 ± 0.0217	0.8000 ± 0.0195	**0.9473 ± 0.0103**	0.8935 ± 0.0200
yeast05679vs4	0.4681 ± 0.0723	0.8063 ± 0.0226	0.5794 ± 0.0448	0.8123 ± 0.0718	0.7999 ± 0.0679	**0.8100 ± 0.0233**	0.7888 ± 0.0416	0.5992 ± 0.0323	0.7818 ± 0.0193	0.7514 ± 0.0452	0.7814 ± 0.0108	0.7935 ± 0.0282
vowel0	0.9880 ± 0.0061	0.9994 ± 0.0111	0.9830 ± 0.0030	0.9994 ± 0.0060	0.9983 ± 0.0066	0.9994 ± 0.0081	0.9994 ± 0.0119	0.9880 ± 0.0058	0.9989 ± 0.0059	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass016vs2	0.0000 ± 0.0000	0.5452 ± 0.0948	0.3225 ± 0.0941	0.4921 ± 0.1411	0.7380 ± 0.0808	0.7103 ± 0.0645	0.6119 ± 0.0766	0.2314 ± 0.0516	0.5057 ± 0.0843	0.7900 ± 0.0687	**0.8568 ± 0.0824**	0.6763 ± 0.1448
glass2	0.0000 ± 0.0000	0.6628 ± 0.1699	0.0000 ± 0.0000	0.5198 ± 0.0898	0.5834 ± 0.0507	0.6882 ± 0.0884	0.5627 ± 0.0728	0.1323 ± 0.1278	0.5013 ± 0.0639	0.7907 ± 0.0620	**0.8707 ± 0.0832**	0.6654 ± 0.1234
yeast1vs7	0.1971 ± 0.0706	0.7323 ± 0.0674	0.5221 ± 0.0585	0.7725 ± 0.0998	0.6986 ± 0.0514	0.7141 ± 0.0989	0.7377 ± 0.0500	0.5601 ± 0.0730	0.6838 ± 0.0632	0.7600 ± 0.0822	**0.7948 ± 0.0499**	0.7208 ± 0.0659
ecoli4	0.8094 ± 0.0373	0.8588 ± 0.0830	0.8567 ± 0.0710	0.8556 ± 0.0600	0.8540 ± 0.0168	0.8267 ± 0.0356	0.8571 ± 0.0304	0.8631 ± 0.0164	0.8620 ± 0.0448	0.9221 ± 0.0297	**0.9834 ± 0.0379**	0.8474 ± 0.0513
yeast1458vs7	0.0000 ± 0.0000	0.6392 ± 0.0360	0.4050 ± 0.0458	0.6481 ± 0.1062	0.6079 ± 0.0677	0.6560 ± 0.0746	0.6436 ± 0.0722	0.2634 ± 0.0897	0.5756 ± 0.0836	0.6700 ± 0.1169	**0.6880 ± 0.0346**	0.5874 ± 0.1133
yeast2vs8	0.6121 ± 0.0588	0.7652 ± 0.0736	0.6070 ± 0.0359	0.7882 ± 0.0900	0.7512 ± 0.0537	0.7485 ± 0.0593	0.7649 ± 0.0341	0.6858 ± 0.0712	0.6359 ± 0.0683	0.6713 ± 0.0664	**0.8032 ± 0.0622**	0.7528 ± 0.0674
yeast4	0.7010 ± 0.0129	0.7541 ± 0.0198	0.7259 ± 0.0221	0.7493 ± 0.0170	0.7533 ± 0.0105	0.7448 ± 0.0097	0.7482 ± 0.0121	0.7328 ± 0.0096	0.7541 ± 0.0205	0.7030 ± 0.0124	0.7330 ± 0.0112	**0.7561 ± 0.0238**
yeast5	0.8371 ± 0.0262	0.9499 ± 0.0243	0.8924 ± 0.0720	0.9499 ± 0.0797	0.9485 ± 0.0407	0.9485 ± 0.0439	0.9492 ± 0.0192	0.8928 ± 0.0076	0.9499 ± 0.0076	0.8286 ± 0.0195	0.9386 ± 0.0081	**0.9660 ± 0.0346**
yeast6	0.6904 ± 0.0568	0.8464 ± 0.0433	0.6872 ± 0.0558	0.8501 ± 0.0836	0.8280 ± 0.0582	0.8389 ± 0.0382	0.8449 ± 0.0467	0.7746 ± 0.0191	0.8260 ± 0.0181	0.8701 ± 0.0431	**0.9054 ± 0.0264**	0.8348 ± 0.0233
poker89vs5	0.0000 ± 0.0000	0.4305 ± 0.0717	0.1284 ± 0.0671	0.4754 ± 0.0308	0.4657 ± 0.1083	0.4305 ± 0.1373	0.4302 ± 0.0774	0.3752 ± 0.1323	0.3227 ± 0.0566	0.3152 ± 0.0649	0.4059 ± 0.0945	**0.6401 ± 0.0960**
poker8vs6	0.3121 ± 0.1086	**0.9948 ± 0.1146**	0.3121 ± 0.1097	0.9080 ± 0.1278	0.9368 ± 0.1118	0.9938 ± 0.0299	0.9945 ± 0.0770	0.9093 ± 0.0998	0.9962 ± 0.0921	0.8372 ± 0.1082	0.8554 ± 0.0765	0.9449 ± 0.0457
microRNA	0.5559 ± 0.0249	0.8383 ± 0.0104	0.6759 ± 0.0242	0.8386 ± 0.0174	0.8506 ± 0.0211	0.8424 ± 0.0137	0.8393 ± 0.0135	0.7743 ± 0.0085	0.8336 ± 0.0182	0.8114 ± 0.0213	0.8214 ± 0.0156	**0.8470 ± 0.0131**

Table 7
G-mean results of twelve comparative algorithms on forty imbalanced data sets based on SVM classifier.

Dataset	ORI	SMOTE	SL-SMOTE	SN-SMOTE	MWMOTE	ADASYN	SMOTE-IPF	SMMO	Supervised_SMOTE	WK-SMOTE	SMOTE-CSELM	IW-SMOTE
banknote	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9993 ± 0.0000	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
seeds2v13	0.9738 ± 0.0077	0.9738 ± 0.0077	0.9763 ± 0.0080	0.9738 ± 0.0077	0.9763 ± 0.0047	0.9775 ± 0.0077	0.9738 ± 0.0077	0.9738 ± 0.0077	0.9726 ± 0.0081	0.9821 ± 0.0106	0.9843 ± 0.0117	**0.9852 ± 0.0060**
ILPD	0.3437 ± 0.0228	0.6574 ± 0.0133	0.6582 ± 0.0108	0.6698 ± 0.0083	0.6605 ± 0.0160	0.6638 ± 0.0118	0.6661 ± 0.0163	0.5618 ± 0.0094	0.6551 ± 0.0229	0.6142 ± 0.0348	0.6532 ± 0.0123	**0.6730 ± 0.0133**
credit card clients	0.5821 ± 0.0006	0.6744 ± 0.0004	0.6697 ± 0.0036	0.6797 ± 0.0012	0.6531 ± 0.0011	0.6711 ± 0.0024	0.6753 ± 0.0023	0.6067 ± 0.0066	0.6463 ± 0.0013	0.6128 ± 0.0068	0.6059 ± 0.0087	**0.6875 ± 0.0010**
mfeatmor01	0.9680 ± 0.0014	0.9712 ± 0.0002	0.9710 ± 0.0019	0.9715 ± 0.0023	0.9720 ± 0.0015	0.9402 ± 0.0023	**0.9727 ± 0.0014**	0.9713 ± 0.0008	0.9708 ± 0.0013	0.9692 ± 0.0026	0.9594 ± 0.0016	0.9453 ± 0.0031
segment1	0.9921 ± 0.0000	0.9924 ± 0.0009	0.9921 ± 0.0000	0.9924 ± 0.0009	0.9919 ± 0.0000	0.9842 ± 0.0004	0.9919 ± 0.0000	0.9926 ± 0.0009	0.9919 ± 0.0002	0.9903 ± 0.0008	0.9921 ± 0.0009	**0.9957 ± 0.0012**
segmentation-grass	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9991 ± 0.0016	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
pageblocks2345	0.8519 ± 0.0024	0.9380 ± 0.0006	0.9080 ± 0.0110	0.9386 ± 0.0010	0.9412 ± 0.0037	0.9403 ± 0.0020	0.9384 ± 0.0008	0.9383 ± 0.0031	0.9266 ± 0.0014	0.9100 ± 0.0070	0.9387 ± 0.0033	**0.9421 ± 0.0027**
mfeatmor0	0.9921 ± 0.0000	0.9928 ± 0.0014	0.9923 ± 0.0017	**0.9935 ± 0.0027**	0.9912 ± 0.0002	0.9923 ± 0.0016	0.9912 ± 0.0000	0.9919 ± 0.0001	0.9920 ± 0.0001	0.9929 ± 0.0028	0.9929 ± 0.0010	**0.9931 ± 0.0019**
ctg23	0.9411 ± 0.0057	0.9464 ± 0.0035	0.9340 ± 0.0070	0.9414 ± 0.0027	0.9363 ± 0.0025	0.9437 ± 0.0027	0.9435 ± 0.0062	0.9396 ± 0.0054	0.9447 ± 0.0036	0.9350 ± 0.0099	0.9411 ± 0.0108	**0.9525 ± 0.0028**
ctg15	0.7654 ± 0.0067	0.8458 ± 0.0248	0.7649 ± 0.0296	0.8510 ± 0.0235	0.8440 ± 0.0140	0.8480 ± 0.0233	0.8457 ± 0.0268	0.8164 ± 0.0220	0.8545 ± 0.0270	0.8101 ± 0.0219	0.8029 ± 0.0200	**0.8722 ± 0.0209**
letterA	0.6535 ± 0.0123	0.7258 ± 0.0072	0.6355 ± 0.0196	0.7331 ± 0.0106	0.7335 ± 0.0080	0.7334 ± 0.0086	0.7309 ± 0.0070	0.6426 ± 0.0042	0.7737 ± 0.0075	0.8245 ± 0.0118	0.8453 ± 0.0089	**0.8485 ± 0.0171**
abalone19	0.0000 ± 0.0000	0.5252 ± 0.0247	0.0000 ± 0.0000	0.5371 ± 0.0153	0.3360 ± 0.0165	0.5506 ± 0.0361	0.5323 ± 0.0121	0.5107 ± 0.0364	0.4564 ± 0.0713	0.7545 ± 0.0876	**0.7961 ± 0.0347**	0.7685 ± 0.0223
ecoli0vs1	0.9697 ± 0.0073	0.9742 ± 0.0048	0.9584 ± 0.0148	0.9706 ± 0.0016	0.9719 ± 0.0055	0.9468 ± 0.0125	0.9744 ± 0.0046	0.9733 ± 0.0064	**0.9776 ± 0.0057**	0.9704 ± 0.0068	0.9762 ± 0.0046	0.9697 ± 0.0073
pima	0.6954 ± 0.0177	0.7085 ± 0.0122	0.7056 ± 0.0115	0.7090 ± 0.0150	0.7185 ± 0.0074	0.6997 ± 0.0159	0.7103 ± 0.0040	0.6925 ± 0.0113	0.7134 ± 0.0048	0.7401 ± 0.0208	**0.7765 ± 0.0100**	0.7458 ± 0.0156
iris0	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass0	0.7975 ± 0.0174	0.7865 ± 0.0135	0.7882 ± 0.0040	0.7965 ± 0.0134	0.7899 ± 0.0094	0.7842 ± 0.0103	0.7946 ± 0.0035	0.8130 ± 0.0128	0.7855 ± 0.0243	0.7800 ± 0.0179	**0.8251 ± 0.0136**	0.8068 ± 0.0177
yeast1	0.6163 ± 0.0031	0.7050 ± 0.0094	0.7082 ± 0.0050	0.7049 ± 0.0079	0.7029 ± 0.0076	0.7002 ± 0.0044	0.7040 ± 0.0064	0.7029 ± 0.0147	0.6995 ± 0.0032	0.6473 ± 0.0155	0.6737 ± 0.0146	**0.7185 ± 0.0041**
haberman	0.3936 ± 0.0307	0.5662 ± 0.0364	0.5622 ± 0.0176	0.5771 ± 0.0187	0.5626 ± 0.0156	0.5865 ± 0.0228	0.5792 ± 0.0075	0.5751 ± 0.0110	0.5537 ± 0.0138	0.6522 ± 0.0237	**0.6602 ± 0.0301**	0.6248 ± 0.0120
vehicle2	0.9804 ± 0.0008	0.9784 ± 0.0017	0.9558 ± 0.0098	0.9778 ± 0.0023	0.9770 ± 0.0004	0.9762 ± 0.0037	0.9776 ± 0.0043	0.9791 ± 0.0016	0.9770 ± 0.0016	0.9925 ± 0.0097	0.9949 ± 0.0067	**0.9993 ± 0.0004**
vehicle3	0.7891 ± 0.0022	0.8085 ± 0.0157	0.7984 ± 0.0114	0.8056 ± 0.0128	**0.8099 ± 0.0179**	0.8078 ± 0.0068	0.8063 ± 0.0145	0.7961 ± 0.0119	0.7996 ± 0.0274	0.7246 ± 0.0255	0.7506 ± 0.0113	0.7963 ± 0.0087
glass0123vs456	0.8725 ± 0.0123	0.8905 ± 0.0092	0.8835 ± 0.0110	0.8686 ± 0.0094	0.8739 ± 0.0146	0.8861 ± 0.0170	0.8780 ± 0.0177	0.8778 ± 0.0177	0.8781 ± 0.0102	0.8994 ± 0.0108	0.9194 ± 0.0172	**0.9427 ± 0.0115**
ecoli1	0.8526 ± 0.0102	0.8700 ± 0.0081	0.8713 ± 0.0164	0.8633 ± 0.0028	0.8795 ± 0.0046	0.8679 ± 0.0066	0.8619 ± 0.0041	0.8658 ± 0.0034	0.8879 ± 0.0052	0.8919 ± 0.0216	**0.9018 ± 0.0092**	0.8720 ± 0.0065
yeast3	0.8499 ± 0.0072	0.8867 ± 0.0089	0.8908 ± 0.0010	0.8903 ± 0.0063	0.8834 ± 0.0097	0.8867 ± 0.0047	0.8848 ± 0.0109	0.8644 ± 0.0061	0.8801 ± 0.0063	0.9144 ± 0.0099	**0.9354 ± 0.0015**	0.9121 ± 0.0037
ecoli3	0.7415 ± 0.0621	0.8285 ± 0.0431	0.8327 ± 0.0162	0.8086 ± 0.0249	0.8438 ± 0.0068	0.8324 ± 0.0140	0.8136 ± 0.0303	0.8122 ± 0.0499	0.8178 ± 0.0411	0.8852 ± 0.0520	**0.9252 ± 0.0227**	0.8896 ± 0.0039
yeast2vs4	0.8159 ± 0.0234	0.8665 ± 0.0071	0.8255 ± 0.0129	0.8621 ± 0.0210	0.8613 ± 0.0128	0.8537 ± 0.0096	0.8543 ± 0.0080	0.8495 ± 0.0145	0.8592 ± 0.0061	0.8000 ± 0.0195	**0.9473 ± 0.0103**	0.8945 ± 0.0084
yeast05679vs4	0.6651 ± 0.0035	0.7324 ± 0.0110	0.6909 ± 0.0036	0.7504 ± 0.0156	0.7210 ± 0.0329	0.7424 ± 0.0243	0.7342 ± 0.0054	0.7227 ± 0.0389	0.7273 ± 0.0163	0.7514 ± 0.0452	0.7814 ± 0.0108	**0.7914 ± 0.0184**
vowel0	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9981 ± 0.0003	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass016vs2	0.0922 ± 0.0409	0.6204 ± 0.0563	0.4533 ± 0.0634	0.5966 ± 0.0852	0.6405 ± 0.0509	0.6286 ± 0.0479	0.6328 ± 0.0712	0.2426 ± 0.0868	0.6154 ± 0.0588	0.7900 ± 0.0687	**0.8568 ± 0.0824**	0.6826 ± 0.0076
glass2	0.2256 ± 0.0499	0.7060 ± 0.0442	0.5411 ± 0.0746	0.6413 ± 0.0444	0.6974 ± 0.0461	0.6961 ± 0.0468	0.6961 ± 0.0485	0.3661 ± 0.0584	0.6658 ± 0.0103	0.7907 ± 0.0620	**0.8707 ± 0.0832**	0.7958 ± 0.0136
yeast1vs7	0.5783 ± 0.0521	0.5486 ± 0.0241	0.4864 ± 0.0133	0.5968 ± 0.0408	0.5895 ± 0.0528	0.5477 ± 0.0677	0.5734 ± 0.0611	0.5658 ± 0.0427	0.5956 ± 0.0527	0.7600 ± 0.0822	**0.7948 ± 0.0499**	0.6951 ± 0.0305
ecoli4	0.8804 ± 0.0024	0.8712 ± 0.0021	0.8742 ± 0.0051	0.8707 ± 0.0029	0.8722 ± 0.0012	0.8707 ± 0.0029	0.8723 ± 0.0010	0.8758 ± 0.0052	0.8754 ± 0.0028	0.9221 ± 0.0297	**0.9834 ± 0.0379**	0.8814 ± 0.0187
yeast1458vs7	0.0385 ± 0.0333	0.4582 ± 0.0598	0.2128 ± 0.0960	0.5663 ± 0.0492	0.5193 ± 0.0076	0.4798 ± 0.0626	0.5066 ± 0.0334	0.3085 ± 0.0698	0.4605 ± 0.0437	0.6700 ± 0.1169	**0.6880 ± 0.0346**	0.5397 ± 0.1048
yeast2vs8	0.4958 ± 0.0233	0.5316 ± 0.0998	0.5148 ± 0.0371	0.5784 ± 0.0560	0.5235 ± 0.0773	0.5561 ± 0.0608	0.5298 ± 0.0993	0.5110 ± 0.0633	0.5386 ± 0.0239	0.6713 ± 0.0664	**0.8032 ± 0.0622**	0.6141 ± 0.0763
yeast4	0.6938 ± 0.0032	**0.7768 ± 0.0079**	0.7553 ± 0.0065	0.7749 ± 0.0063	0.7722 ± 0.0051	0.7613 ± 0.0039	0.7756 ± 0.0080	0.7448 ± 0.0217	0.7530 ± 0.0056	0.7030 ± 0.0124	0.7330 ± 0.0112	**0.7640 ± 0.0062**
yeast5	0.8372 ± 0.0069	0.9163 ± 0.0132	0.8579 ± 0.0453	0.9217 ± 0.0160	0.9207 ± 0.0154	0.9163 ± 0.0135	0.9166 ± 0.0138	0.9028 ± 0.0122	0.9171 ± 0.0127	0.8286 ± 0.0195	0.9386 ± 0.0081	0.9566 ± 0.0022
yeast6	0.6112 ± 0.0478	0.7532 ± 0.0297	0.6972 ± 0.0464	0.7363 ± 0.0165	0.7631 ± 0.0313	0.7509 ± 0.0251	0.7594 ± 0.0225	0.7408 ± 0.0344	0.7711 ± 0.0129	0.8701 ± 0.0431	**0.9054 ± 0.0264**	0.8130 ± 0.0267
poker89vs5	0.0576 ± 0.0576	0.1926 ± 0.0138	0.0890 ± 0.0437	0.2244 ± 0.0269	0.2483 ± 0.0314	0.2161 ± 0.0359	0.1927 ± 0.0139	0.1851 ± 0.0700	0.2401 ± 0.0988	0.3152 ± 0.0649	0.4059 ± 0.0945	**0.4065 ± 0.0691**
poker8vs6	0.8162 ± 0.0355	0.8788 ± 0.0577	0.8061 ± 0.0213	0.8397 ± 0.0514	0.8259 ± 0.0514	0.8788 ± 0.0577	0.8788 ± 0.0577	0.8162 ± 0.0355	**0.8886 ± 0.0408**	0.8372 ± 0.1082	0.8554 ± 0.0765	0.8592 ± 0.0731
microRNA	0.7889 ± 0.0154	0.7876 ± 0.0155	0.7724 ± 0.0019	0.7866 ± 0.0106	0.7834 ± 0.0022	0.7921 ± 0.0106	0.7887 ± 0.0138	0.7857 ± 0.0070	0.7986 ± 0.0152	0.8114 ± 0.0213	0.8214 ± 0.0156	**0.8742 ± 0.0111**

Table 8
AUC results of twelve comparative algorithms on forty imbalanced data sets based on CART classifier.

Dataset	ORI	SMOTE	SL-SMOTE	SN-SMOTE	MWMOTE	ADASYN	SMOTE-IPF	SMMO	Supervised_SMOTE	WK-SMOTE	SMOTE-CSELM	IW-SMOTE
banknote	0.9855 ± 0.0018	0.9848 ± 0.0009	0.9852 ± 0.0008	0.9872 ± 0.0026	0.9866 ± 0.0012	0.9850 ± 0.0016	0.9846 ± 0.0028	0.9874 ± 0.0012	0.9834 ± 0.0028	0.9981 ± 0.0014	0.9975 ± 0.0014	**0.9984 ± 0.0007**
seeds2v13	0.9690 ± 0.0055	0.9738 ± 0.0021	0.9760 ± 0.0092	0.9679 ± 0.0129	0.9726 ± 0.0074	0.9655 ± 0.0109	0.9679 ± 0.0062	0.9583 ± 0.0144	0.9750 ± 0.0036	0.9689 ± 0.0045	0.9832 ± 0.0219	**0.9847 ± 0.0000**
ILPD	0.5739 ± 0.0175	0.6360 ± 0.0114	0.6063 ± 0.0049	0.6222 ± 0.0247	0.6045 ± 0.0235	0.5981 ± 0.0061	0.6329 ± 0.0355	0.6076 ± 0.0252	0.6099 ± 0.0106	0.6542 ± 0.0348	0.6623 ± 0.0123	**0.6656 ± 0.0257**
credit card clients	0.6156 ± 0.0047	0.6205 ± 0.0034	0.6103 ± 0.0052	0.6185 ± 0.0035	0.6209 ± 0.0074	0.6032 ± 0.0044	0.6150 ± 0.0042	0.6098 ± 0.0038	0.6193 ± 0.0012	0.6830 ± 0.0096	0.6775 ± 0.0096	**0.6789 ± 0.0055**
mfeatmor01	0.9581 ± 0.0038	0.9597 ± 0.0039	0.9575 ± 0.0013	0.9591 ± 0.0033	0.9583 ± 0.0041	0.9337 ± 0.0011	0.9569 ± 0.0030	0.9582 ± 0.0016	0.9640 ± 0.0027	**0.9588 ± 0.0026**	0.9494 ± 0.0016	0.9454 ± 0.0034
segment1	0.9850 ± 0.0008	0.9900 ± 0.0047	0.9884 ± 0.0033	0.9914 ± 0.0026	0.9901 ± 0.0044	0.9878 ± 0.0016	0.9923 ± 0.0025	0.9864 ± 0.0016	0.9895 ± 0.0015	0.9903 ± 0.0008	0.9921 ± 0.0009	**0.9924 ± 0.0015**
segmentation-grass	0.9889 ± 0.0096	0.9944 ± 0.0096	0.9889 ± 0.0096	0.9889 ± 0.0096	0.9944 ± 0.0096	1.0000 ± 0.0000	0.9889 ± 0.0096	0.9944 ± 0.0096	0.9889 ± 0.0096	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
pageblocks2345	0.9071 ± 0.0027	0.9237 ± 0.0032	0.9157 ± 0.0025	0.9257 ± 0.0069	0.9259 ± 0.0029	0.9287 ± 0.0078	0.9256 ± 0.0049	0.9144 ± 0.0089	0.9267 ± 0.0033	0.9633 ± 0.0094	0.9582 ± 0.0094	**0.9685 ± 0.0034**
mfeatmor0	0.9923 ± 0.0003	0.9917 ± 0.0013	0.9919 ± 0.0005	0.9929 ± 0.0013	0.9914 ± 0.0003	0.9935 ± 0.0003	0.9923 ± 0.0011	0.9917 ± 0.0005	0.9923 ± 0.0003	0.9929 ± 0.0028	0.9929 ± 0.0010	**0.9948 ± 0.0014**
ctg23	0.9478 ± 0.0016	0.9340 ± 0.0079	0.9477 ± 0.0062	0.9382 ± 0.0035	0.9387 ± 0.0071	0.9227 ± 0.0097	0.9275 ± 0.0034	0.9225 ± 0.0106	**0.9544 ± 0.0051**	0.9350 ± 0.0099	0.9411 ± 0.0108	0.9465 ± 0.0015
ctg15	0.7666 ± 0.0046	0.7822 ± 0.0231	0.7399 ± 0.0361	0.7856 ± 0.0082	0.7873 ± 0.0155	0.7864 ± 0.0329	0.7891 ± 0.0156	0.7644 ± 0.0114	0.7721 ± 0.0295	0.7328 ± 0.0424	0.8128 ± 0.0424	**0.8248 ± 0.0190**
letterA	0.8036 ± 0.0014	0.7972 ± 0.0079	0.7816 ± 0.0031	0.7980 ± 0.0086	0.7823 ± 0.0175	0.7903 ± 0.0042	0.7961 ± 0.0128	0.7503 ± 0.0108	0.8200 ± 0.0068	0.8700 ± 0.0113	0.9065 ± 0.0113	**0.9486 ± 0.0011**
abalone19	0.5203 ± 0.0234	0.5797 ± 0.0369	0.5271 ± 0.0316	0.5558 ± 0.0612	0.5471 ± 0.0166	0.5706 ± 0.0089	0.5754 ± 0.0277	0.5377 ± 0.0184	0.5153 ± 0.0089	0.7768 ± 0.0483	0.8266 ± 0.0483	**0.8280 ± 0.0114**
ecoli0vs1	0.9747 ± 0.0038	0.9692 ± 0.0051	0.9677 ± 0.0088	0.9729 ± 0.0087	0.9723 ± 0.0047	0.9697 ± 0.0030	0.9679 ± 0.0045	0.9662 ± 0.0051	0.9705 ± 0.0037	0.9904 ± 0.0068	0.9960 ± 0.0046	**0.9988 ± 0.0042**
pima	0.6776 ± 0.0224	0.6740 ± 0.0205	0.6787 ± 0.0094	0.6646 ± 0.0055	0.6797 ± 0.0200	0.6573 ± 0.0043	0.6636 ± 0.0173	0.6561 ± 0.0103	0.6787 ± 0.0076	0.7921 ± 0.0122	**0.8149 ± 0.0122**	0.7227 ± 0.0091
iris0	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass0	0.7902 ± 0.0052	0.7953 ± 0.0146	0.7825 ± 0.0151	0.8080 ± 0.0138	0.7997 ± 0.0277	0.8044 ± 0.0220	0.8201 ± 0.0059	0.7575 ± 0.0148	0.7976 ± 0.0208	**0.8981 ± 0.0301**	0.8508 ± 0.0301	0.8515 ± 0.0039
yeast1	0.6471 ± 0.0027	0.6544 ± 0.0157	0.6526 ± 0.0050	0.6550 ± 0.0111	0.6811 ± 0.0134	0.6526 ± 0.0143	0.6658 ± 0.0106	0.6557 ± 0.0111	0.6604 ± 0.0034	0.6473 ± 0.0155	0.6737 ± 0.0146	**0.6966 ± 0.0066**
haberman	0.5476 ± 0.0318	0.5643 ± 0.0089	0.5449 ± 0.0166	0.5642 ± 0.0389	0.5692 ± 0.0153	0.5806 ± 0.0062	0.5750 ± 0.0229	0.5623 ± 0.0082	0.5793 ± 0.0112	0.6721 ± 0.0338	**0.7077 ± 0.0338**	0.6451 ± 0.0190
vehicle2	0.9435 ± 0.0007	0.9560 ± 0.0055	0.9406 ± 0.0081	0.9523 ± 0.0021	0.9502 ± 0.0017	0.9577 ± 0.0034	0.9534 ± 0.0073	0.9455 ± 0.0061	0.9513 ± 0.0037	0.9980 ± 0.0031	0.9972 ± 0.0031	**0.9987 ± 0.0080**
vehicle3	0.6629 ± 0.0067	0.6885 ± 0.0102	0.7072 ± 0.0072	0.6717 ± 0.0160	0.6944 ± 0.0066	0.6874 ± 0.0196	0.6899 ± 0.0174	0.6835 ± 0.0253	0.7051 ± 0.0150	0.7246 ± 0.0255	0.7206 ± 0.0113	**0.7493 ± 0.0230**
glass0123vs456	0.9282 ± 0.0136	0.9037 ± 0.0241	0.9258 ± 0.0093	0.9184 ± 0.0195	0.9387 ± 0.0139	0.9058 ± 0.0283	0.8990 ± 0.0225	0.9087 ± 0.0168	0.9332 ± 0.0104	**0.9888 ± 0.0108**	0.9833 ± 0.0172	0.9308 ± 0.0136
ecoli1	0.8339 ± 0.0207	0.8556 ± 0.0143	0.8422 ± 0.0105	0.8689 ± 0.0099	0.8596 ± 0.0141	0.8478 ± 0.0136	0.8522 ± 0.0270	0.8544 ± 0.0303	0.8631 ± 0.0016	0.8919 ± 0.0216	0.9018 ± 0.0092	**0.9073 ± 0.0006**
yeast3	0.8237 ± 0.0025	0.8564 ± 0.0060	0.8574 ± 0.0033	0.8643 ± 0.0195	0.8508 ± 0.0067	0.8618 ± 0.0008	0.8624 ± 0.0116	0.8440 ± 0.0146	0.8523 ± 0.0059	0.9514 ± 0.0107	**0.9705 ± 0.0107**	0.9308 ± 0.0032
ecoli3	0.7776 ± 0.0435	0.7327 ± 0.0276	0.7519 ± 0.0473	0.7462 ± 0.0153	0.7674 ± 0.0063	0.7929 ± 0.0468	0.7749 ± 0.0569	0.7818 ± 0.0289	0.7374 ± 0.0072	0.9224 ± 0.0690	**0.9450 ± 0.0690**	0.8519 ± 0.0085
yeast2vs4	0.8536 ± 0.0303	0.8335 ± 0.0219	0.8277 ± 0.0320	0.8604 ± 0.0335	0.8561 ± 0.0128	0.8709 ± 0.0131	0.8646 ± 0.0285	0.8636 ± 0.0245	0.8465 ± 0.0020	0.9342 ± 0.0208	0.9665 ± 0.0208	**0.9670 ± 0.0113**
yeast05679vs4	0.6861 ± 0.0131	0.7179 ± 0.0164	0.6772 ± 0.0393	0.7351 ± 0.0316	0.7288 ± 0.0400	0.7221 ± 0.0204	0.6998 ± 0.0147	0.6549 ± 0.0221	0.7133 ± 0.0240	0.7514 ± 0.0452	0.7814 ± 0.0108	**0.7972 ± 0.0197**
vowel0	0.9559 ± 0.0093	0.9629 ± 0.0016	0.9572 ± 0.0109	0.9459 ± 0.0023	0.9611 ± 0.0058	0.9511 ± 0.0084	0.9631 ± 0.0084	0.9650 ± 0.0053	0.9540 ± 0.0067	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass016vs2	0.5745 ± 0.0119	0.6755 ± 0.0442	0.6261 ± 0.0387	0.6321 ± 0.0338	0.6255 ± 0.0420	0.6447 ± 0.0866	0.6538 ± 0.1133	0.6100 ± 0.0518	0.6442 ± 0.0160	0.8352 ± 0.1083	**0.8807 ± 0.1083**	0.7220 ± 0.0176
glass2	0.5728 ± 0.0301	0.7119 ± 0.0448	0.6090 ± 0.0450	0.6604 ± 0.0727	0.6880 ± 0.0390	0.6788 ± 0.0504	0.7030 ± 0.0142	0.5996 ± 0.0386	0.6915 ± 0.0604	0.7900 ± 0.0718	**0.8644 ± 0.0718**	0.7484 ± 0.0433
yeast1vs7	0.6492 ± 0.0268	0.6776 ± 0.0221	0.6367 ± 0.0348	0.6212 ± 0.0787	0.6977 ± 0.0610	0.6415 ± 0.0131	0.7083 ± 0.0083	0.6718 ± 0.0475	0.6919 ± 0.0450	0.6473 ± 0.0155	0.6737 ± 0.0146	**0.7289 ± 0.0306**
ecoli4	0.8217 ± 0.0135	0.8749 ± 0.0166	0.8779 ± 0.0163	0.8916 ± 0.0274	0.8983 ± 0.0181	0.8629 ± 0.0266	0.8822 ± 0.0317	0.8426 ± 0.0235	0.8770 ± 0.0131	0.9642 ± 0.0243	**0.9969 ± 0.0243**	0.8801 ± 0.0146
yeast1458vs7	0.5234 ± 0.0042	0.5810 ± 0.0047	0.5549 ± 0.0592	0.5929 ± 0.0034	0.5744 ± 0.0376	0.5838 ± 0.0197	0.6010 ± 0.0144	0.5398 ± 0.0324	0.5158 ± 0.0284	0.7491 ± 0.1169	**0.7589 ± 0.0346**	0.6553 ± 0.0079
yeast2vs8	0.7167 ± 0.0022	0.7918 ± 0.0179	0.6857 ± 0.0019	0.7472 ± 0.0357	0.7839 ± 0.0327	0.7306 ± 0.0525	0.8081 ± 0.0138	0.7145 ± 0.0250	0.7573 ± 0.0147	0.8565 ± 0.0685	**0.8741 ± 0.0685**	0.7437 ± 0.0377
yeast4	0.7043 ± 0.0021	0.7185 ± 0.0080	0.7050 ± 0.0098	0.7186 ± 0.0144	0.7092 ± 0.0111	0.6987 ± 0.0014	0.7096 ± 0.0150	0.6942 ± 0.0058	0.7114 ± 0.0029	0.7030 ± 0.0124	0.7330 ± 0.0112	**0.7576 ± 0.0096**
yeast5	0.8361 ± 0.0254	0.8680 ± 0.0313	0.8388 ± 0.0606	0.8765 ± 0.0293	0.8715 ± 0.0200	0.8784 ± 0.0116	0.8709 ± 0.0244	0.8316 ± 0.0403	0.8523 ± 0.0115	0.8286 ± 0.0195	0.9386 ± 0.0081	**0.9666 ± 0.0013**
yeast6	0.7496 ± 0.0059	0.7561 ± 0.0122	0.7438 ± 0.0318	0.7548 ± 0.0110	0.7643 ± 0.0124	0.7590 ± 0.0283	0.7625 ± 0.0101	0.6963 ± 0.0229	0.7214 ± 0.0120	**0.9522 ± 0.0444**	0.9351 ± 0.0444	0.8458 ± 0.0115
poker89vs5	0.6052 ± 0.0132	0.5822 ± 0.0446	0.5628 ± 0.0329	0.5775 ± 0.0212	0.5621 ± 0.0500	0.6044 ± 0.0412	0.5674 ± 0.0533	0.5444 ± 0.0256	0.5501 ± 0.0224	0.6352 ± 0.0649	0.6559 ± 0.0945	**0.6572 ± 0.0162**
poker8vs6	0.5382 ± 0.0144	0.6890 ± 0.0377	0.5874 ± 0.0502	0.6172 ± 0.0644	0.6053 ± 0.1014	0.7724 ± 0.0490	0.7034 ± 0.0631	0.7654 ± 0.1023	0.7225 ± 0.0237	0.7423 ± 0.0523	0.7633 ± 0.0523	**0.7979 ± 0.0260**
microRNA	0.7382 ± 0.0021	0.7735 ± 0.0078	0.7263 ± 0.0139	0.7823 ± 0.0116	0.7785 ± 0.0039	0.7705 ± 0.0099	0.7746 ± 0.0088	0.7271 ± 0.0080	0.7759 ± 0.0029	0.8036 ± 0.0193	0.8305 ± 0.0011	**0.8329 ± 0.0007**

Table 9
AUC results of twelve comparative algorithms on forty imbalanced data sets based on KNN classifier.

Dataset	ORI	SMOTE	SL-SMOTE	SN-SMOTE	MWMOTE	ADASYN	SMOTE-IPF	SMMO	Supervised_SMOTE	WK-SMOTE	SMOTE-CSELM	IW-SMOTE
banknote	0.9975 ± 0.0014	0.9975 ± 0.0014	0.9975 ± 0.0014	0.9975 ± 0.0014	0.9975 ± 0.0014	0.9975 ± 0.0014	0.9975 ± 0.0014	0.9975 ± 0.0014	0.9975 ± 0.0014	**0.9981 ± 0.0014**	0.9975 ± 0.0014	0.9975 ± 0.0014
seeds2v13	0.9690 ± 0.0009	0.9668 ± 0.0024	0.9668 ± 0.0024	0.9763 ± 0.0016	0.9749 ± 0.0010	0.9723 ± 0.0039	0.9766 ± 0.0025	0.9630 ± 0.0042	0.9763 ± 0.0021	0.9689 ± 0.0045	0.9832 ± 0.0219	**0.9882 ± 0.0008**
ILPD	0.6241 ± 0.0097	0.6351 ± 0.0111	0.6125 ± 0.0201	0.6418 ± 0.0107	0.6393 ± 0.0177	0.6231 ± 0.0138	0.6329 ± 0.0141	0.6130 ± 0.0135	0.6403 ± 0.0170	0.6542 ± 0.0348	**0.6623 ± 0.0123**	0.6596 ± 0.0185
credit card clients	0.7021 ± 0.0009	0.6908 ± 0.0015	0.6805 ± 0.0064	0.6890 ± 0.0016	0.7037 ± 0.0010	0.6782 ± 0.0012	0.6906 ± 0.0034	0.6438 ± 0.0018	**0.7131 ± 0.0010**	0.6830 ± 0.0096	0.6775 ± 0.0096	0.7055 ± 0.0011
mfeatmor01	0.9814 ± 0.0002	0.9832 ± 0.0016	0.9823 ± 0.0010	0.9820 ± 0.0011	0.9842 ± 0.0016	0.9760 ± 0.0033	0.9819 ± 0.0008	0.9818 ± 0.0007	0.9824 ± 0.0003	0.9588 ± 0.0026	0.9494 ± 0.0016	**0.9852 ± 0.0011**
segment1	0.9908 ± 0.0009	0.9906 ± 0.0002	0.9903 ± 0.0010	0.9900 ± 0.0008	0.9917 ± 0.0010	0.9919 ± 0.0000	0.9916 ± 0.0003	0.9904 ± 0.0014	0.9904 ± 0.0018	0.9903 ± 0.0008	0.9921 ± 0.0009	**0.9926 ± 0.0000**
segmentation-grass	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
pageblocks2345	0.9474 ± 0.0032	0.9556 ± 0.0039	0.9454 ± 0.0013	0.9573 ± 0.0009	0.9695 ± 0.0022	0.9538 ± 0.0027	0.9540 ± 0.0020	0.9448 ± 0.0017	0.9677 ± 0.0008	0.9633 ± 0.0094	0.9582 ± 0.0094	**0.9709 ± 0.0023**
mfeatmor0	0.9922 ± 0.0014	0.9926 ± 0.0030	0.9922 ± 0.0014	0.9924 ± 0.0027	0.9926 ± 0.0026	0.9925 ± 0.0030	0.9928 ± 0.0029	0.9922 ± 0.0014	0.9922 ± 0.0014	0.9929 ± 0.0028	0.9929 ± 0.0010	**0.9949 ± 0.0028**
ctg23	0.9590 ± 0.0020	0.9637 ± 0.0003	0.9538 ± 0.0027	0.9615 ± 0.0008	0.9695 ± 0.0025	0.9643 ± 0.0019	0.9648 ± 0.0019	0.9575 ± 0.0014	**0.9704 ± 0.0030**	0.9350 ± 0.0099	0.9411 ± 0.0108	0.9696 ± 0.0014
ctg15	0.7936 ± 0.0085	0.8709 ± 0.0100	0.8038 ± 0.0256	0.8559 ± 0.0135	0.9039 ± 0.0074	0.8699 ± 0.0095	0.8675 ± 0.0058	0.8151 ± 0.0085	0.8737 ± 0.0058	0.7328 ± 0.0424	0.8128 ± 0.0424	**0.8817 ± 0.0123**
letterA	0.7057 ± 0.0028	0.7890 ± 0.0085	0.7052 ± 0.0021	0.7849 ± 0.0030	0.8407 ± 0.0066	0.7890 ± 0.0084	0.7890 ± 0.0084	0.7252 ± 0.0019	0.7979 ± 0.0057	0.8700 ± 0.0113	**0.9065 ± 0.0113**	0.8516 ± 0.0034
abalone19	0.5257 ± 0.0239	0.6731 ± 0.0280	0.5235 ± 0.0235	0.6478 ± 0.0069	0.5731 ± 0.0142	0.6747 ± 0.0259	0.6740 ± 0.0267	0.5682 ± 0.0144	0.5355 ± 0.0216	0.7768 ± 0.0483	**0.8266 ± 0.0483**	0.7767 ± 0.0059
ecoli0vs1	0.9861 ± 0.0001	0.9871 ± 0.0000	0.9853 ± 0.0011	0.9867 ± 0.0003	0.9872 ± 0.0003	0.9790 ± 0.0038	0.9866 ± 0.0004	0.9867 ± 0.0010	0.9888 ± 0.0035	0.9904 ± 0.0068	**0.9960 ± 0.0046**	0.9888 ± 0.0001
pima	0.7702 ± 0.0012	0.7636 ± 0.0029	0.7548 ± 0.0056	0.7667 ± 0.0071	0.7758 ± 0.0018	0.7480 ± 0.0022	0.7658 ± 0.0067	0.7336 ± 0.0100	0.7838 ± 0.0065	0.7921 ± 0.0122	**0.8149 ± 0.0122**	0.7853 ± 0.0013
iris0	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass0	0.8731 ± 0.0078	0.8741 ± 0.0088	0.8749 ± 0.0149	0.8732 ± 0.0050	0.8749 ± 0.0118	0.8721 ± 0.0017	0.8723 ± 0.0095	0.8698 ± 0.0089	0.8715 ± 0.0074	**0.8981 ± 0.0301**	0.8508 ± 0.0301	0.8655 ± 0.0055
yeast1	0.6579 ± 0.0093	0.6537 ± 0.0049	0.6371 ± 0.0076	0.6507 ± 0.0084	0.6597 ± 0.0100	0.6417 ± 0.0131	0.6501 ± 0.0088	0.6519 ± 0.0082	0.6735 ± 0.0086	0.6473 ± 0.0155	0.6737 ± 0.0146	**0.6768 ± 0.0050**
haberman	0.6135 ± 0.0188	0.6151 ± 0.0196	0.5804 ± 0.0124	0.6299 ± 0.0163	0.6489 ± 0.0278	0.6175 ± 0.0081	0.6169 ± 0.0315	0.5930 ± 0.0138	0.6418 ± 0.0200	0.6721 ± 0.0338	**0.7077 ± 0.0338**	0.6516 ± 0.0198
vehicle2	0.9915 ± 0.0005	0.9888 ± 0.0012	0.9855 ± 0.0017	0.9889 ± 0.0012	0.9904 ± 0.0007	0.9887 ± 0.0012	0.9895 ± 0.0011	0.9876 ± 0.0021	0.9910 ± 0.0011	**0.9980 ± 0.0031**	0.9972 ± 0.0031	0.9915 ± 0.0012
vehicle3	0.7050 ± 0.0092	0.7005 ± 0.0116	0.6899 ± 0.0033	0.7065 ± 0.0059	0.7025 ± 0.0046	0.7010 ± 0.0018	0.7048 ± 0.0052	0.7629 ± 0.0071	**0.7955 ± 0.0058**	0.7246 ± 0.0255	0.7206 ± 0.0113	0.7239 ± 0.0021
glass0123vs456	0.9694 ± 0.0052	0.9685 ± 0.0025	0.9669 ± 0.0048	0.9679 ± 0.0019	0.9735 ± 0.0008	0.9711 ± 0.0017	0.9687 ± 0.0011	0.9688 ± 0.0028	0.9729 ± 0.0040	**0.9888 ± 0.0108**	0.9833 ± 0.0172	0.9634 ± 0.0073
ecoli1	0.9391 ± 0.0075	0.9388 ± 0.0042	**0.9427 ± 0.0118**	0.9364 ± 0.0045	0.9325 ± 0.0035	0.9283 ± 0.0060	0.9363 ± 0.0026	0.9310 ± 0.0060	0.9396 ± 0.0076	0.8919 ± 0.0216	0.9018 ± 0.0092	0.9237 ± 0.0059
yeast3	0.9306 ± 0.0053	0.9360 ± 0.0018	0.9294 ± 0.0017	0.9331 ± 0.0060	0.9479 ± 0.0055	0.9326 ± 0.0024	0.9378 ± 0.0033	0.9316 ± 0.0059	0.9498 ± 0.0026	0.9514 ± 0.0107	**0.9705 ± 0.0107**	0.9308 ± 0.0053
ecoli3	0.9106 ± 0.0036	0.8946 ± 0.0053	0.8950 ± 0.0022	0.8866 ± 0.0054	0.9005 ± 0.0033	0.8840 ± 0.0084	0.8926 ± 0.0048	0.8996 ± 0.0034	0.8994 ± 0.0056	0.9224 ± 0.0690	**0.9450 ± 0.0690**	0.8910 ± 0.0009
yeast2vs4	0.9155 ± 0.0059	0.9330 ± 0.0062	0.9166 ± 0.0076	0.9278 ± 0.0076	0.9376 ± 0.0106	0.9359 ± 0.0027	0.9326 ± 0.0052	0.9294 ± 0.0069	0.9353 ± 0.0028	0.9342 ± 0.0208	0.9665 ± 0.0208	**0.9758 ± 0.0016**
yeast05679vs4	0.7033 ± 0.0106	0.7444 ± 0.0115	0.7324 ± 0.0032	0.7480 ± 0.0053	0.7538 ± 0.0127	0.7472 ± 0.0135	0.7475 ± 0.0120	0.7379 ± 0.0015	0.7470 ± 0.0060	0.7514 ± 0.0452	0.7814 ± 0.0108	**0.8003 ± 0.0089**
vowel0	0.9996 ± 0.0004	**1.0000 ± 0.0000**	0.9990 ± 0.0002	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9996 ± 0.0004	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass016vs2	0.6955 ± 0.0432	0.7715 ± 0.0382	0.7108 ± 0.0911	0.7494 ± 0.0521	0.7535 ± 0.0735	0.7822 ± 0.0118	0.7916 ± 0.0179	0.7014 ± 0.0373	0.7208 ± 0.0568	0.8352 ± 0.1083	**0.8807 ± 0.1083**	0.7549 ± 0.0136
glass2	0.6879 ± 0.0296	0.8149 ± 0.0183	0.6323 ± 0.0854	0.7432 ± 0.0204	0.7899 ± 0.0389	0.7959 ± 0.0217	0.8099 ± 0.0265	0.6818 ± 0.0246	0.7708 ± 0.0064	0.7900 ± 0.0718	**0.8644 ± 0.0718**	0.8072 ± 0.0263
yeast1vs7	0.7864 ± 0.0074	0.7770 ± 0.0045	0.7321 ± 0.0056	0.7791 ± 0.0091	0.7758 ± 0.0061	0.7746 ± 0.0176	0.7834 ± 0.0040	0.7724 ± 0.0094	**0.7895 ± 0.0037**	0.6473 ± 0.0155	0.6737 ± 0.0146	0.7655 ± 0.0097
ecoli4	0.9361 ± 0.0131	0.9340 ± 0.0135	0.9330 ± 0.0142	0.9335 ± 0.0126	0.9410 ± 0.0024	0.9287 ± 0.0128	0.9332 ± 0.0134	0.9343 ± 0.0122	0.9423 ± 0.0023	0.9642 ± 0.0243	**0.9969 ± 0.0243**	0.9295 ± 0.0129
yeast1458vs7	0.6793 ± 0.0128	0.7181 ± 0.0165	0.6740 ± 0.0181	0.7334 ± 0.0137	0.7040 ± 0.0156	0.7225 ± 0.0178	0.7113 ± 0.0164	0.6852 ± 0.0101	0.7006 ± 0.0134	0.7491 ± 0.1169	**0.7589 ± 0.0346**	0.6915 ± 0.0235
yeast2vs8	0.8118 ± 0.0030	0.8518 ± 0.0048	0.8135 ± 0.0119	0.8492 ± 0.0063	0.8550 ± 0.0156	0.8376 ± 0.0019	0.8491 ± 0.0119	0.8204 ± 0.0019	0.8132 ± 0.0006	0.8565 ± 0.0685	**0.8741 ± 0.0685**	0.8403 ± 0.0082
yeast4	**0.8198 ± 0.0092**	0.8087 ± 0.0131	0.8066 ± 0.0077	0.8064 ± 0.0151	0.8188 ± 0.0020	0.7991 ± 0.0083	0.8084 ± 0.0110	0.8057 ± 0.0141	0.8284 ± 0.0069	0.7030 ± 0.0124	0.7330 ± 0.0112	0.8164 ± 0.0085
yeast5	0.9701 ± 0.0014	0.9836 ± 0.0063	0.9721 ± 0.0069	0.9887 ± 0.0058	0.9871 ± 0.0066	0.9827 ± 0.0069	0.9837 ± 0.0066	0.9694 ± 0.0014	0.9874 ± 0.0059	0.8286 ± 0.0195	0.9386 ± 0.0081	**0.9880 ± 0.0055**
yeast6	0.8812 ± 0.0057	0.8759 ± 0.0085	0.8817 ± 0.0041	0.8787 ± 0.0136	0.8834 ± 0.0090	0.8715 ± 0.0109	0.8779 ± 0.0096	0.8844 ± 0.0143	0.8868 ± 0.0132	**0.9522 ± 0.0444**	0.9351 ± 0.0444	0.8865 ± 0.0123
poker89vs5	0.5683 ± 0.0128	0.6923 ± 0.0097	0.5653 ± 0.0121	0.6989 ± 0.0182	0.6795 ± 0.0071	0.6964 ± 0.0195	0.6974 ± 0.0190	0.6257 ± 0.0095	0.6131 ± 0.0423	0.6352 ± 0.0649	0.6559 ± 0.0945	**0.7438 ± 0.0450**
poker8vs6	0.9143 ± 0.0149	0.9907 ± 0.0146	0.9119 ± 0.0158	0.9385 ± 0.0134	0.9394 ± 0.0151	0.9906 ± 0.0148	0.9906 ± 0.0148	0.9481 ± 0.0004	**0.9907 ± 0.0152**	0.7423 ± 0.0523	0.7633 ± 0.0523	0.9649 ± 0.0173
microRNA	0.8392 ± 0.0041	0.8949 ± 0.0077	0.8351 ± 0.0050	0.8916 ± 0.0088	0.9055 ± 0.0052	0.8928 ± 0.0067	0.8948 ± 0.0063	0.8610 ± 0.0067	0.8852 ± 0.0095	0.8036 ± 0.0193	0.8305 ± 0.0011	**0.9008 ± 0.0041**

Table 10
AUC results of twelve comparative algorithms on forty imbalanced data sets based on SVM classifier.

Dataset	ORI	SMOTE	SL-SMOTE	SN-SMOTE	MWMOTE	ADASYN	SMOTE-IPF	SMMO	Supervised_SMOTE	WK-SMOTE	SMOTE-CSELM	IW-SMOTE
banknote	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.9981 ± 0.0014	0.9975 ± 0.0014	**1.0000 ± 0.0000**
seeds2v13	0.9783 ± 0.0006	0.9786 ± 0.0012	0.9759 ± 0.0027	0.9783 ± 0.0006	0.9783 ± 0.0010	0.9783 ± 0.0010	0.9783 ± 0.0006	0.9783 ± 0.0006	0.9766 ± 0.0012	0.9689 ± 0.0045	0.9832 ± 0.0219	**0.9889 ± 0.0027**
ILPD	0.5958 ± 0.0100	0.6421 ± 0.0025	0.6235 ± 0.0103	0.6439 ± 0.0049	0.6264 ± 0.0041	0.6469 ± 0.0118	0.6419 ± 0.0090	0.6484 ± 0.0107	0.6396 ± 0.0078	0.6542 ± 0.0348	0.6623 ± 0.0123	**0.7297 ± 0.0087**
credit card clients	0.6199 ± 0.0023	0.6409 ± 0.0007	0.6381 ± 0.0069	0.6401 ± 0.0008	0.6274 ± 0.0014	0.6308 ± 0.0003	0.6391 ± 0.0005	0.6042 ± 0.0002	0.6291 ± 0.0025	0.6830 ± 0.0096	0.6775 ± 0.0096	**0.7481 ± 0.0021**
mfeatmor01	0.9835 ± 0.0020	0.9858 ± 0.0017	0.9861 ± 0.0013	0.9870 ± 0.0025	0.9836 ± 0.0018	**0.9888 ± 0.0012**	0.9858 ± 0.0009	0.9849 ± 0.0004	0.9849 ± 0.0024	0.9588 ± 0.0026	0.9494 ± 0.0016	0.9860 ± 0.0032
segment1	0.9990 ± 0.0002	0.9993 ± 0.0001	0.9992 ± 0.0001	0.9994 ± 0.0001	0.9993 ± 0.0001	**0.9995 ± 0.0002**	0.9994 ± 0.0001	0.9992 ± 0.0001	0.9992 ± 0.0002	0.9903 ± 0.0008	0.9921 ± 0.0009	0.9998 ± 0.0001
segmentation-grass	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
pageblocks2345	0.9003 ± 0.0001	0.9597 ± 0.0007	0.9387 ± 0.0018	0.9593 ± 0.0005	0.9618 ± 0.0008	0.9579 ± 0.0006	0.9507 ± 0.0005	0.9581 ± 0.0005	0.9516 ± 0.0010	0.9633 ± 0.0094	0.9582 ± 0.0094	**0.9718 ± 0.0007**
mfeatmor0	0.9964 ± 0.0007	0.9967 ± 0.0017	0.9972 ± 0.0012	0.9965 ± 0.0021	0.9971 ± 0.0013	0.9977 ± 0.0004	0.9962 ± 0.0021	0.9974 ± 0.0022	0.9955 ± 0.0020	0.9929 ± 0.0028	0.9929 ± 0.0010	**0.9978 ± 0.0007**
ctg23	0.9340 ± 0.0008	0.9423 ± 0.0010	0.9295 ± 0.0014	0.9428 ± 0.0003	0.9417 ± 0.0015	0.9410 ± 0.0018	0.9424 ± 0.0005	0.9420 ± 0.0005	0.9443 ± 0.0006	0.9350 ± 0.0099	0.9411 ± 0.0108	**0.9543 ± 0.0011**
ctg15	0.7525 ± 0.0029	0.7755 ± 0.0010	0.7503 ± 0.0039	0.7765 ± 0.0041	0.7785 ± 0.0045	0.7754 ± 0.0017	0.7754 ± 0.0015	0.7794 ± 0.0091	0.7818 ± 0.0013	0.7328 ± 0.0424	0.8128 ± 0.0424	**0.8223 ± 0.0041**
letterA	0.9042 ± 0.0089	0.8853 ± 0.0058	0.8566 ± 0.0291	0.8855 ± 0.0098	0.8950 ± 0.0050	0.8813 ± 0.0057	0.8866 ± 0.0057	0.8641 ± 0.0137	0.9110 ± 0.0041	0.8700 ± 0.0113	0.9065 ± 0.0113	**0.9265 ± 0.0090**
abalone19	0.6942 ± 0.0101	0.7641 ± 0.0182	0.6289 ± 0.0515	0.7669 ± 0.0177	0.7284 ± 0.0096	0.7630 ± 0.0182	0.7646 ± 0.0192	0.7569 ± 0.0168	0.7634 ± 0.0123	0.7768 ± 0.0483	0.8266 ± 0.0483	**0.8442 ± 0.0061**
ecoli0vs1	0.9935 ± 0.0018	0.9939 ± 0.0026	0.9959 ± 0.0047	0.9916 ± 0.0043	0.9932 ± 0.0019	0.9821 ± 0.0051	0.9919 ± 0.0025	0.9919 ± 0.0021	0.9954 ± 0.0009	0.9904 ± 0.0068	**0.9960 ± 0.0046**	0.9935 ± 0.0018
pima	0.7778 ± 0.0050	0.7741 ± 0.0022	0.7682 ± 0.0068	0.7767 ± 0.0044	0.7758 ± 0.0029	0.7671 ± 0.0031	0.7732 ± 0.0057	0.7646 ± 0.0023	0.7868 ± 0.0057	0.7921 ± 0.0122	**0.8149 ± 0.0122**	0.8132 ± 0.0091
iris0	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass0	0.8459 ± 0.0173	0.8636 ± 0.0133	0.8472 ± 0.0226	0.8625 ± 0.0117	0.8474 ± 0.0129	0.8546 ± 0.0061	0.8646 ± 0.0099	0.8605 ± 0.0164	0.8589 ± 0.0148	**0.8981 ± 0.0301**	0.8508 ± 0.0301	0.8637 ± 0.0069
yeast1	0.6761 ± 0.0046	0.6805 ± 0.0033	0.6763 ± 0.0038	0.6788 ± 0.0066	0.6780 ± 0.0034	0.6769 ± 0.0035	0.6792 ± 0.0018	0.6699 ± 0.0066	0.6838 ± 0.0027	0.6473 ± 0.0155	0.6737 ± 0.0146	**0.6921 ± 0.0021**
haberman	0.5217 ± 0.0166	0.6177 ± 0.0130	0.5859 ± 0.0054	0.6101 ± 0.0470	0.6223 ± 0.0378	0.6188 ± 0.0202	0.6229 ± 0.0098	0.6096 ± 0.0291	0.6408 ± 0.0173	0.6721 ± 0.0338	**0.7077 ± 0.0338**	0.6826 ± 0.0158
vehicle2	0.9914 ± 0.0014	0.9966 ± 0.0015	0.9918 ± 0.0033	0.9966 ± 0.0014	0.9963 ± 0.0012	0.9961 ± 0.0014	0.9962 ± 0.0013	0.9965 ± 0.0014	0.9959 ± 0.0007	**0.9980 ± 0.0031**	0.9972 ± 0.0031	0.9943 ± 0.0022
vehicle3	**0.9066 ± 0.0038**	0.9002 ± 0.0003	0.8727 ± 0.0038	0.9030 ± 0.0023	0.9057 ± 0.0026	0.9000 ± 0.0056	0.8990 ± 0.0040	0.9016 ± 0.0015	0.9029 ± 0.0045	0.7246 ± 0.0255	0.7206 ± 0.0113	0.8694 ± 0.0071
glass0123vs456	0.9335 ± 0.0105	0.9406 ± 0.0095	0.9277 ± 0.0024	0.9396 ± 0.0025	0.9506 ± 0.0028	0.9419 ± 0.0056	0.9336 ± 0.0099	0.9417 ± 0.0032	0.9433 ± 0.0013	**0.9888 ± 0.0108**	0.9833 ± 0.0172	0.9799 ± 0.0042
ecoli1	0.8315 ± 0.0049	0.8512 ± 0.0100	0.8318 ± 0.0047	0.8472 ± 0.0075	0.8497 ± 0.0059	0.8494 ± 0.0061	0.8465 ± 0.0062	0.8507 ± 0.0039	0.8528 ± 0.0039	0.8919 ± 0.0216	0.9018 ± 0.0092	**0.9038 ± 0.0037**
yeast3	0.9461 ± 0.0041	0.9583 ± 0.0049	0.9511 ± 0.0042	0.9587 ± 0.0041	0.9604 ± 0.0042	0.9573 ± 0.0040	0.9600 ± 0.0033	0.9606 ± 0.0039	0.9640 ± 0.0032	0.9514 ± 0.0107	**0.9705 ± 0.0107**	0.9631 ± 0.0036
ecoli3	0.8578 ± 0.0019	0.9384 ± 0.0075	0.9336 ± 0.0028	0.9399 ± 0.0149	0.9385 ± 0.0093	0.9328 ± 0.0066	0.9419 ± 0.0105	**0.9456 ± 0.0112**	0.9431 ± 0.0078	0.9224 ± 0.0690	0.9450 ± 0.0690	0.9409 ± 0.0058
yeast2vs4	0.9250 ± 0.0019	0.9653 ± 0.0103	0.9648 ± 0.0032	0.9579 ± 0.0133	0.9498 ± 0.0194	0.9634 ± 0.0105	0.9622 ± 0.0086	0.9704 ± 0.0091	0.9746 ± 0.0042	0.9342 ± 0.0208	0.9665 ± 0.0208	**0.9819 ± 0.0024**
yeast05679vs4	0.7009 ± 0.0101	0.7753 ± 0.0066	0.7481 ± 0.0062	0.7650 ± 0.0124	0.7612 ± 0.0041	0.7669 ± 0.0092	0.7739 ± 0.0050	**0.7896 ± 0.0075**	0.7894 ± 0.0100	0.7514 ± 0.0452	0.7814 ± 0.0108	0.7690 ± 0.0031
vowel0	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**0.9998 ± 0.0004**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**
glass016vs2	0.5446 ± 0.0325	0.6763 ± 0.0190	0.6026 ± 0.0573	0.6673 ± 0.0208	0.6779 ± 0.0202	0.6801 ± 0.0148	0.6802 ± 0.0221	0.6651 ± 0.0187	0.6623 ± 0.0156	0.8352 ± 0.1083	**0.8807 ± 0.1083**	0.7222 ± 0.0480
glass2	0.8796 ± 0.0152	0.8960 ± 0.0074	0.7746 ± 0.0057	0.8776 ± 0.0150	**0.9079 ± 0.0088**	0.9029 ± 0.0061	0.8965 ± 0.0229	0.8664 ± 0.0287	0.8775 ± 0.0155	0.7900 ± 0.0718	0.8644 ± 0.0718	0.8585 ± 0.0345
yeast1vs7	0.6594 ± 0.0087	0.6492 ± 0.0049	0.6525 ± 0.0272	0.6510 ± 0.0023	0.6530 ± 0.0124	0.6436 ± 0.0131	0.6493 ± 0.0144	0.6758 ± 0.0091	**0.6898 ± 0.0163**	0.6473 ± 0.0155	0.6737 ± 0.0146	0.6748 ± 0.0071
ecoli4	0.9764 ± 0.0015	0.9712 ± 0.0036	0.9759 ± 0.0094	0.9606 ± 0.0057	0.9514 ± 0.0091	0.9690 ± 0.0048	0.9696 ± 0.0071	0.9670 ± 0.0057	0.9628 ± 0.0066	0.9642 ± 0.0243	**0.9969 ± 0.0243**	0.9670 ± 0.0048
yeast1458vs7	0.6483 ± 0.0121	0.6437 ± 0.0070	0.6237 ± 0.0601	0.6602 ± 0.0080	0.6502 ± 0.0035	0.6415 ± 0.0118	0.6381 ± 0.0034	0.6389 ± 0.0311	0.6688 ± 0.0185	0.7491 ± 0.1169	**0.7589 ± 0.0346**	0.6471 ± 0.0098
yeast2vs8	0.8179 ± 0.0039	0.6836 ± 0.0298	0.7788 ± 0.0192	0.6994 ± 0.0175	0.7186 ± 0.0252	0.6668 ± 0.0336	0.6760 ± 0.0237	0.7044 ± 0.0257	0.7875 ± 0.0207	0.8565 ± 0.0685	**0.8741 ± 0.0685**	0.7257 ± 0.0105
yeast4	**0.8605 ± 0.0054**	0.8522 ± 0.0017	0.8421 ± 0.0147	0.8520 ± 0.0017	0.8507 ± 0.0060	0.8388 ± 0.0075	0.8518 ± 0.0015	0.8446 ± 0.0122	0.8557 ± 0.0017	0.7030 ± 0.0124	0.7330 ± 0.0112	0.8537 ± 0.0033
yeast5	0.8849 ± 0.0008	0.8880 ± 0.0006	0.8847 ± 0.0036	0.9084 ± 0.0009	0.9078 ± 0.0001	0.9081 ± 0.0005	0.9079 ± 0.0004	0.9086 ± 0.0010	0.9076 ± 0.0010	0.8286 ± 0.0195	0.9386 ± 0.0081	**0.9904 ± 0.0007**
yeast6	0.8651 ± 0.0096	0.9280 ± 0.0080	0.8957 ± 0.0220	0.9184 ± 0.0142	0.9241 ± 0.0152	0.9220 ± 0.0126	0.9262 ± 0.0112	0.9066 ± 0.0249	0.9356 ± 0.0137	**0.9522 ± 0.0444**	0.9351 ± 0.0444	0.9214 ± 0.0076
poker89vs5	0.8314 ± 0.0172	0.8009 ± 0.0035	0.8193 ± 0.0208	0.8057 ± 0.0051	0.8307 ± 0.0343	0.8039 ± 0.0106	0.7832 ± 0.0018	**0.8344 ± 0.0163**	0.8282 ± 0.0436	0.6352 ± 0.0649	0.6559 ± 0.0945	0.7884 ± 0.0055
poker8vs6	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**0.9995 ± 0.0004**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	**1.0000 ± 0.0000**	0.7423 ± 0.0523	0.7633 ± 0.0523	0.9814 ± 0.0065
microRNA	0.7931 ± 0.0064	0.8540 ± 0.0035	0.8035 ± 0.0041	0.8097 ± 0.0045	0.8167 ± 0.0052	0.8205 ± 0.0032	0.8107 ± 0.0032	0.8539 ± 0.0062	0.8521 ± 0.0074	0.8036 ± 0.0193	0.8305 ± 0.0011	**0.8564 ± 0.0049**

(1) On most datasets, SMOTE and various SMOTE variants can effectively promote the classification performance compared with directly using the original skewed dataset (that is, ORI), which verifies the effectiveness of the SMOTE series algorithms again. Additionally, we note that in some datasets, such as iris0, segmentation-grass, and banknote, ORI can acquire comparative results compared with various SMOTE-based algorithms. Generally, these datasets have simple data distributions, which means that the data have a better separability; thus, CIL algorithms are useless for improving the classification quality on these datasets. This observation has also been reported in many previous studies [29,30,56].

(2) Surprisingly, compared with several complex SMOTE variants, except WK-SMOTE, SMOTE-ELM, and our proposed IW-SMOTE, the original SMOTE algorithm does not present a clearly poorer performance. We consider that although SMOTE fails to take advantage of prior data distribution information, it can still strengthen the dense regions and weaken the sparse regions by randomly selecting seed instances. Once these simple variants wrongly recognize the noise and borderline region, the sampling results would extremely bias the original data distribution, which decreases the quality of classification modeling. These results indicate the significance of accurately exploring the prior data distribution.

(3) The proposed IW-SMOTE algorithm presents an overwhelming advantage over the other SMOTE variants. In particular, the IW-SMOTE produces the best Recall results on 24, 23, and 22 datasets, the best G-mean results on 30, 23, and 20 datasets, the best AUC results on 26, 15, and 18 datasets using CART, KNN, and SVM as classifiers, respectively. The results demonstrated the effectiveness and superiority of the proposed algorithm. Moreover, it verifies the necessity of accurately estimating data distribution.

(4) As indicated in Section 3, data distribution estimation based on neighborhood calculations may be extremely inaccurate for severely skewed or complicated datasets. This deduction seems to be verified by the experimental results for datasets with high imbalance ratios, such as letterA, abalone19, poker89vs5, and microRNA, and the G-mean performance was significantly improved using the IW-SMOTE algorithms compared with the other algorithms. This observation illustrates that our proposed indirect data-distribution exploration strategy is robust and universal.

(5) Compared with SMMO and Supervised_SMOTE which also adopt indirect strategies to explore the distribution of skewed data, the IW-SMOTE algorithm shows an obviously better performance. We believe that this relates to the impartiality of estimation using supervised learning models, as it does not matter whether SMMO or Supervised_SMOTE directly trains the supervised learning model on original skewed data, thus providing feedback on biased distribution information. The IW-SMOTE algorithm uses an UnderBagging-alike model in which each sub-model is created on a totally balanced data subset to estimate the data distribution information, thereby reflecting the data distribution in a more accurate manner. Additionally, as indicated in Section 2, SMMO augments noise because it only generates new instances among those misclassified minority instances, while Supervised_SMOTE fails to strengthen the minority borderline because it generates minority instances in an extremely safe manner.

(6) Both WK-SMOTE and SMOTE-CSELM are superior to all other algorithms except IW-SMOTE, which certifies the superiority of integrating sampling techniques with cost-sensitive learning strategies for dealing with CIL problems. These two algorithms are classifier-dependent; hence, most of their success might be attributed to the employment of cost-sensitive SVM and class-specific ELM models.

(7) Although different classification results were acquired using different classifiers, the proposed IW-SMOTE algorithm outperformed all other algorithms regardless of the use of a single classifier. Therefore, we can say that the effectiveness of IW-SMOTE is irrelevant to the specific classification model, but only relates to sampling based on data distribution exploration.

5.4. Statistical analysis

To present a thorough understanding of the comparison of the various algorithms, we also provide their statistical results. We employed the Friedman ranking test and Holm post-hoc test [57,58] to differentiate the performance of the comparative methods on 40 datasets. The statistical analysis results were obtained at a confidence level of $\alpha = 0.05$, which are presented in Tables 11–13.

The statistical results in Tables 11–13 show that regardless of the use of the CART, KNN or SVM classifier, the proposed IW-SMOTE algorithm acquires the lowest average ranking value in any term of Recall, G-mean and AUC metrics, indicating that it is the best among the comparative algorithms. When using CART as a classifier, the Holm post-hoc test rejects all comparative algorithms, excluding WK-SMOTE and SMOTE-CSELM, which verifies that IW-SMOTE significantly outperforms these comparative algorithms. When we use SVM as a classifier, the Holm post-hoc test rejects nearly all hypotheses except those that relate to SMOTE-CSELM on all metrics and Supervised_SMOTE on the AUC metric. As for the KNN classifier, it rejects fewer hypotheses, but IW-SMOTE has a significantly lower average ranking on each metric than all other comparative algorithms. Of course, we cannot say that there exists a significant difference between IW-SMOTE and SMOTE-CSELM, although the former has a lower average ranking than the latter. Overall, the proposed IW-SMOTE algorithm presented superior classification performance and lower average ranking than all other competitors; hence, we can safely recommend it as an effective and efficient choice for oversampling skewed data.

5.5. Comparison with the underBagging-alike undersampling ensemble algorithm

Considering that the proposed UnderBagging-alike undersampling ensemble algorithm is not only a distribution information estimation tool, but can also be used to directly classify the instances, we further compare the performance of the IW-SMOTE algorithm with CART, KNN, and SVM. In particular, we counted the Win/Tie/Loss number of IW-SMOTE versus UnderBagging-alike based on the pairwise t-test at the 5% significance level, for all 40 data sets. Table 14 presented the results.

In Table 14, we observe that regardless of using any one classifier and any one performance metric, IW-SMOTE significantly outperforms UnderBagging-alike on most datasets, while performing significantly worse than the UnderBagging-alike algorithm on only a few datasets. Thus, we are curious why both algorithms adopt the same threshold parameter, but produce a completely different classification performance. We believe that this is related to the fineness of the borderline modeling. In other words, the UnderBagging-alike algorithm cannot handle the local detail around the boundary well, but IW-SMOTE can solve this problem by highlighting the borderline region with personalized oversampling.

Table 11
Statistical analysis results of comparative algorithms at the confidence level of $\alpha = 0.05$ based on CART classifier.

Algorithm	Recall		G-mean		AUC	
	Average ranking	Hypothesis	Average ranking	Hypothesis	Average ranking	Hypothesis
IW-SMOTE	2.2500	–	2.2250	–	2.3750	–
ORI	9.9375	Rejected	9.4750	Rejected	9.1375	Rejected
SMOTE	6.8375	Rejected	6.5375	Rejected	7.0500	Rejected
SL-SMOTE	8.9750	Rejected	9.2375	Rejected	9.1125	Rejected
SN-SMOTE	7.1625	Rejected	7.2125	Rejected	6.7500	Rejected
MWMOTE	6.7125	Rejected	6.3625	Rejected	6.5125	Rejected
ADASYN	6.9500	Rejected	7.7375	Rejected	7.4625	Rejected
SMOTE-IPF	6.5125	Rejected	6.0625	Rejected	6.6250	Rejected
SMMO	9.8375	Rejected	9.1125	Rejected	9.4500	Rejected
Supervised_SMOTE	7.9125	Rejected	7.9125	Rejected	7.0500	Rejected
WK-SMOTE	2.8500	Not rejected	3.8625	Not rejected	3.9000	Not rejected
SMOTE-CSELM	2.0625	Not rejected	2.2625	Not rejected	2.5750	Not rejected

Table 12
Statistical analysis results of comparative algorithms at the confidence level of $\alpha = 0.05$ based on KNN classifier.

Algorithm	Recall		G-mean		AUC	
	Average ranking	Hypothesis	Average ranking	Hypothesis	Average ranking	Hypothesis
IW-SMOTE	3.3375	–	3.5750	–	4.5250	–
ORI	11.238	Rejected	10.7875	Rejected	8.0625	Rejected
SMOTE	6.4250	Rejected	5.8125	Rejected	6.2375	Not rejected
SL-SMOTE	9.9000	Rejected	9.4875	Rejected	9.6750	Rejected
SN-SMOTE	6.7375	Rejected	6.0000	Rejected	6.7250	Rejected
MWMOTE	5.2000	Not rejected	5.4750	Rejected	4.7500	Not rejected
ADASYN	4.7875	Not rejected	5.9000	Rejected	7.4375	Rejected
SMOTE-IPF	6.0250	Rejected	6.2875	Rejected	6.1875	Not rejected
SMMO	8.9750	Rejected	8.4750	Rejected	8.7750	Rejected
Supervised_SMOTE	7.8000	Rejected	6.4750	Rejected	5.1125	Not rejected
WK-SMOTE	4.2125	Not rejected	5.8500	Rejected	5.6500	Not rejected
SMOTE-CSELM	3.3625	Not rejected	3.8750	Not rejected	4.8625	Not rejected

Table 13
Statistical analysis results of comparative algorithms at the confidence level of $\alpha = 0.05$ based on SVM classifier.

Algorithm	Recall		G-mean		AUC	
	Average ranking	Hypothesis	Average ranking	Hypothesis	Average ranking	Hypothesis
IW-SMOTE	3.1750	–	3.1750	–	3.9875	–
ORI	9.6650	Rejected	9.6650	Rejected	8.0750	Rejected
SMOTE	6.1500	Rejected	6.1500	Rejected	6.0125	Rejected
SL-SMOTE	8.8750	Rejected	8.8750	Rejected	8.8625	Rejected
SN-SMOTE	5.9150	Rejected	5.9150	Rejected	6.3375	Rejected
MWMOTE	6.4625	Rejected	6.4625	Rejected	6.4500	Rejected
ADASYN	6.7500	Rejected	6.7500	Rejected	7.1000	Rejected
SMOTE-IPF	6.2875	Rejected	6.2875	Rejected	6.6625	Rejected
SMMO	8.2875	Rejected	8.2875	Rejected	6.3750	Rejected
Supervised_SMOTE	6.8000	Rejected	6.8000	Rejected	5.2875	Not rejected
WK-SMOTE	5.8000	Rejected	5.8000	Rejected	7.6125	Rejected
SMOTE-CSELM	3.8375	Not rejected	3.8375	Not rejected	5.2375	Not rejected

Table 14
Statistical comparison between the IW-SMOTE and UnderBagging-alike based on pairwise t-test with $\alpha = 0.05$.

Classifier	Recall			G-mean			AUC		
	Win	Tie	Loss	Win	Tie	Loss	Win	Tie	Loss
CART	22	14	4	27	12	1	20	17	3
KNN	17	17	6	20	16	4	19	15	6
SVM	27	11	2	31	6	3	21	16	3

5.6. Discussion on the parameters

Three key parameters were associated with the quality of the proposed IW-SMOTE algorithm. The first is the noise interceptive threshold λ, the second is the size of the undersampling ensemble M, and the third is the sampling coefficient β. Considering that β is a self-adapting parameter, we seek out the influence rule for the two other parameters.

First, we detect the impact of the noise interceptive threshold λ. Fig. 6 presents its influence on three artificial data sets, which are used in Fig. 4 by varying the threshold λ from 0.1 to 0.9 with an increment of 0.2. Fig. 6 shows that with an increase in λ, noise filtering becomes increasingly difficult. In fact, the visual results in Fig. 6 indicate that regardless of whether the threshold λ is too small or too large, we cannot acquire an excellent data distribution. When λ is too small, the description of the boundary region is destroyed, whereas if λ is too large, we cannot find and remove the noise, which causes noise propagation during the following oversampling procedure.

To further verify the above deduction, we also designed a new group of experiments to observe the variation in classification performance with the variance of parameter λ. In particular, we

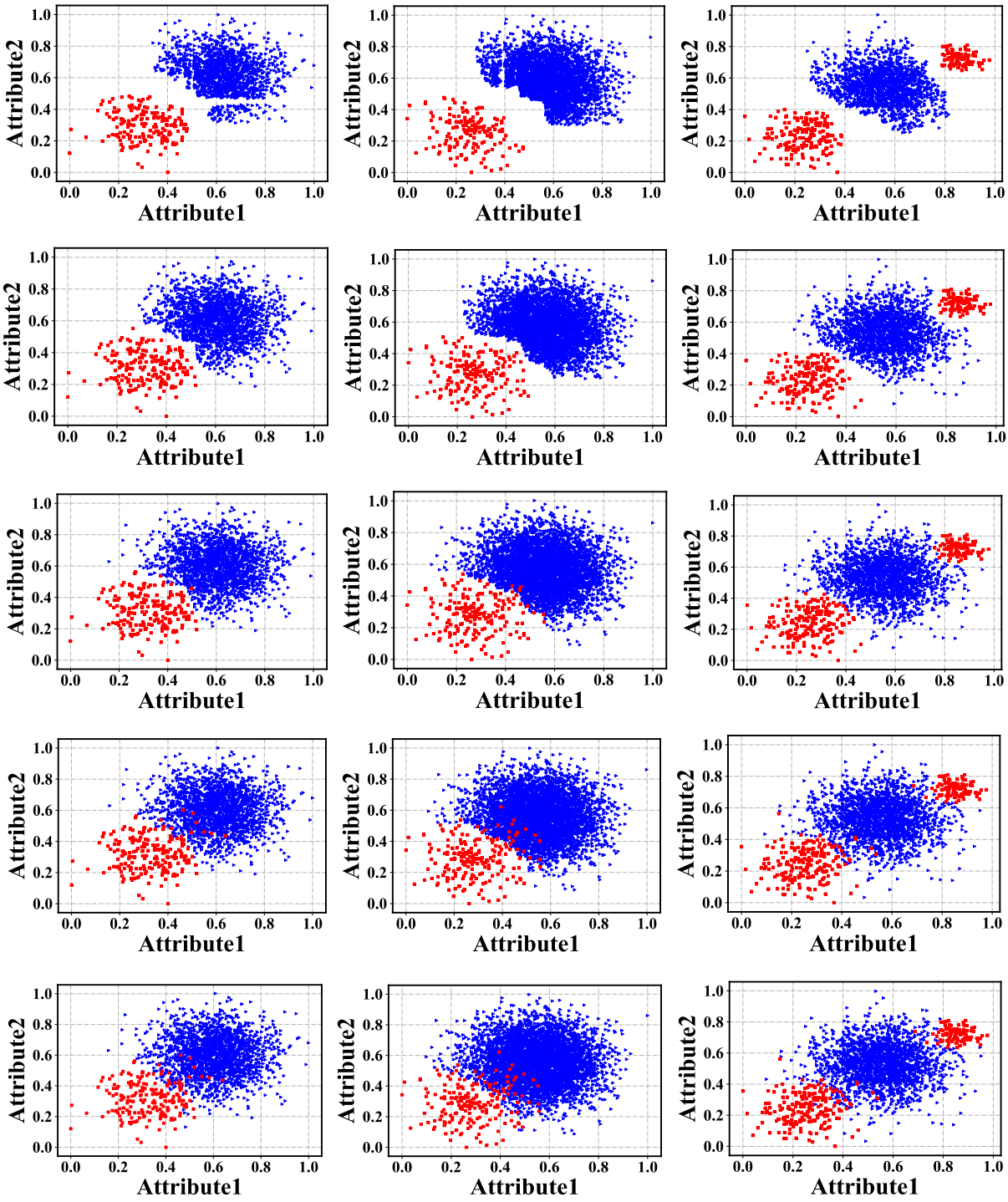


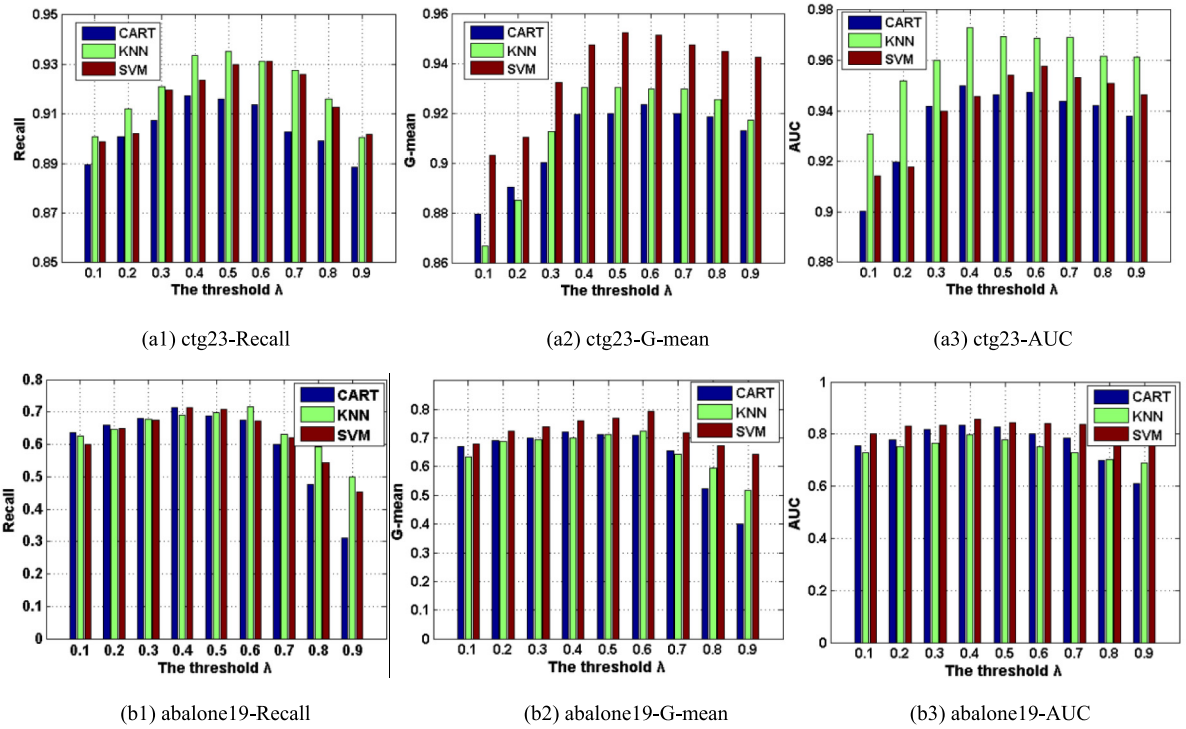
Fig. 6. The data distribution after noise filter by adopting five different λ thresholds, where each row corresponds a λ value, 0.1, 0.3, 0.5, 0.7, 0.9 (from top to down), and each column denotes a data set.

randomly selected the ctg23 dataset, which has a medium IR, and the abalone19 dataset which holds a very high IR, and designated λ to change from 0.1 to 0.9 with an increment of 0.1, to observe the varying trend of classification performance with the variance of λ. The results are shown in Fig. 7.

Fig. 7 shows that although there are some fluctuations, a common varying trend can still be observed, that is, the best classification performance generally corresponds to a medium λ value, whereas setting a too large or too small λ will decrease the modeling quality. Generally, designating λ in the range [0.4, 0.6] can achieve the best classification performance. The above observations verify our hypothesis, that is, a small λ may lose the details of the borderline and overlapping regions, whereas a large λ may aggravate the noise propagation.

Next, we focus on the impact of the other parameter M. To easily observe the impact of this parameter, we consider it to be a linear function associated with the class imbalance ratio IR, that is,

$$M = \delta \times \mathrm{IR} \tag{7}$$



(a1) ctg23-Recall (a2) ctg23-G-mean (a3) ctg23-AUC

(b1) abalone19-Recall (b2) abalone19-G-mean (b3) abalone19-AUC

Fig. 7. The varying trend of classification performance with the variance of the parameter λ.

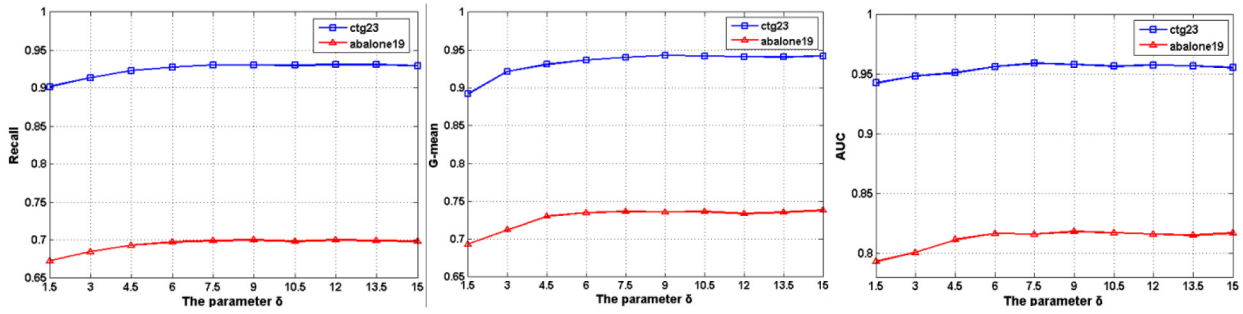


Fig. 8. The varying trend of classification performance with the variance of the parameter δ.

where δ is the multiple of IR to indicate how many times each majority instance would be extracted on average in UnderBagging-alike procedure. Here, we designate δ to vary from 1.5 to 15 in increments of 1.5. The experiments were conducted on the ctg23 and abalone19 datasets, and the average results across the three different classifiers are shown in Fig. 8.

Fig. 8 shows that with the increase in the ensemble size of the UnderBagging-alike algorithm, the classification performance acquires a rapid promotion and then stabilizes, which means that if we continue to increase the parameter, the classification performance cannot be continuously improved. The results indicate that when δ is designated as [6,9], an excellent classification performance can be guaranteed. To explore the reason, we consider that if M is designated too small a value, it is not enough to accurately estimate the confusion information and the data distribution, but it is also unnecessary to assign an overlarge value.

5.7. Comparison of the running time

Nevertheless, computation time should also be considered. Tables 15–17 show the running times of various comparative algorithms by adopting CART, KNN, and SVM as classifiers. The results show that the SMOTE and simple SMOTE variants, such as ADASYN, SN-SMOTE, SMMO,WK-SMOTE, and SMOTE-CSELM, are relatively time-saving, although they have conducted many neighborhood calculations, ensemble modeling, and optimization calculations. The three other complex SMOTE variants, SL-SMOTE, MWMOTE, and SMOTE-IPF, are slightly time-consuming. This is because SL-SMOTE is required to conduct much discriminant calculation, MWMOTE involves clustering analysis, and SMOTE-IPF introduces an iterative noise deduction procedure. Compared with other sampling algorithms, both the Supervised_SMOTE and our proposed IW-SMOTE algorithms require significantly more running time. For Supervised_SMOTE, we believe that it wastes much time to generate those illegal synthetic instances, whereas for our proposed IW-SMOTE algorithms, it must consume much time to train the UnderBagging ensemble model and calculate the confusion information of each instance. Additionally, we also note that on datasets with more instances, such as credit card clients, pageblocks2345, and letterA, or those datasets with higher IR, such as abalone19 and microRNA, our proposed IW-SMOTE algorithm is more time-consuming. The former requires more time to train each individual CART estimator, and the latter requires training many CART estimators to guarantee evaluative quality. Therefore, we can say that the proposed IW-SMOTE algorithm obtains excellent sampling quality by sacrificing running time.

Table 15
Running time (seconds) of twelve comparative algorithms on forty imbalanced data sets based on CART classifier.

Dataset	ORI	SMOTE	SL-SMOTE	SN-SMOTE	MWMOTE	ADASYN	SMOTE-IPF	SMMO	Supervised_SMOTE	WK-SMOTE	SMOTE-CSELM	IW-SMOTE
banknote	0.0007	0.0037	0.0100	0.1897	0.3448	0.0018	0.0771	0.0087	0.5558	0.0241	0.0066	5.2955
seeds2v13	0.0007	0.0021	0.0054	0.0254	0.0334	0.0041	0.0305	0.0065	0.3542	0.0162	0.0036	1.6527
ILPD	0.0013	0.0024	0.0152	0.0457	0.1992	0.0073	0.1105	0.0117	1.4876	0.0164	0.0053	2.3784
credit card clients	0.0028	0.1108	2.1879	0.8021	29.0284	0.4975	6.3648	0.3503	59.4336	0.4210	0.1115	69.4625
mfeatmor01	0.0011	0.0065	0.0431	0.1415	0.3165	0.0160	0.1209	0.0287	3.9435	0.0244	0.0095	5.7011
segment1	0.0012	0.0073	0.1464	0.1294	0.3739	0.0241	0.6198	0.0492	5.7476	0.0517	0.7602	7.6959
segmentation-grass	0.0009	0.0027	0.0085	0.0129	0.0301	0.0029	0.0366	0.0051	0.6591	0.0316	0.0040	1.6960
pageblocks2345	0.0015	0.0134	0.1980	0.2779	1.4166	0.0327	1.3938	0.1343	20.7635	0.0859	0.0221	16.8998
mfeatmor0	0.0013	0.0067	0.0516	0.0866	0.1945	0.0117	0.0810	0.0336	5.2525	0.0541	0.0102	5.5180
ctg23	0.0013	0.0057	0.1296	0.0801	0.2854	0.0141	0.4814	0.0507	8.3677	0.0527	0.0114	6.3422
ctg15	0.0016	0.0052	0.1419	0.0497	0.2056	0.0090	0.9022	0.0492	10.8369	0.0572	0.0114	5.9596
letterA	0.0041	0.0493	7.2514	0.3946	7.7161	0.1061	15.4384	0.4681	133.4758	0.1652	0.0878	61.3257
abalone19	0.0020	0.0096	0.1560	0.0815	0.2044	0.0145	1.3302	0.0956	401.2884	0.0301	1.4452	10.8154
ecoli0vs1	0.0012	0.0025	0.0036	0.0215	0.0350	0.0043	0.0223	0.0049	0.2602	0.0132	0.0063	1.3444
pima	0.0006	0.0029	0.0160	0.0546	0.3657	0.0109	0.1020	0.0122	0.9975	0.0128	0.3338	3.0317
iris0	0.0009	0.0016	0.0032	0.0163	0.0207	0.0024	0.0197	0.0029	0.2018	0.0115	0.0061	1.0963
glass0	0.0006	0.0026	0.0048	0.0214	0.0429	0.0037	0.0378	0.0055	0.3106	0.0090	0.0044	1.3212
yeast1	0.0001	0.0056	0.0498	0.1104	1.2161	0.0198	0.2218	0.0263	3.6316	0.0173	0.0095	6.0400
haberman	0.0006	0.0025	0.0063	0.0239	0.0660	0.0036	0.0332	0.0069	0.8968	0.0101	0.0045	1.4350
vehicle2	0.0013	0.0029	0.0315	0.0627	0.3030	0.0122	0.2264	0.0174	1.6351	0.0154	0.0069	3.8074
vehicle3	0.0010	0.0056	0.0361	0.0614	0.3775	0.0123	0.2932	0.0203	2.9978	0.0140	0.0070	4.1705
glass0123vs456	0.0007	0.0036	0.0056	0.0185	0.0300	0.0031	0.0344	0.0055	0.4156	0.0091	0.0044	1.3634
ecoli1	0.0008	0.0021	0.0084	0.0215	0.0485	0.0036	0.0384	0.0070	0.6666	0.0100	0.0048	1.5433
yeast3	0.0007	0.0061	0.0660	0.0656	0.2567	0.0134	0.3004	0.0293	5.0491	0.0254	0.6333	5.2080
ecoli3	0.0006	0.0034	0.0105	0.0123	0.0331	0.0039	0.0490	0.0091	1.1771	0.0132	0.0050	1.5426
yeast2vs4	0.0003	0.0049	0.0162	0.0178	0.0510	0.0060	0.0602	0.0111	1.8346	0.0201	0.0080	2.2247
yeast05679vs4	0.0002	0.0020	0.0194	0.0234	0.0644	0.0066	0.0792	0.0144	3.2975	0.0172	0.0081	2.4846
vowel0	0.0026	0.0045	0.0355	0.0512	0.1003	0.0077	0.2602	0.0261	3.7661	0.0152	0.0076	3.7896
glass016vs2	0.0009	0.0016	0.0066	0.0062	0.0227	0.0030	0.0434	0.0068	1.5052	0.0098	0.0047	1.2021
glass2	0.0011	0.0029	0.0075	0.0066	0.0245	0.0030	0.0446	0.0069	1.8390	0.0090	0.0048	1.2683
yeast1vs7	0.0013	0.0029	0.0145	0.0142	0.0405	0.0032	0.0751	0.0144	4.0880	0.0080	0.0056	1.9807
ecoli4	0.0008	0.0027	0.0102	0.0082	0.0285	0.0035	0.0364	0.0092	1.0938	0.0101	0.0053	1.6048
yeast1458vs7	0.0033	0.0032	0.0260	0.0183	0.0543	0.0034	0.1235	0.0156	17.3614	0.0153	0.0084	2.5806
yeast2vs8	0.0001	0.0023	0.0168	0.0134	0.0376	0.0042	0.0656	0.0128	2.8985	0.0172	0.0079	2.2679
yeast4	0.0004	0.0072	0.0637	0.0758	0.3983	0.0141	0.2723	0.0335	6.3244	0.0173	0.0085	5.4322
yeast5	0.0019	0.0048	0.0574	0.0384	0.1091	0.0099	0.2128	0.0329	6.1606	0.0201	0.0097	4.9794
yeast6	0.0017	0.0068	0.0620	0.0353	0.1064	0.0062	0.2777	0.0323	8.7404	0.0234	0.5770	4.9896
poker89vs5	0.0011	0.0069	0.0734	0.0395	0.1078	0.0099	0.5045	0.0443	35.3476	0.0261	0.0116	5.3248
poker8vs6	0.0012	0.0078	0.0635	0.0304	0.0990	0.0056	0.3597	0.0366	16.4739	0.0200	0.0098	4.1112
microRNA	0.0031	0.0229	1.6611	0.1735	2.0103	0.0561	14.5395	0.2768	77.7448	0.0921	0.0459	25.3202

Table 16
Running time (seconds) of twelve comparative algorithms on forty imbalanced data sets based on KNN classifier.

Dataset	ORI	SMOTE	SL-SMOTE	SN-SMOTE	MWMOTE	ADASYN	SMOTE-IPF	SMMO	Supervised_SMOTE	WK-SMOTE	SMOTE-CSELM	IW-SMOTE
banknote	0.0012	0.0033	0.0094	0.1730	0.3033	0.0011	0.0701	0.0090	0.5162	0.0241	0.0066	4.9153
seeds2v13	0.0010	0.0026	0.0053	0.0258	0.0341	0.0046	0.0317	0.0064	0.3473	0.0162	0.0036	1.6539
ILPD	0.0010	0.0029	0.0142	0.0424	0.1808	0.0060	0.1021	0.0101	1.3487	0.0164	0.0053	2.2018
credit card clients	0.0030	0.0996	1.9550	0.7251	26.5189	0.4630	5.9127	0.3165	52.8413	0.4210	0.1115	63.0326
mfeatmor01	0.0012	0.0049	0.0388	0.1235	0.2927	0.0136	0.1110	0.0266	3.6947	0.0244	0.0095	5.3973
segment1	0.0013	0.0071	0.1487	0.1308	0.4009	0.0246	0.6320	0.0492	5.8687	0.0517	0.7602	7.7635
segmentation-grass	0.0008	0.0022	0.0056	0.0110	0.0265	0.0037	0.0337	0.0041	0.6100	0.0316	0.0040	1.5216
pageblocks2345	0.0012	0.0137	0.2061	0.2825	1.4495	0.0324	1.4189	0.1379	21.1288	0.0859	0.0221	17.1134
mfeatmor0	0.0010	0.0043	0.0463	0.0791	0.1806	0.0108	0.0749	0.0313	4.8209	0.0541	0.0102	5.0951
ctg23	0.0011	0.0057	0.1191	0.0755	0.2657	0.0136	0.4629	0.0456	7.6911	0.0527	0.0114	5.8161
ctg15	0.0012	0.0055	0.1287	0.0455	0.1942	0.0088	0.8799	0.0433	10.0964	0.0572	0.0114	5.5136
letterA	0.0057	0.0430	7.1541	0.3894	7.6137	0.1028	15.0336	0.4544	130.0591	0.1652	0.0878	59.1522
abalone19	0.0013	0.0101	0.1559	0.0829	0.1995	0.0146	1.3377	0.0916	377.8557	0.0301	1.4452	10.5499
ecoli0vs1	0.0009	0.0022	0.0038	0.0179	0.0330	0.0033	0.0213	0.0055	0.2407	0.0132	0.0063	1.2315
pima	0.0010	0.0035	0.0145	0.0477	0.3205	0.0089	0.0920	0.0109	0.9168	0.0128	0.3338	2.7908
iris0	0.0006	0.0018	0.0034	0.0159	0.0195	0.0023	0.0186	0.0036	0.1951	0.0115	0.0061	1.0065
glass0	0.0008	0.0022	0.0044	0.0196	0.0374	0.0035	0.0340	0.0059	0.2941	0.0090	0.0044	1.2122
yeast1	0.0016	0.0068	0.0498	0.1062	1.2299	0.0205	0.2303	0.0305	3.7242	0.0173	0.0095	6.1594
haberman	0.0010	0.0024	0.0065	0.0222	0.0595	0.0035	0.0301	0.0066	0.8396	0.0101	0.0045	1.3495
vehicle2	0.0012	0.0039	0.0344	0.0664	0.3294	0.0109	0.2324	0.0186	1.7071	0.0154	0.0069	4.0431
vehicle3	0.0006	0.0061	0.0363	0.0653	0.4031	0.0095	0.3087	0.0188	3.1446	0.0140	0.0070	4.3510
glass0123vs456	0.0008	0.0021	0.0052	0.0163	0.0264	0.0035	0.0309	0.0061	0.3749	0.0091	0.0044	1.1952
ecoli1	0.0008	0.0025	0.0076	0.0191	0.0449	0.0037	0.0371	0.0069	0.6212	0.0100	0.0048	1.4198
yeast3	0.0017	0.0020	0.0631	0.0688	0.2586	0.0091	0.3076	0.0312	4.9920	0.0254	0.6333	5.2981
ecoli3	0.0009	0.0031	0.0091	0.0116	0.0303	0.0034	0.0453	0.0081	1.0888	0.0132	0.0050	1.4276
yeast2vs4	0.0029	0.0052	0.0150	0.0220	0.0504	0.0047	0.0613	0.0135	1.9650	0.0201	0.0080	2.4036
yeast05679vs4	0.0020	0.0052	0.0193	0.0215	0.0603	0.0077	0.0773	0.0133	3.2701	0.0172	0.0081	2.4823
vowel0	0.0000	0.0059	0.0338	0.0530	0.1025	0.0078	0.2696	0.0301	3.9196	0.0152	0.0076	3.8476
glass016vs2	0.0007	0.0025	0.0062	0.0058	0.0207	0.0030	0.0399	0.0063	1.4941	0.0098	0.0047	1.1260
glass2	0.0009	0.0024	0.0063	0.0057	0.0219	0.0030	0.0411	0.0066	1.7529	0.0090	0.0048	1.1706
yeast1vs7	0.0000	0.0027	0.0166	0.0173	0.0497	0.0047	0.0803	0.0133	4.7659	0.0080	0.0056	2.1843
ecoli4	0.0010	0.0027	0.0098	0.0088	0.0257	0.0035	0.0339	0.0087	1.0377	0.0101	0.0053	1.5367
yeast1458vs7	0.0002	0.0048	0.0295	0.0212	0.0626	0.0116	0.1272	0.0159	18.9955	0.0153	0.0084	2.7014
yeast2vs8	0.0000	0.0016	0.0168	0.0110	0.0388	0.0016	0.0634	0.0122	2.7426	0.0172	0.0079	2.2886
yeast4	0.0019	0.0057	0.0589	0.0760	0.3975	0.0188	0.2789	0.0279	6.4819	0.0173	0.0085	5.4750
yeast5	0.0000	0.0072	0.0699	0.0431	0.1187	0.0026	0.2400	0.0279	6.7092	0.0201	0.0097	5.4492
yeast6	0.0000	0.0063	0.0614	0.0346	0.1113	0.0092	0.2884	0.0237	9.5062	0.0234	0.5770	5.2866
poker89vs5	0.0025	0.0047	0.0674	0.0381	0.0998	0.0085	0.4959	0.0380	32.9590	0.0261	0.0116	5.1386
poker8vs6	0.0004	0.0049	0.0628	0.0323	0.1018	0.0100	0.3704	0.0346	17.2133	0.0200	0.0098	4.1728
microRNA	0.0031	0.0214	1.5822	0.1777	1.9212	0.0529	14.7692	0.2828	80.1725	0.0921	0.0459	25.6180

6. Concluding remarks

In this study, we focus on the issue of accurately estimating prior data distribution information, and propose an indirect distribution exploration strategy based on the idea of an undersampling ensemble. Furthermore, we fuse the distribution information into the SMOTE sampling procedure and proposed a novel SMOTE variant called the IW-SMOTE algorithm. Many experimental results demonstrate the merits of the proposed IW-SMOTE algorithm as follows:

(1) It contains an accurate, robust and universal data-distribution exploration ability that can adapt to various data distributions.
(2) It has an effective and efficient noise-filtering mechanism that can remove real noisy instances to prevent noise propagation.
(3) It implements personalized oversampling that could highlight the real classification boundaries, and simultaneously maintain a safe region.

In future work, we will focus on searching for more efficient and robust data-distribution exploration strategies to improve

Table 17
Running time (seconds) of twelve comparative algorithms on forty imbalanced data sets based on SVM classifier.

Dataset	ORI	SMOTE	SL-SMOTE	SN-SMOTE	MWMOTE	ADASYN	SMOTE-IPF	SMMO	Supervised_SMOTE	WK-SMOTE	SMOTE-CSELM	IW-SMOTE
banknote	0.0010	0.0055	0.0134	0.1990	0.3669	0.0005	0.0811	0.0106	0.5998	0.0241	0.0066	5.6550
seeds2v13	0.0009	0.0022	0.0038	0.0226	0.0304	0.0034	0.0288	0.0051	0.3098	0.0162	0.0036	1.4576
ILPD	0.0018	0.0031	0.0200	0.0556	0.2359	0.0103	0.1229	0.0119	1.6939	0.0164	0.0053	2.7287
credit_card_clients	0.0034	0.0963	1.8505	0.7155	25.7508	0.4424	5.7025	0.3036	51.4815	0.4210	0.1115	60.8008
mfeatmor01	0.0011	0.0051	0.0478	0.1586	0.3871	0.0173	0.1434	0.0342	4.7647	0.0244	0.0095	6.8468
segment1	0.0011	0.0062	0.1298	0.1153	0.3364	0.0212	0.5395	0.0434	5.0117	0.0517	0.7602	6.5765
segmentation-grass	0.0005	0.0043	0.0057	0.0109	0.0242	0.0036	0.0331	0.0037	0.5667	0.0316	0.0040	1.4694
pageblocks2345	0.0014	0.0119	0.1628	0.2191	1.2378	0.0271	1.2213	0.1152	17.9658	0.0859	0.0221	14.5259
mfeatmor0	0.0008	0.0059	0.0563	0.1016	0.2204	0.0143	0.0921	0.0390	6.1194	0.0541	0.0102	6.3545
ctg23	0.0023	0.0061	0.1356	0.0905	0.3115	0.0150	0.5022	0.0545	9.0197	0.0527	0.0114	6.6982
ctg15	0.0018	0.0050	0.1464	0.0533	0.2174	0.0106	0.9693	0.0480	11.6495	0.0572	0.0114	6.5097
letterA	0.0038	0.0413	6.7077	0.3460	6.9710	0.0963	13.3696	0.4166	117.7982	0.1652	0.0878	54.4966
abalone19	0.0015	0.0098	0.1389	0.0761	0.1928	0.0151	1.2499	0.0870	365.0296	0.0301	1.4452	9.9523
ecoli0vs1	0.0009	0.0024	0.0044	0.0208	0.0367	0.0032	0.0227	0.0048	0.2704	0.0132	0.0063	1.4290
pima	0.0020	0.0048	0.0196	0.0599	0.4274	0.0111	0.1155	0.0137	1.1721	0.0128	0.3338	3.4463
iris0	0.0001	0.0030	0.0036	0.0197	0.0262	0.0020	0.0207	0.0027	0.2278	0.0115	0.0061	1.2554
glass0	0.0017	0.0006	0.0045	0.0248	0.0501	0.0031	0.0423	0.0063	0.3722	0.0090	0.0044	1.5986
yeast1	0.0018	0.0057	0.0415	0.0858	0.9795	0.0173	0.1878	0.0202	2.9883	0.0173	0.0095	4.9463
haberman	0.0014	0.0046	0.0110	0.0302	0.0794	0.0062	0.0411	0.0111	1.0769	0.0101	0.0045	1.7248
vehicle2	0.0004	0.0031	0.0332	0.0630	0.3059	0.0109	0.2208	0.0167	1.6073	0.0154	0.0069	3.8140
vehicle3	0.0008	0.0047	0.0359	0.0603	0.3722	0.0108	0.2925	0.0187	3.0027	0.0140	0.0070	4.1782
glass0123vs456	0.0004	0.0022	0.0059	0.0196	0.0306	0.0030	0.0363	0.0072	0.4416	0.0091	0.0044	1.4188
ecoli1	0.0008	0.0023	0.0088	0.0235	0.0557	0.0059	0.0428	0.0085	0.7272	0.0100	0.0048	1.6838
yeast3	0.0010	0.0043	0.0511	0.0510	0.1995	0.0100	0.2538	0.0246	4.0236	0.0254	0.6333	4.2398
ecoli3	0.0006	0.0019	0.0110	0.0151	0.0361	0.0036	0.0544	0.0092	1.2567	0.0132	0.0050	1.6917
yeast2vs4	0.0010	0.0027	0.0136	0.0197	0.0478	0.0055	0.0527	0.0109	1.5744	0.0201	0.0080	1.9557
yeast05679vs4	0.0020	0.0041	0.0182	0.0242	0.0578	0.0063	0.0746	0.0136	2.8769	0.0172	0.0081	2.1971
vowel0	0.0022	0.0037	0.0343	0.0515	0.1007	0.0083	0.2682	0.0237	3.9029	0.0152	0.0076	3.8224
glass016vs2	0.0008	0.0021	0.0073	0.0052	0.0241	0.0029	0.0452	0.0084	1.7841	0.0098	0.0047	1.3508
glass2	0.0008	0.0018	0.0089	0.0054	0.0288	0.0038	0.0525	0.0070	2.1009	0.0090	0.0048	1.5271
yeast1vs7	0.0008	0.0027	0.0153	0.0133	0.0385	0.0032	0.0671	0.0099	3.7615	0.0080	0.0056	1.8011
ecoli4	0.0007	0.0021	0.0100	0.0087	0.0297	0.0040	0.0390	0.0088	1.1794	0.0101	0.0053	1.7404
yeast1458vs7	0.0014	0.0029	0.0233	0.0174	0.0480	0.0055	0.1130	0.0153	15.8139	0.0153	0.0084	2.2939
yeast2vs8	0.0006	0.0032	0.0157	0.0111	0.0356	0.0037	0.0573	0.0116	2.4296	0.0172	0.0079	1.9228
yeast4	0.0008	0.0052	0.0480	0.0597	0.3214	0.0109	0.2287	0.0238	5.1620	0.0173	0.0085	4.4142
yeast5	0.0019	0.0040	0.0510	0.0335	0.0964	0.0083	0.1943	0.0278	5.3324	0.0201	0.0097	4.3031
yeast6	0.0003	0.0026	0.0541	0.0302	0.0886	0.0072	0.2417	0.0285	7.6461	0.0234	0.5770	4.2006
poker89vs5	0.0021	0.0063	0.0846	0.0456	0.1263	0.0105	0.5781	0.0475	40.0469	0.0261	0.0116	5.9194
poker8vs6	0.0008	0.0044	0.0612	0.0292	0.0973	0.0085	0.3458	0.0317	14.4131	0.0200	0.0098	3.8781
microRNA	0.0028	0.0200	1.4306	0.1587	1.7574	0.0454	13.0188	0.2484	70.2926	0.0921	0.0459	22.4918

the performance of CIL algorithms. Additionally, the use of the proposed data distribution exploration strategy to improve other CIL techniques will be investigated.

CRediT authorship contribution statement

Aimin Zhang: Conceptualization, Data curation, Investigation, Codes, Experiments, Writing – original draft. **Hualong Yu:** Methods, Resources, Writing – review & editing, Funding acquisition, Supervision. **Shanlin Zhou:** Data analysis, Methods. **Zhangjun Huan:** Experiments, Data analysis. **Xibei Yang:** Formal analysis, Writing – review & editing, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The datasets used in this paper were collected from theopenly UCI machine learning repository and Keel data repository (http://archive.ics.uci.edu/ml/datasets.php and https://sci2s.ugr.es/keel/datasets.php). The Python codes of the proposed IW-SMOTE algorithm can be downloaded from https://github.com/Jung-Alen/IW-SMOTE.

Acknowledgments

The work was supported by Natural Science Foundation of Jiangsu Province of China under grant No. BK20191457, National Natural Science Foundation of China under grants No. 62176107, No. 62076111 and No. 62076215.

References

[1] C. Huang, X. Huang, Y. Fang, J. Xu, Y. Qu, P. Zhai, et al., Sample imbalance disease classification model based on association rule feature selection, Pattern Recognit. Lett. 133 (2020) 280–286.
[2] S. Fotouhi, S. Asadi, M.W. Kattan, A comprehensive data level analysis for cancer diagnosis on imbalanced data, J. Biomed. Inform. 90 (2019) 103089.
[3] R. Malhotra, S. Kamal, An empirical study to investigate oversampling methods for improving software defect prediction using imbalanced data, Neurocomputing 343 (2019) 120–140.
[4] K.E. Bennin, J. Keung, P. Phannachitta, A. Monden, S. Mensah, MAHAKIL: Diversity based oversampling approach to alleviate the class imbalance issue in software defect prediction, IEEE Trans. Softw. Eng. 44 (2018) 534–550.
[5] Y. Qian, S. Ye, Y. Zhang, J. Zhang, SUMO-Forest: A cascade forest based method for the prediction of SUMOylation sites on imbalanced data, Gene. 741 (2020) 144536.
[6] H.-J. Dai, C.-K. Wang, Classifying adverse drug reactions from imbalanced twitter data, Int. J. Med. Inform. 129 (2019) 122–132.
[7] L. Wang, Y. Chen, H. Jiang, J. Yao, Imbalanced credit risk evaluation based on multiple sampling, multiple kernel fuzzy self-organizing map and local accuracy ensemble, Appl. Soft Comput. 91 (2020) 106262.
[8] P.D. Mahajan, A. Maurya, A. Megahed, A. Elwany, R. Strong, J. Blomberg, Optimizing predictive precision in imbalanced datasets for actionable revenue change prediction, European J. Oper. Res. 285 (2020) 1095–1113.
[9] D. Veganzones, E. Séverin, An investigation of bankruptcy prediction in imbalanced datasets, Decis. Support Syst. 112 (2018) 111–124.
[10] W. Qian, S. Li, A novel class imbalance-robust network for bearing fault diagnosis utilizing raw vibration signals, Measurement 156 (2020) 107567.
[11] G. Chen, Z. Ge, Svm-tree and svm-forest algorithms for imbalanced fault classification in industrial processes, IFAC J. Syst. Control 8 (2019) 100052.
[12] A. Sharififar, F. Sarmadian, B. Minasny, Mapping imbalanced soil classes using Markov chain random fields models treated with data Resampling technique, Comput. Electron. Agric. 159 (2019) 110–118.
[13] A. Sharififar, F. Sarmadian, B.P. Malone, B. Minasny, Addressing the issue of digital mapping of soil classes with imbalanced class observations, Geoderma 350 (2019) 84–92.
[14] H. Jeong, Y. Jang, P.J. Bowman, N. Masoud, Classification of motor vehicle crash injury severity: A hybrid approach for imbalanced data, Accid. Anal. Prev. 120 (2018) 250–261.
[15] Y.-S. Li, H. Chi, X.-Y. Shao, M.-L. Qi, B.-G. Xu, A novel random forest approach for imbalance problem in crime linkage, Knowl.-Based Syst. 195 (2020) 105738.
[16] B. Krawczyk, Learning from imbalanced data: Open challenges and future directions, Prog. Artif. Intell. 5 (2016) 221–232.
[17] Haibo. He, E.A. Garcia, Learning from imbalanced data, IEEE Trans. Knowl. Data Eng. 21 (2009) 1263–1284.

[18] P. Branco, L. Torgo, R.P. Ribeiro, A survey of predictive modeling on imbalanced domains, ACM Comput. Surv. 49 (2016) 1–50.
[19] G. Haixiang, L. Yijing, J. Shang, G. Mingyun, H. Yuanyue, G. Bing, Learning from class-imbalanced data: Review of methods and applications, Expert Syst. Appl. 73 (2017) 220–239.
[20] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, Smote: Synthetic minority over-sampling technique, J. Artif. Intell. Res. 16 (2002) 321–357.
[21] G. Kovács, An empirical comparison and evaluation of minority oversampling techniques on a large number of imbalanced datasets, Appl. Soft Comput. 83 (2019) 105662.
[22] H. He, Y. Bai, E.A. Garcia, S. Li, Adasyn: Adaptive synthetic sampling approach for imbalanced learning, in: 2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence), 2008, pp. 1322–1328.
[23] H. Yu, J. Ni, J. Zhao, ACOSampling: AN ant colony optimization-based undersampling method for CLASSIFYING imbalanced DNA microarray data, Neurocomputing 101 (2013) 309–318.
[24] W.W. Ng, J. Hu, D.S. Yeung, S. Yin, F. Roli, Diversified sensitivity-based undersampling for imbalance classification problems, IEEE Trans. Cybern. 45 (2015) 2402–2412.
[25] G.E. Batista, R.C. Prati, M.C. Monard, A study of the behavior of several methods for balancing machine learning training data, ACM SIGKDD Explor. Newsl. 6 (2004) 20–29.
[26] J.A. Sáez, J. Luengo, J. Stefanowski, F. Herrera, SMOTE–IPF: ADdressing the noisy and borderline examples problem in imbalanced classification by A re-sampling method with filtering, Inform. Sci. 291 (2015) 184–203.
[27] R. Batuwita, V. Palade, FSVM-CIL: FUzzy support vector machines for class imbalance learning, IEEE Trans. Fuzzy Syst. 18 (2010) 558–571.
[28] H. Yu, C. Sun, X. Yang, S. Zheng, H. Zou, Fuzzy support vector machine with relative density information for classifying imbalanced data, IEEE Trans. Fuzzy Syst. 27 (2019) 2353–2367.
[29] H. Yu, C. Sun, X. Yang, W. Yang, J. Shen, Y. Qi, ODOC-ELM: OPtimal decision outputs compensation-based extreme learning machine for classifying imbalanced data, Knowl.-Based Syst. 92 (2016) 55–70.
[30] H. Yu, C. Mu, C. Sun, W. Yang, X. Yang, X. Zuo, Support vector machine-based optimized decision threshold adjustment strategy for classifying imbalanced data, Knowl.-Based Syst. 76 (2015) 67–78.
[31] Z. Wang, B. Wang, Y. Cheng, D. Li, J. Zhang, Cost-sensitive fuzzy multiple kernel learning for imbalanced problem, Neurocomputing 366 (2019) 178–193.
[32] H. Yu, D. Sun, X. Xi, X. Yang, S. Zheng, Q. Wang, Fuzzy one-class extreme auto-encoder, Neural Process. Lett. 50 (2018) 701–727.
[33] S. Wang, X. Yao, Diversity analysis on imbalanced data sets by using ensemble models, in: 2009 IEEE Symposium on Computational Intelligence and Data Mining, 2009, pp. 324–331.
[34] P. Lim, C.K. Goh, K.C. Tan, Evolutionary cluster-based synthetic oversampling ensemble (eco-ensemble) for imbalance learning, IEEE Trans. Cybern. 47 (2017) 2850–2861.
[35] Z. Sun, Q. Song, X. Zhu, H. Sun, B. Xu, Y. Zhou, A novel ensemble method for classifying imbalanced data, Pattern Recognit. 48 (2015) 1623–1637.
[36] J.F. Díez-Pastor, J.J. Rodríguez, C.I. García-Osorio, L.I. Kuncheva, Diversity techniques improve the performance of the best imbalance learning ensembles, Inform. Sci. 325 (2015) 98–117.
[37] B. Tang, H. He, Gir-based ensemble sampling approaches for imbalanced learning, Pattern Recognit. 71 (2017) 306–319.
[38] X. Zhang, Y. Zhuang, W. Wang, W. Pedrycz, Transfer boosting with synthetic instances for class imbalanced object recognition, IEEE Trans. Cybern. 48 (2018) 357–370.
[39] C. Bunkhumpornpat, K. Sinapiromsaran, C. Lursinsap, Safe-level-smote: Safe-level-synthetic minority over-sampling technique for handling the class imbalanced problem, Adv. Knowl. Discov. Data Min. (2009) 475–482.
[40] S. Barua, M.M. Islam, X. Yao, K. Murase, MWMOTE-Majority weighted minority oversampling technique for imbalanced data set learning, IEEE Trans. Knowl. Data Eng. 26 (2014) 405–425.
[41] V. García, J.S. Sánchez, R. Martín-Félez, R.A. Mollineda, Surrounding neighborhood-based SMOTE for learning from imbalanced data sets, Prog. Artif. Intell. 1 (2012) 347–362.
[42] J. Mathew, C.K. Pang, M. Luo, W.H. Leong, Classification of imbalanced data by oversampling in kernel space of support vector machines, IEEE Trans. Neural Netw. Learn. Syst. 29 (2018) 4065–4076.
[43] B.S. Raghuwanshi, S. Shukla, Smote based class-specific extreme learning machine for imbalanced learning, Knowl.-Based Syst. 287 (2020) 104814.
[44] A. Fernandez, S. Garcia, F. Herrera, N.V. Chawla, SMOTE For learning from imbalanced DATA: Progress and challenges, marking the 15-year anniversary, J. Artif. Intell. Res. 61 (2018) 863–905.
[45] M. Li, A. Xiong, L. Wang, S. Deng, J. Ye, ACO Resampling: Enhancing the performance of oversampling methods for class imbalance classification, Knowl.-Based Syst. 196 (2020) 105818.
[46] B.S. Raghuwanshi, S. Shukla, Class-specific kernelized extreme learning machine for binary class imbalance learning, Appl. Soft Comput. 73 (2018) 1026–1038.
[47] J. Hu, X. He, D.J. Yu, X. Yang, J. Yang, H. Shen, A new supervised over-sampling algorithm with application to protein-nucleotide binding residue prediction, PLoS One 9 (2014) e107676.
[48] J. Calleja, O. Fuentes, J. Gonzalez, Selecting minority examples from misclassified data for over-sampling, in: Proceedings of the Twenty-First International Florida Artificial Intelligence Research Society Conference, 2008, pp. 276-281.
[49] V. López, A. Fernández, S. García, V. Palade, F. Herrera, An insight into classification with imbalanced data: Empirical results and current trends on using data intrinsic characteristics, Inform. Sci. 250 (2013) 113–141.
[50] H. Zhang, H. Zhang, S. Pirbhulal, W. Wu, V.H. Albuquerque, Active balancing mechanism for imbalanced medical data in deep learning–based classification models, ACM Trans. Multimedia Comput. Commun. Appl. 16 (2020) 1–15.
[51] A.D. Gordon, L. Breiman, J.H. Friedman, R.A. Olshen, C.J. Stone, Classification and regression trees, Biometrics 40 (1984) 874.
[52] C. Blake, E. Keogh, C.J. Merz, UCI Repository of Machine Learning Databases, Technical Report 213, Department of Information and Computer Science, University of California, Irvine, CA, 1998.
[53] I. Triguero, S. González, J.M. Moyano, S. García, J. Alcalá-Fdez, J. Luengo, et al., KEEL 3.0: AN open source software for multi-stage analysis in data mining, Int. J. Comput. Intell. Syst. 10 (2017) 1238.
[54] C. Xue, F. Li, T. He, G.-P. Liu, Y. Li, X. Zhang, Classification of real and pseudo microrna precursors using local structure-sequence features and support vector machine, BMC Bioinform.. 6 (2005) 310.
[55] G. Kovács, Smote-variants: A Python implementation of 85 Minority oversampling techniques, Neurocomputing 366 (2019) 352–354.
[56] H. Yu, J. Ni, S. Xu, B. Qin, H. Jv, Estimating harmfulness of class imbalance by scatter matrix based class separability measure, Intell. Data Anal. 18 (2014) 203–216.
[57] J. Demsar, Statistical comparisons of classifiers over multiple data sets, J. Mach. Learn. Res. 7 (2006) 1–30.
[58] S. García, A. Fernández, J. Luengo, F. Herrera, Advanced nonparametric tests for multiple comparisons in the design of experiments in computational intelligence and data mining: Experimental analysis of power, Inform. Sci. 180 (2010) 2044–2064.